
MICROGATE SERIAL COMMUNICATIONS SOFTWARE DEVELOPMENT KIT WINDOWS GUIDE

MicroGate Systems, Ltd

<http://www.microgate.com>

MicroGate® and SyncLink® are registered trademarks of MicroGate Systems, Ltd.
Copyright © 2012-2025 MicroGate Systems, Ltd. All Rights Reserved

CONTENTS

Overview.....	5
Software Installation	6
Hardware Installation	7
PCI Cards	7
USB Adapter.....	7
Cables	8
Verifying Installation.....	9
Windows Device Manager	9
SyncLink Device Properties	10
Trace Utility	13
Device Instances and Names	14
Location Locked and Device Locked Instances	14
Serial API Names	14
Removing Device Instances of Non-present Hardware	15
Serial API Programming	16
C/C++	17
Open/Close Device	17
Configure Device.....	18
Receiving Data	19
Sending Data.....	21
Waiting for All Data Sent	22
Asynchronous API Notification	23
Functions	23
Structures	53
C#.....	62
Port Object.....	63
Open/Close	63
Configuration.....	64
Send Data.....	65
Receive Data	66
Class Reference.....	67
Python.....	82
Port Object.....	83

Open/Close	83
Configuration	84
Send Data.....	85
Receive Data	86
Class Reference.....	87
Protocols.....	102
HDLC/SDLC.....	103
Transmit.....	103
Receive.....	104
Asynchronous	105
Isochronous	105
Raw Synchronous	106
Transmit.....	106
Receive.....	107
Monosync and Bisync	108
Transmit.....	108
Receive.....	109
Extended Sync (XSync)	110
Transmit.....	110
Receive.....	111
Time Division Multiplexing (TDM)	112
Transmit.....	113
Receive.....	114
TDM Signal Mapping.....	114
Encoding	115
Serial Clocks	116
Base Clock and Frequency Synthesizer	116
Baud Rate Generator	117
Clock Recovery.....	117
Asynchronous Protocol.....	118
Isochronous Protocol.....	118
Clock Examples	119
Synchronous, DCE Clocks.....	119
Synchronous, DCE Receive Clock, DTE Transmit Clock	119

Synchronous, Clock Recovery	120
Asynchronous	121
Isochronous, DCE Clocks	122
Isochronous, DCE Receive Clock, DTE Transmit Clock	122
Windows Communication API (COM Port Mode)	123

OVERVIEW

This guide describes developing with SyncLink serial communication devices and the Windows operating system.

Use the guide in the order:

Software Installation
Hardware Installation
Verifying Installation
Serial API Programming

Other sections cover general serial communication topics.

Supported Windows Versions

Windows Server and Windows 11 are 64-bit only. Windows 10 has 32-bit and 64-bit support.

Windows 10, Server 2016 to 2022

Windows 11, Server 2025

Legacy driver packages available for earlier Windows OS, contact support for details.

Required Developer Knowledge

Developing with SyncLink devices on Windows **requires** the following knowledge:

1. A supported language: C, C++, C# or Python
2. Basic Windows administration
3. Serial communication details for target application
4. Reading MicroGate documentation

MicroGate offers paid consulting and development services for projects where this knowledge is absent. Contact MicroGate for details.

SOFTWARE INSTALLATION

Before using or installing SyncLink hardware, install the device drivers. The latest drivers can be downloaded from www.microgate.com

`microgate-sdk-windows.zip` contains the SDK and drivers. It expands to a directory `mgapi`.

Development Files

These files are required to develop applications and are not required for application use.

<code>readme.txt</code>	General information
<code>serial-api-windows.pdf</code>	This document
<code>c</code>	C/C++ development files
<code>csharp</code>	C# development files
<code>python</code>	Python development files

Run Time Files

These files may be distributed with the application and hardware.

<code>drivers\win32</code>	drivers for 32-bit
<code>drivers\win64</code>	drivers for 64-bit
<code>tools\win32</code>	32-bit support tools
<code>tools\win64</code>	64-bit support tools

Run `setup.exe` in the `drivers` directory as Administrator. This installs drivers so Windows can automatically detect and support SyncLink hardware. Setup updates previously installed devices to the latest drivers.

```
C:\mgapi\drivers\>setup
```

After software installation, install hardware as described in the next section. Windows detects the hardware and installs drivers for each device. If Windows does not find the drivers automatically, manually specify the search location as the above directories and follow the displayed instructions to complete device installation.

Software Removal

Run `setup.exe` in the appropriate directory with the `/u` option to remove all device instances and the driver packages.

```
C:\mgapi\drivers>setup /u
```

HARDWARE INSTALLATION

This section describes the configuration and installation of the serial hardware. Configuration must match application requirements.

Each SyncLink port must be configured for one of three electrical specifications using jumpers or DIP switches on PCI cards and software for USB.

RS-232/V.28	single ended, unbalanced signals
V.35	differential data/clock signals and single ended control/status signals
RS-422/RS-485/V.11	differential signals

Refer to the hardware guide (PDF) for details. Hardware guides are available at www.microgate.com

PCI CARDS

PCI and PCI Express cards are installed in expansion slots on a host system. The card and slot type must match. SyncLink PCI cards are compatible with 3.3V, 5V, 32-bit, 64-bit and PCI-X expansion slots. PCI-X and PCI Express are different slot types. SyncLink PCI Express cards are compatible with 1x, 4x, and 16x PCI Express expansion slots.

- Verify interface selection (RS232,V.35,RS422) for cards with jumpers or switch settings.
- Shutdown system.
- Remove system case cover.
- Insert adapter in compatible slot.
- Secure card bracket with screw or clamp.
- Replace system case cover.
- Start system.

USB ADAPTER

The USB serial adapter plugs into a host USB port using the supplied Type B male to Type A male USB cable.

SyncLink USB should be plugged into a USB 2.0 or later Hi-speed (480Mbps) USB port. Operating on a slower USB port is not recommended. Install directly into a host USB port instead of a USB hub for better performance.

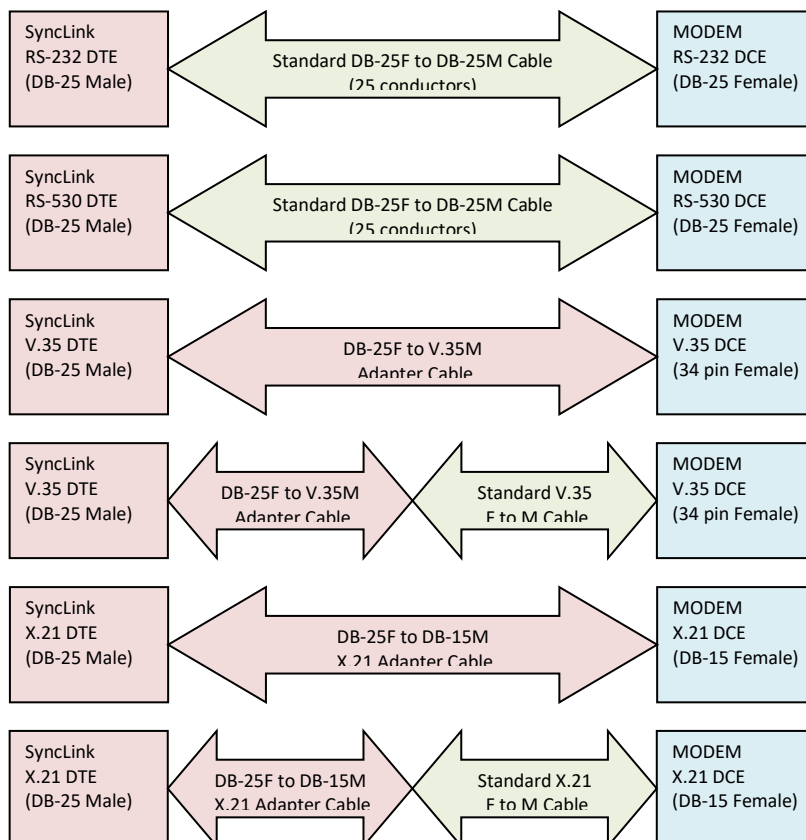
SyncLink USB requires 500mA of power from the USB port, which is standard and supported by most USB ports. Some USB ports may not provide a full 500mA, such as unpowered hubs or ports in small mobile devices.

After insertion and driver installation select the SyncLink USB interface type in the [Windows Device Manager](#).

CABLES

Serial devices are DTE (data terminal equipment) or DCE (data circuit-terminating equipment). A DTE connects directly to a DCE. Two DTEs connect with a cross over cable or null MODEM. A DTE sends and receives data. A DCE converts data to a signal suitable for links like a phone line or radio. SyncLink devices are DTE with a DB25 male connector.

The following diagram shows where standard cables (green) are used and where adapter cables (red) must be purchased from MicroGate to convert the SyncLink DB25 connector to the DCE connector.



If the attached device does not use any of the above connectors, consult documentation for the SyncLink and attached devices to create a custom cable. Pinouts, electrical specification and configuration options are contained in the hardware user's manual (PDF) for your SyncLink device. Pay close attention to differential signal polarity.

VERIFYING INSTALLATION

Before developing an application, verify the correct installation of serial hardware and device drivers.

WINDOWS DEVICE MANAGER

The primary tool for verification is the Windows Device Manager, an administrative tool included with Windows. This tool displays a tree diagram of hardware devices arranged by type or connection.

The Windows device manager can be started from a command prompt.

To open a command prompt, click the **Start** button, select **All Programs** then **Accessories** and right click on **Command Prompt** and select **Run as administrator** from the pop-up menu. If prompted for permission to continue, select allow. In the command prompt, run the following command:

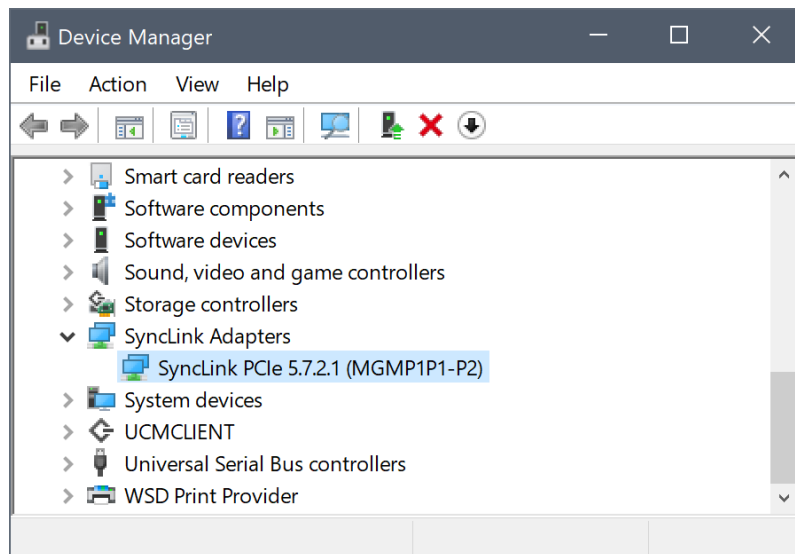
```
C:\>devmgmt.msc
```

Once the device manager is running, look for the branch labeled **SyncLink Adapters**.

If **SyncLink Adapters** is not present, select the **Computer** branch, click the **Action** menu and select **Scan for hardware changes**. Windows will detect new hardware and install drivers.

If **SyncLink Adapters** is still not present, look for entries with a yellow question mark symbol labeled either **PCI Simple Communications Controller** or **SyncLink USB**. Right click these entries, select **Update Driver** and follow any displayed instructions. If Windows does not find the drivers automatically, manually specify the search location as the driver directory in the SDK package.

Once the **SyncLink Adapters** branch is present, expand it to display SyncLink devices. Devices with a yellow symbol on the icon have a problem. If no yellow symbol is visible, the device and driver have been correctly installed. Right click on a SyncLink device and select **Properties**.

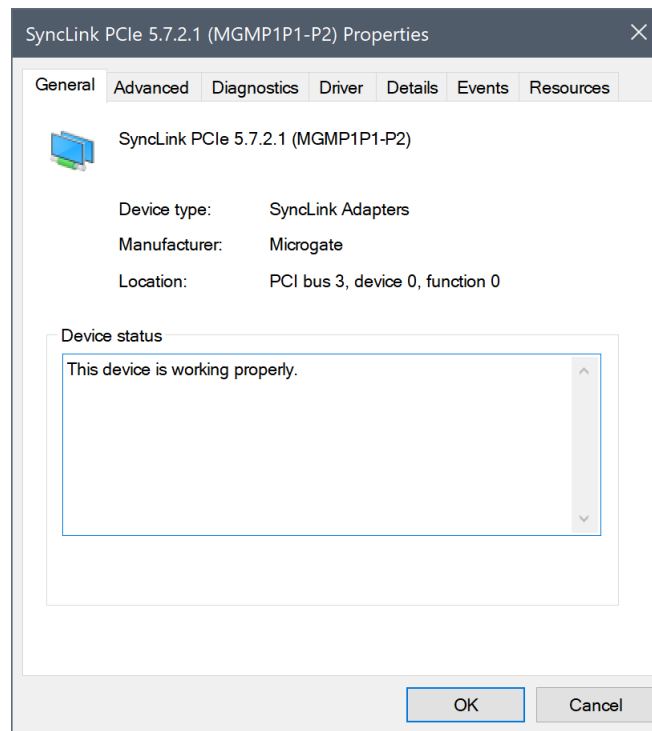


SYNCLINK DEVICE PROPERTIES

The SyncLink device properties in the Windows device manager displays device and driver information, configures the device and allows testing the device. The device properties window has multiple tabs for different purposes.

GENERAL TAB

This tab displays the device status and location (slot or port number). If the device status is working properly then proceed to the next tab. Otherwise note the error message for diagnosing an installation problem. The device driver version and device names are displayed to the right of the card icon near the top.



ADVANCED TAB

These settings are applied at system start or device insertion. Some settings apply only to select device types.

The screenshot shows the 'Advanced' tab of the 'SyncLink PCIe 5.7.2.1 (MGMP1P1-P2) Properties' dialog box. The 'Device' dropdown is set to 'MGMP1P1'. The 'Serial Interface' dropdown is set to 'RS422(RS530/X.21)'. There are four checkboxes: 'Disable Input Termination' (unchecked), 'Disable RS422/485 outputs when RTS is off.' (unchecked), 'Disable RS422/485 outputs when not sending data.' (unchecked), and 'DMA Buffer Size in Bytes (4 - 4096, 0=use default)' (set to 0). The 'DMA Buffer Count (0=use default)' is also set to 0. The 'Max Frame Size in Bytes (4096 - 65535)' is set to 4096. The 'OK' and 'Cancel' buttons are at the bottom right.

The **Device** pull down list has an entry for each port on the device. Other settings apply to the selected port.

Max Frame Size

The maximum data size sent or received in a single API call. Default:4096.

Serial Interface (SyncLink USB and SyncLink PCIe Only)

Select interface electrical specification (RS232, V.35, RS422, etc). GT cards use jumper selection. Selection **must** match application requirements for correct operation and to prevent damaging the device.

Disable Input Termination (SyncLink USB and SyncLink PCIe Only)

When checked, this option disables 120ohm input termination on the USB adapter serial interface when differential modes are selected (RS422/RS530/V.35/X.21). GT cards control termination with resistor packs or switch settings.

Disable RS422/485 Outputs when RTS is off.

Choose this option when the state of the RTS output signal should be used to control output drivers (enabled or tri-state). An application controls RTS to manually tri-state drivers in a 2-wire half duplex (multidrop) environment.

Disable RS422/485 Outputs when not sending data.

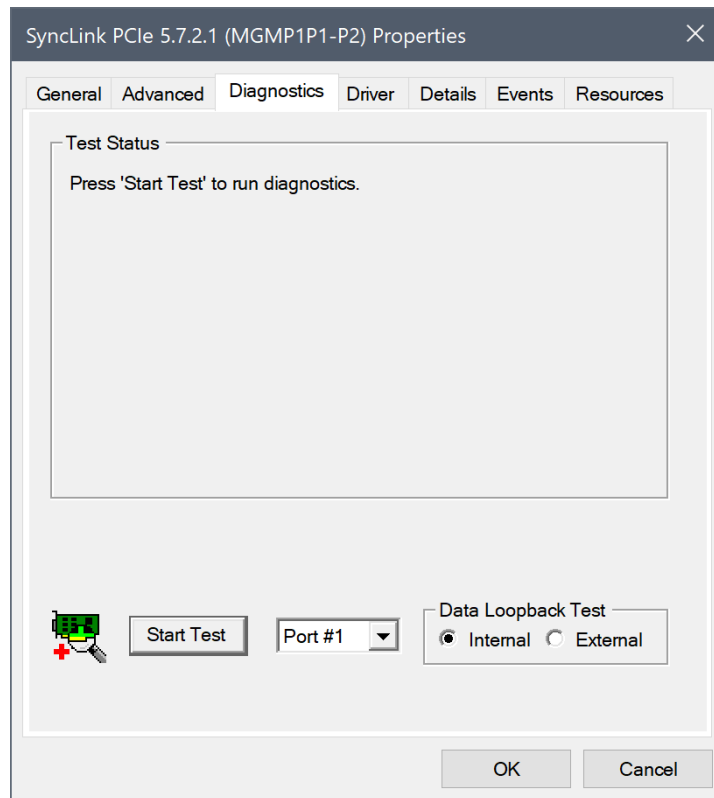
Choose this option if outputs should be disabled (tri-state) when not sending data. Hardware automatically controls driver outputs in a 2-wire half duplex (multidrop) environment.

DMA Buffer Size and DMA Buffer Count

These options control buffer allocation in the driver. Use 0 unless otherwise directed by Microgate support.

DIAGNOSTICS TAB

The **Diagnostics** tab allows you to test the device using an internal or external loopback of data.



Devices with more than one port will have a pull down list of ports. Select the port to test before proceeding. Devices with only one port will not have this list.

Select the **Data Loopback Test** type: **Internal** or **External**. The internal test does not access the serial connector and only tests the ability of Windows to talk to the device. If external is chosen, install the loopback plug that came with the device on the serial connector.

Then click the **Start Test** button. Send and receive data counts will start incrementing in the **Test Status** area. The test continues until the **Stop Test** button is clicked. Check the **Test Status** area for any error indications.

TRACE UTILITY

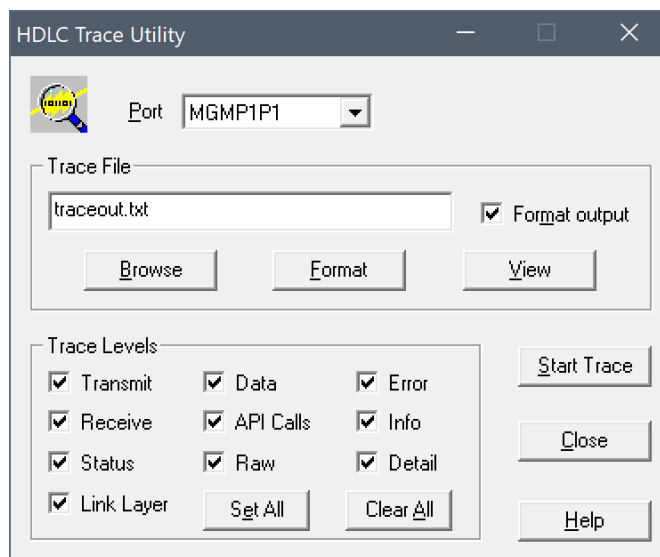
The trace utility records API events for debugging applications and is located at:

tools\win32\mgsltrc.exe	32-bit Windows
tools\win64\mgsltrc.exe	64-bit Windows

Requirements

- Trace type (32-bit/64-bit) must match application. 32-bit applications may run on 64-bit Windows but never at the same time as 64-bit applications. Application type is shown in Windows Task Manager. For C, C++ and C# the application type is determined by the build target. For Python, the application type is determined by the Python installation type.
- Trace must run as same user as application.
- The user must have administrative privilege.

If **access denied** or **port in use** error is reported by the application or trace utility then carefully review the above requirements. Tracing may start before or after starting application. Run trace from a command line or the Windows explorer. The trace program appears as shown below.



1. Enter the trace file name and select **Format output**.
2. Select trace levels. If unsure, enable all levels.
3. Select the port in the **Port** pull down list.
4. Click the **Start Trace** button to start the trace.
5. Perform the tasks to record (run application, connect, etc)
6. Click the **Stop Trace** button to stop the trace.

The output file may be examined for debugging or provided to MicroGate for support.

DEVICE INSTANCES AND NAMES

When hardware is first installed, Windows creates a unique collection of data for the hardware describing the location, configuration and associated software. This data is called a device instance. Each device instance for hardware present in the system is displayed in the Windows device manager. When hardware is removed from the system, the device instance remains but is not displayed in the device manager.

LOCATION LOCKED AND DEVICE LOCKED INSTANCES

Devices with a unique serial number accessible to Windows (SyncLink USB) use a device instance tied to the specific device called a device locked instance. PCI cards use a device instance tied to the location (PCI slot) called a location locked instance.

Hardware with a device locked instance may be moved to any location (USB port) and the same device instance is used. If the hardware is replaced, a new device instance is created for the new hardware with a different serial number.

Hardware with a location locked instance (PCI cards) may be replaced with the same card type in the same location and the same device instance is used. Moving hardware to a different location (PCI slot) creates a new device instance.

SERIAL API NAMES

When drivers are installed for a SyncLink hardware device, a device name is assigned to the device instance. The name is used to access the device with the serial API. The name is based on an instance number that is assigned sequentially starting with one.

Example:

The first SyncLink GT4 PCI card is adapter number one, with device names MGMP1P1 to MGMP1P4.
The second SyncLink GT4 PCI card is adapter number two, with device names MGMP2P1 to MGMP2P4.

If a PCI card is moved to a different slot, a new device instance is created with a different device name and applications using the original name will fail. The application must use the new name or the old device instance must be removed before creating a new device instance with the old name.

Example:

The first SyncLink GT4 PCI card is adapter number one, with device names MGMP1P1 to MGMP1P4.
The second SyncLink GT4 PCI card is adapter number two, with device names MGMP2P1 to MGMP2P4.
If the first card is moved to a different PCI slot, it becomes MGMP3P1 to MGMP3P2.

To reuse the name MGMP1P1 to MGMP1P4 in the new location, the first device instance must be removed before installing the card in the new location using these steps:

1. Remove hardware from system.
2. Start system and remove device instance. (see next section)
3. Install hardware in new location.

REMOVING DEVICE INSTANCES OF NON-PRESENT HARDWARE

Device instances of non-present hardware are not displayed in the device manager by default. The device manager can be configured using an environment variable to display device instances of non-present hardware:

1. Set the environment variable `devmgr_show_nonpresent_devices = 1`.
2. Start device manager.
3. Select **Show Hidden Devices** item from **View** menu.

Setting environment variable in Windows XP

- Click the **Start** button in the lower left of the desktop
- Right click **My Computer**
- Select **Properties** from pop-up menu
- Click the **Advanced** tab
- Click the **Environment Variables** button near bottom of window
- In the **System variables** section, click the **New** button
- In the **Variable name** field, type `devmgr_show_nonpresent_devices`
- In the **Variable value** field, type `1`
- Click the **OK** button to dismiss **New System Variable** window
- Click the **OK** button to dismiss **Environment Variables** window
- Click the **OK** button to dismiss **System Properties** window

Setting environment variable in Windows Vista and Windows 7

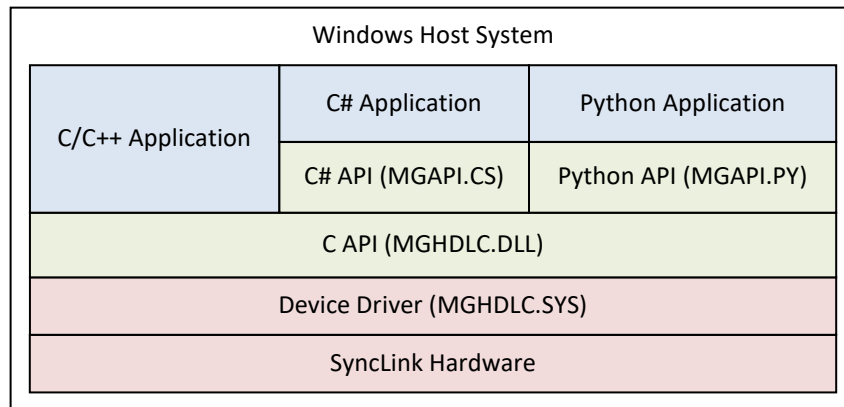
- Click the **Start** button in the lower left of the desktop
- Right click **Computer**
- Select **Properties** from pop-up menu
- Click the **Advanced system settings** in left side of window
- Click the **Environment Variables** button near bottom of window
- In the **System variables** section, click the **New** button
- In the **Variable name** field, type `devmgr_show_nonpresent_devices`
- In the **Variable value** field, type `1`
- Click the **OK** button to dismiss **New System Variable** window
- Click the **OK** button to dismiss **Environment Variables** window
- Click the **OK** button to dismiss **System Properties** window

Be careful typing the variable name to ensure it is entered **exactly** the same as above.

Now non-present devices are displayed in the device manager with a grayed out icon. Uninstall the device instance by right clicking on the non-present device and selecting **Uninstall** from the pop-up menu. Repeat this for all grayed out devices in the **Synclink Adapters** and **Synclink USB Service Ports** branches of the device tree. Once the non-present device instances have been removed, new hardware can be installed or old hardware moved to a new location.

SERIAL API PROGRAMMING

This section describes control of a serial device by an application using the serial API. The primary API is contained in a Windows library (DLL) which provides a C language interface using the Windows standard call convention. C# and Python APIs are provided by modules (mgapi.cs and mgapi.py) that translate the C interface. A working knowledge of the target language in a Windows environment is required.



SOFTWARE ORGANIZATION

Sample code is provided for each supported language. Use the sample code as a starting point for writing your own application. Samples operate on a single device with a loopback plug/cable or two devices connected by a null modem or crossover cable.

Note: The sample code is not intended to be run without modification as an end user application. The sample code WILL NOT match the specific requirements of your environment. The sample code is provided ONLY as a development aid.

C/C++

The following files are used to develop C/C++ serial applications.

<code>mgapi\c\include\mghdlc.h</code>	header file defining functions and structures
<code>mgapi\c\lib\mghdlc.lib</code>	import library for linking 32-bit applications
<code>mgapi\c\lib\x64\mghdlc.lib</code>	import library for linking 64-bit applications

Include the header file in the application source.

```
#include "mghdlc.h"
```

Link the application against the import library. The method for integrating these files into a build environment depends on the tools used. Microsoft Visual Studio requires the import library to be specified in the project settings for **additional libraries**.

Samples are Visual Studio solution/projects. Visual Studio 2010SP1 or later is required. Samples have been tested with Visual Studio 2010SP1 and Visual Studio 2022.

Contents of `mgapi\c\samples` directory:

<code>c.sln</code>	Visual Studio solution containing sample C/C++ projects
<code>hdlc\</code>	synchronous HDLC/SDLC
<code>raw\</code>	synchronous raw
<code>async\</code>	asynchronous/isochronous
<code>commapi\</code>	using Windows asynchronous COM API
<code>bisync\</code>	byte synchronous (BISYNC/MONOSYNC)
<code>2wire\</code>	HDLC in 2 wire half duplex mode
<code>tdm\</code>	Time Division Multiplexing (TDM)
<code>fsynth\</code>	use frequency synthesizer as base clock
<code>gpio\</code>	control and monitor GPIO signals

OPEN/CLOSE DEVICE

Call `Mgs1Open` or `Mgs1OpenByName` to get a handle to a serial port for use with other API calls. `Mgs1Open` takes a port identifier and `Mgs1OpenByName` takes a port name. If the port exists and is not in use, `ERROR_SUCCESS` and a device handle are returned. The handle is only valid for the Serial API. Do not use the handle with standard Windows calls.

Call `Mgs1EnumeratePorts` for a list of identifiers, types and names for each available port. The type and name can be used for display purposes when prompting the user for a port selection.

A port identifier is a 32-bit value identifying an adapter and a port. The upper 16 bits is the port number, and the lower 16 bits is the adapter number. The `MGSL_GET_ADAPTER()` macro returns the adapter number of a port ID. `MGSL_GET_PORT()` returns the port number of a port ID. `MGSL_MAKE_PORT_ID()` creates a port ID from an adapter and port number. Single port adapters always have a port number of 0, so the port ID is the same as the adapter number. Multiport adapters have port numbers starting with 1 up to the number of ports on the adapter.

Port names for single port adapters have the form MGHDLCx, where 'x' is the adapter number. Port names for multiport adapters have the form MGMPxPn, where x is the adapter number and 'n' is the port number. Adapter numbers are assigned automatically when the adapter is installed.

The following two calls both open the third port of the first multiport adapter.

```
/* open device with integer port identifier */
rc = MgsOpen(MGSL_MAKE_PORT_ID(1, 3), &dev);
if (rc != ERROR_SUCCESS)
    printf("MgsOpen error=%d\n", rc);

/* open device with device name */
rc = MgsOpenByName("MGMP1P3", &dev);
if (rc != ERROR_SUCCESS)
    printf("MgsOpen error=%d\n", rc);
```

The following two calls both open the only port of the second single port adapter.

```
/* open device with integer port identifier */
rc = MgsOpen(MGSL_MAKE_PORT_ID(2, 0), &dev);
if (rc != ERROR_SUCCESS)
    printf("MgsOpen error=%d\n", rc);

/* open device with device name */
rc = MgsOpenByName("MGHDL2", &dev);
if (rc != ERROR_SUCCESS)
    printf("MgsOpen error=%d\n", rc);
```

Call MgsClose with an open handle after the port is no longer needed so other processes can open the port.

CONFIGURE DEVICE

A device must be configured to match application specific requirements. This is done using the following API calls.

MgsSetPortConfigEx	Set options that take effect at driver load time, such as serial interface type. These settings are stored in the Windows registry and are read by the driver when loading, usually when Windows is starting. These options are usually set using the device properties in the Windows Device Manager instead of from an application with this call.
MgsSetParams	Set options that take effect at start of communications session. This is the main configuration call. Calling this function resets the transmitter and receiver.
MgsSetIdleMode	Set the idle (SDLC/HDLC/raw) or sync (monosync/bisync) pattern.
MgsSetOption	Set options that can change during a communications session.

The main configuration call is MgsSetParams, which uses an MGSL_PARAMS structure to specify protocol options. This call is documented in the MgsSetParams and MGSL_PARAMS sections. The following sample configures a port for HDLC mode. The actual settings used depend on the application requirements.

```

HANDLE dev;
int rc;
MGSL_PARAMS params;

/*
 * SDLC/HDLC mode, loopback disabled
 * receive clock source = RxC input pin
 * transmit clock source = TxC input pin
 * NRZ encoding
 * output 9600bps clock on AUXCLK output pin
 * use ITU/CCITT 16-bit CRC frame check
 */
params.Mode = MGSL_MODE_HDLC;
params.Loopback = 0;
params.Flags = HDLC_FLAG_RXC_RXCPIN + HDLC_FLAG_TXC_TXCPIN;
params.Encoding = HDLC_ENCODING_NRZ;
params.ClockSpeed = 9600;
params.CrcType = HDLC_CRC_16_CCITT;

/* set current device parameters */
rc = Mgs1SetParams(dev, &params);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetParams error=%d", rc);

/* set transmit idle pattern (sent between frames) */
rc = Mgs1SetIdleMode(dev, HDLC_TXIDLE_ONES);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetIdleMode error=%d", rc);

```

RECEIVING DATA

An application gets receive data using the `Mgs1Read` call.

```

unsigned char buf[4096];
int size = sizeof(buf);
int count;

/* get receive data */
count = Mgs1Read(dev, buf, size);
if (count) {
    /* count bytes returned in buf */
} else {
    /* no data available (polled mode) or error (blocking mode) */
}

```

`Mgs1ReadWithStatus` lets applications process receive errors in addition to valid receive data or inspect received CRC values. `Mgs1Receive` adds asynchronous notification in addition to error reporting and should only be used in the very rare cases where the extra features are needed and well understood.

- `Mgs1Read` = preferred method to receive data (easy)
- `Mgs1ReadWithStatus` = receive data and error reporting (more complex)
- `Mgs1Receive` = receive data, error reporting and asynchronous notification (most complex)

Behavior of the receive functions depends on the configured serial protocol (HDLC, bisync, async, etc) as described in later sections and in the provided sample code.

HDLC/SDLC

Each buffer returned by the API is one frame of variable size. Set `size` argument to largest expected frame size (typically 4K). Actual frame size is reported by return code.

Monosync/Bisync/Raw/Asynchronous/Isosynchronous

Each buffer returned by the API is a fixed size block of data, set by the `size` argument, and does not imply any message boundaries. In these modes the API does not detect message boundaries, which is the responsibility of the application. Data is not returned until `size` bytes are received.

Blocking and Polled Modes

Receive calls can be used in blocking or polled mode by calling `MgslSetOption` with the `MGSL_OPT_RX_POLL` identifier. When this option is enabled, receive calls return immediately when no data is available (polling mode). When not enabled, receive calls block until data is available (blocking mode = default).

A blocked receive call can be canceled from a different application thread with `MgslCancelReceive`.

MgslReceive (Advanced Features)

`MgslReceive` has extra error reporting and asynchronous notification features for advanced applications and uses the `MGSL_RECEIVE_REQUEST` structure. The receive request structure and overlapped structure must be initialized before each call. If data is available, `ERROR_SUCCESS` is returned. If no data is available and the receiver is enabled then `ERROR_IO_PENDING` is returned and the caller monitors the event member of the overlapped structure for notification of request completion. When the request is complete, the request structure contains data and status information.

This sample code demonstrates request preparation, call and processing. `CreateEvent`, `ResetEvent` and `WaitForSingleObject` are Windows system calls. Refer to the Windows SDK documentation for details. The request and overlapped structures **must** remain valid (heap or stack allocation) until the request completes.

```

OVERLAPPED ol;
MGSL_RECEIVE_REQUEST *req;
int buffer_size = 4096;

/* request preparation */
ol.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
req = malloc(sizeof(MGSL_RECEIVE_REQUEST) + buffer_size);
req->DataLength = buffer_size;
ResetEvent(ol.hEvent);

rc = Mgs1Receive(dev, req, &ol);
if (rc == ERROR_IO_PENDING) {
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS) {
    /* Mgs1Receive error */
}

/* process completed request */
if (req->Status == RxStatus_OK) {
    /* req->DataLength contains count of returned data */
    /* req->DataBuffer contains returned data */
} else
    /* receive error (CRC etc), no data returned */

```

SENDING DATA

An application sends data using the `Mgs1Write` call.

```

unsigned char buf[4096];
int size = sizeof(buf);
int count;

/* initialize buffer with application data */

/* send data */
count = Mgs1Write(dev, buf, size);
if (count) {
    /* count bytes stored in API send buffers */
} else {
    /* API send buffers full (polled) or error (blocked) */
}

```

`Mgs1Write` is a simplified wrapper function for the original `Mgs1Transmit` function described below. If you do not need the extra error reporting and asynchronous notification features of `Mgs1Transmit`, use the simpler `Mgs1Write`.

The format of the data depends on the protocol. For frame oriented SDLC/HDLC, each call sends a single frame of data. For other protocols, each call sends data with no formatting, requiring the application to implement message boundaries.

Blocking and Polled Modes

`MgslWrite` (and `MgslTransmit`) can be used in blocking or polled mode by calling `MgslSetOption` with the `MGSL_OPT_TX_POLL` identifier. When this option is enabled, `MgslWrite` (or `MgslTransmit`) returns immediately with a return code of zero when all API send buffers are full (polling mode). When not enabled, `MgslWrite` blocks until API send buffers are available and data is accepted (blocking mode = default).

A blocked call to `MgslWrite` can be aborted by calling `MgslCancelTransmit` from a different application thread.

MgslTransmit (advanced)

`MgslTransmit` offers additional error reporting and asynchronous notification. If data is accepted, the call returns `ERROR_SUCCESS`. If no buffers are available then `ERROR_IO_PENDING` is returned and the caller monitors the event member of the overlapped structure for notification of request completion.

Buffers are sent as soon as possible, but the request completes before the data is actually sent. `MgslTransmit` can be used to determine when all buffered data has been sent. Refer to the function reference for details.

This sample code demonstrates request preparation, call and processing. `CreateEvent`, `ResetEvent` and `WaitForSingleObject` are Windows system calls. Refer to the Windows SDK documentation for details. The send buffer and overlapped structure **must** remain valid (heap or stack allocation) until the request completes.

```
int rc;
int size = 1024;
unsigned char buf[1024];
OVERLAPPED ol;

ol.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

/* submit data to send */
ResetEvent(ol.hEvent);
rc = MgslTransmit(dev, buf, size, NULL, &ol);
if (rc == ERROR_IO_PENDING) {
    /* wait for free buffer */
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS) {
    /* MgslTransmit error */
} else {
    /* data accepted */
}
```

WAITING FOR ALL DATA SENT

Data given to the API with `MgslWrite` or `MgslTransmit` is saved to API buffers and sent as soon as possible. To determine when the buffered data has been completely sent use the `MgslWaitAllSent` call.

```

int rc;

/* check for all buffered data sent */
rc = Mgs1WaitAllSent(dev);
if (!rc) {
    /* all buffered data sent, safe to close port */
} else {
    /* busy sending buffered data (polled) or error (blocking) */
}

```

Mgs1WaitAllSent is a simplified wrapper function for the original Mgs1Transmit function. If you do not need the extra error reporting and asynchronous notification features of Mgs1Transmit, use the simpler Mgs1WaitAllSent.

Blocking and Polled Modes

Mgs1WaitAllSent can be used in blocking or polled mode by calling Mgs1SetOption with the MGSL_OPT_TX_POLL identifier. When this option is enabled, Mgs1WaitAllSent returns immediately with a return code of zero when all data is sent or non zero if still sending (polled mode). When not enabled, Mgs1WaitAllSent blocks until all buffered data has been sent (blocking mode = default).

A blocked call to Mgs1WaitAllSent can be aborted by calling Mgs1CancelTransmit from a different application thread.

ASYNCHRONOUS API NOTIFICATION

API calls that may not complete immediately (Mgs1Transmit, Mgs1Receive, Mgs1WaitEvent, Mgs1WaitGpio, Mgs1GetTraceEvent) require an application allocated Windows OVERLAPPED structure.

Initialize the hEvent member of the overlapped structure with the handle of a Windows manual reset event allocated with CreateEvent. Set the other members of the overlapped structure to zero. If an API call returns ERROR_IO_PENDING, monitor the event with the Windows functions WaitForSingleObject or WaitForMultipleObjects. When the API signals the event object, the call is complete.

The overlapped structure must remain allocated (on the stack or heap) until request completion.

For more information about the OVERLAPPED structure, Windows events and synchronization, refer to the Windows SDK documentation.

API calls using asynchronous notification can have only one pending instance of each API call. For example, if a Mgs1Transmit call is pending, the application cannot call Mgs1Transmit again until the original call completes. Different API calls, such as Mgs1Transmit and Mgs1Receive, may be simultaneously pending. Calling an API function that uses asynchronous notification while an instance of that call is already pending results in a return code of ERROR_BUSY.

FUNCTIONS

This section documents the application programming interface (API) calls used with serial devices. The calls use the Windows standard call (__stdcall) calling convention used by most Windows libraries. The documentation uses the C programming language, but other languages that can access the standard call conventions can also be used.

Call interface details are defined in the C language header file `mghdlc.h`, which must be included in C language source files. The user mode application interface is implemented in the `mghdlc.dll` library which translates the calls to access the device driver `mghdlc.sys`. Applications need to link against the import library `mghdlc.lib`.

MGSLCANCELGETTRACEEVENT

```
ULONG MgslCancelGetTraceEvent(HANDLE dev);
```

Arguments

<code>dev</code>	handle to open device
------------------	-----------------------

Return Value

<code>ERROR_SUCCESS</code>	call success
<code>ERROR_INVALID_HANDLE</code>	device handle is invalid

This function cancels a pending call to `MgslGetTraceEvent`. The API completes the pending request with `EventType_None` and signals the event member of the overlapped structure passed to `MgslGetTraceEvent`. Pending requests are automatically cancelled when a device handle is closed.

MGSLCANCELGETWAITGPIO

```
ULONG MgslCancelGetWaitGpio(HANDLE dev);
```

Arguments

<code>dev</code>	handle to open device
------------------	-----------------------

Return Value

<code>ERROR_SUCCESS</code>	call success
<code>ERROR_INVALID_HANDLE</code>	device handle is invalid

This function cancels a pending call to `MgslWaitGpio`. The API signals the `hEvent` member of the overlapped structure passed to `MgslWaitGpio` and the contents of the `GPIO_DESC` structure is undefined. Pending requests are automatically cancelled when a device handle is closed.

MGSLCANCELRECEIVE

```
ULONG Mgs1CancelReceive(HANDLE dev);
```

Arguments

dev	handle to open device
-----	-----------------------

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function cancels a pending `Mgs1Receive` call. The API signals the `hEvent` member of the overlapped structure passed to `Mgs1Receive` and the receive status is set to `RxStatus_Cancel`. Pending requests are automatically cancelled when a device handle is closed.

MGSLCANCELTRANSMIT

```
ULONG Mgs1CancelTransmit(HANDLE dev);
```

Arguments

dev	handle to open device
-----	-----------------------

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function cancels a pending `Mgs1Transmit` call. The API signals the `hEvent` member of the overlapped structure passed to `Mgs1Transmit` and the transmit status is set to `TxStatus_Cancel`. Pending requests are automatically cancelled when a device handle is closed.

Calling this function does not disable the transmitter. The transmitter continues sending the idle pattern until more data is sent or the user disables the transmitter with the `Mgs1EnableTransmitter` call.

MGSLCANCELWAITEVENT

```
ULONG Mgs1CancelWaitEvent(HANDLE dev);
```

Arguments

dev	handle to open device
-----	-----------------------

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function cancels a pending `Mgs1WaitEvent` call. The API signals the `hEvent` member of the overlapped structure passed to `Mgs1WaitEvent` and the returned value of serial events is set to zero. Pending requests are automatically cancelled when a device handle is closed.

MGSLCLOSE

```
ULONG MgsIclose(HANDLE dev);
```

Arguments

dev	handle to open device
-----	-----------------------

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function closes an open device handle. The handle is not valid after this call. Pending requests are automatically cancelled when a device handle is closed. A handle must be closed before another process can open the device.

MGSENABLERECEIVER

```
ULONG MgsIenableReceiver(HANDLE dev, BOOL enable);
```

Arguments

dev	handle to open device
enable	enable command value
	0 = disable
	1 = enable
	2 = enable and force hunt mode

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function controls the receiver state:

Disabled	receive data signal ignored
Idle	receive data signal scanned for synchronization pattern (flag, start bit, etc)
Active	receive data signal stored for application

The receiver starts disabled. If the receiver is disabled, the enable command makes the receiver idle and discards buffered data, otherwise it does nothing. The disable command disables the receiver. The hunt mode command makes the receiver idle.

A synchronization pattern (start bit, flag, sync) makes an idle receiver active. Raw synchronous mode does not use a synchronization pattern, and the receiver is either disabled or active with both the enable and hunt mode commands making the receiver active.

```
/* enable receiver (store data on receive data input) */
rc = MgsIenableReceiver(dev, 1);
if (rc != ERROR_SUCCESS)
    /* process error */

/* disable receiver (ignore receive data input) */
rc = MgsIenableReceiver(dev, 0);
if (rc != ERROR_SUCCESS)
    /* process error */
```

```

/* force receiver to idle state (look for sync pattern) */
rc = Mgs1EnableReceiver(dev, 2);
if (rc != ERROR_SUCCESS)
    /* process error */

```

MGSLENABLETRANSMITTER

```

ULONG Mgs1EnableTransmitter(HANDLE dev, BOOL enable);

```

Arguments

dev	handle to open device
enable	enable command value
	0 = disable
	1 = enable

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	device handle is invalid

This function controls the transmitter state:

Disabled	transmit data signal is constant mark (one)
Idle	transmit data signal sends idle or sync pattern
Active	transmit data signal sends data

The transmitter starts disabled. The enable command makes the transmitter idle if disabled, otherwise it does nothing. The disable command disables the transmitter and discards buffered data. Calling `Mgs1Transmit` enables the transmitter if disabled. When data is ready to send, the transmitter is active.

```

/* enable transmitter (start sending idle pattern) */
rc = Mgs1EnableTransmitter(dev, 1);
if (rc != ERROR_SUCCESS)
    /* process error */

/* disable transmitter (discard unsent data) */
rc = Mgs1EnableTransmitter(dev, 0);
if (rc != ERROR_SUCCESS)
    /* process error */

```

MGSLENUMERATEPORTS

```
ULONG MgsEnumeratePorts(MGSL_PORT *ports, ULONG size, ULONG *count);
```

Arguments

ports	pointer to buffer to receive array of MGSL_PORT structures
size	size of ports buffer in bytes
count	returned count of port structures

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_PARAMETER	ports buffer invalid or too small
ERROR_GEN_FAILURE	unspecified failure

This function returns information on available serial API ports in an array of MGSL_PORT structures. Use this information to identify and open ports with the MgsOpen call. On return the ports buffer contains the port information and the count argument contains the number of returned entries.

Call this function with the ports argument set to NULL to return only the number of available ports in the count argument. Use this to allocate a buffer for the ports argument on a subsequent call.

```
char name[] = "MGHDL1";
unsigned long i, rc, count;
int port_id = 0;
MGSL_PORT *ports;

/* get count of available ports */
rc = MgsEnumeratePorts(NULL, 0, &count);
if (rc != ERROR_SUCCESS)
    /* process error */

/* allocate memory to hold port information */
ports = malloc(count * sizeof(MGSL_PORT));

/* get port information */
rc = MgsEnumeratePorts(ports, count * sizeof(MGSL_PORT), &count);
if (rc != ERROR_SUCCESS)
    /* process error */

/* convert device name to port_id */
for (i=0; i < count; i++) {
    if (!strcmp(ports[i].DeviceName, name)) {
        port_id = ports[i].PortID;
        break;
    }
}

free(ports);
```

MGSLGETASSIGNEDRESOURCES

```
ULONG MgsIGetAssignedResources(HANDLE dev, MGSL_ASSIGNED_RESOURCES *res);
```

Arguments

dev	handle to open device
res	pointer to MGSL_ASSIGNED_RESOURCES structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid res buffer

This function returns the embedded serial number of the device, if available. Only the `SerialNumber` field of the structure is used. All other fields of the structure are unused and undefined. Only the SyncLink USB device has an embedded serial number. For all other devices, this function has no purpose.

```
MGSL_ASSIGNED_RESOURCES res;

rc = MgsIGetAssignedResources(dev, &res);
if (rc != ERROR_SUCCESS)
    printf("MgsIGetAssignedResources error=%d\n", rc);
else
    printf("serial number is %s\n", res.SerialNumber);
```

MGSLGETGPIO

```
ULONG MgsIGetGpio(HANDLE dev, GPIO_DESC *gpio);
```

Arguments

dev	handle to open device
gpio	pointer to GPIO_DESC structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid gpio buffer

This function returns the current direction configuration and state of all general purpose I/O (GPIO) signals.

```
GPIO_DESC gpio;

rc = MgsIGetGpio(dev, &gpio);
if (rc != ERROR_SUCCESS)
    printf("MgsIGetGpio error=%d\n", rc);
else {
    /* process structure contents */

    if (gpio.dir & (1 << 5))
        printf("GPIO 5 is an output\n");
    else
```

```

        printf("GPIO 5 is an input\n");

    if (gpio.state & (1 << 5))
        printf("GPIO 5 is on\n");
    else
        printf("GPIO 5 is off\n");
}

```

MGSLGETOPTION

```
ULONG Mgs1GetOption(HANDLE dev, UINT option, UINT *value);
```

Arguments

dev	handle to open device
option	option identifier
value	pointer to returned option value

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid option identifier or value buffer

This function returns the specified configuration or status value. Refer to `Mgs1SetOption` for a description of valid option identifiers.

```

UINT value;

rc = Mgs1GetOption(dev, MGSL_OPT_RTS_DRIVER_CONTROL, &value);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetOption error=%d\n", rc);
else if (value)
    printf("RTS controls output drivers\n");
else
    printf("RTS does not control output drivers\n");

```

MGSLGETPARAMS

```
ULONG Mgs1GetParams(HANDLE dev, MGSL_PARAMS *params);
```

Arguments

dev	handle to open device
params	pointer to MGSL_PARAMS structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid params buffer

This function returns the current serial device configuration in a `MGSL_PARAMS` structure. This call gets options that are specified once for a communications session. `Mgs1GetPortConfigEx` is used for driver load time settings and `Mgs1GetOption` is used for settings that may change during a communications session.

```

MGSL_PARAMS params;

rc = MgsIGetParams(dev, &params);
if (rc != ERROR_SUCCESS)
    printf("MgsIGetParams error=%d\n", rc);
else
    /* process structure contents */

```

MGSLGETPORTCONFIGEX

```

ULONG MgsIGetPortConfigEx(ULONG port_id, MGSL_PORT_CONFIG_EX *config);

```

Arguments

port_id	integer port identifier (not an open port handle)
config	pointer to returned MGSL_PORT_CONFIG_EX structure

Return Value

ERROR_SUCCESS	call success
ERROR_ACCESS_DENIED	insufficient privilege to access registry
ERROR_INVALID_PARAMETER	invalid config buffer

This function returns the current serial device configuration in a MGSL_PORT_CONFIG_EX structure, which is used for options that take effect at driver load time. These options are usually set in the Windows Device Manager, but this call is implemented so the options can be programmatically set.

```

MGSL_PORT_CONFIG_EX config;
unsigned int port_id;

/* operate on port 3 of first adapter */
port_id = MGSL_MAKE_PORT_ID(1,3);

rc = MgsIGetPortConfigEx(port_id, &config);
if (rc != ERROR_SUCCESS)
    printf("MgsISetPortConfigEx error=%d\n", rc);
else
    /* process structure contents */

```

MGSLGETSERIALSIGNALS

```
ULONG MgsIGetSerialSignals(HANDLE dev, UCHAR *signals);
```

Arguments

dev	open port handle
config	returned signal states

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid port handle
ERROR_INVALID_PARAMETER	invalid signals buffer

This function returns the state of serial control and status signals. Signal states are identified by macros defined in the `mghdlc.h` header file. A set bit indicates an active signal. Depending on the application, some signals may not be used or may be used for a non-standard purpose.

SerialSignal_DCD	Data Carrier Detect input (MODEM or DCE detects signal from remote device)
SerialSignal_DSR	Data Set Ready input (MODEM or DCE is turned on)
SerialSignal_DTR	Data Terminal Ready output (serial device is active)
SerialSignal_RTS	Request to Send output (serial device needs to send data)
SerialSignal_CTS	Clear to Send input (serial device is allowed to send data)
SerialSignal_RI	Ring Indicator input (MODEM or DCE detects incoming call)

```
UCHAR signals;
```

```
rc = MgsIGetSerialSignals(dev, &signals);
if (rc != ERROR_SUCCESS)
    /* process error */
else if (signals & SerialSignal_DCD)
    /* DCD is active */
```

MGSLGETTRACEEVENT

```
ULONG MgsIGetTraceEvent(HANDLE dev, MGSL_TRACE_EVENT *event, OVERLAPPED *ol);
```

Arguments

dev	open port handle
event	pointer returned trace event structure
ol	pointer to Windows overlapped structure for asynchronous notification of call completion

Return Value

ERROR_SUCCESS	call success (immediate completion)
ERROR_IO_PENDING	waiting for trace event (monitor overlapped structure)
ERROR_INVALID_HANDLE	invalid port handle
ERROR_INVALID_PARAMETER	invalid event buffer
ERROR_BUSY	request already pending

This function returns a trace event. If a trace event is immediately available, ERROR_SUCCESS is returned, otherwise ERROR_IO_PENDING is returned and the application should monitor the hEvent member of the overlapped structure for indication of request completion. MgsICancelGetTraceEvent cancels a pending request. Only one MgsIGetTraceEvent request can be active at a time.

This function uses standard Windows asynchronous notification. This sample code demonstrates request preparation, call and processing. CreateEvent, ResetEvent and WaitForSingleObject are Windows system calls. Refer to Windows SDK documentation for details. The request and overlapped structures **must** remain valid (heap or stack allocation) until the request completes.

```
OVERLAPPED ol;
MGSL_TRACE_EVENT event;

ol.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

/* request preparation */
ResetEvent(ol.hEvent);

rc = MgsIGetTraceEvent(dev, &event, &ol);
if (rc == ERROR_IO_PENDING) {
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS) {
    /* MgsIGetTraceEvent error */
}

/* process contents of MGSL_TRACE_EVENT structure */
```

MGSLGETTRACELEVEL

```
ULONG Mgs1GetTraceLevel(HANDLE dev, ULONG *level);
```

Arguments

dev	open port handle
level	pointer to returned trace level value

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid port handle
ERROR_INVALID_PARAMETER	invalid level buffer

This function returns the current trace level that specifies which events are recorded in the trace buffer. Levels are identified with `TraceLevel_XXX` macros defined in the `mghdlc.h` header file. See `Mgs1SetTraceLevel` for a description of the different trace levels.

```
ULONG level;
```

```
rc = Mgs1GetTraceLevel(dev, &level);
if (rc != ERROR_SUCCESS)
    printf("Mgs1GetTraceLevel error=%d\n", rc);
else if (level & TraceLevel_Data)
    printf("Data tracing is enabled\n");
```

MGSLOPEN

```
ULONG Mgs1Open(ULONG port_id, HANDLE *dev);
```

Arguments

port_id	port identifier
dev	pointer to returned port handle

Return Value

ERROR_SUCCESS	call success
ERROR_DEVICE_IN_USE	port is in use by another application
ERROR_BAD_DEVICE	port_id does not exist or is not functioning

This function opens a serial device identified by `port_id` and returns a port handle for use by other API functions. Call `Mgs1Close` to close the returned handle when it is no longer needed. The port ID can be obtained from `Mgs1EnumeratePorts`.

The format of the port ID depends on the hardware type:

Single Port Adapter	32-bit integer adapter number Example: 3 (single port adapter 3)
Multiple Port Adapter	32-bit integer with upper 16 bits = port number and lower 16 bits = adapter number Example: 0x00040003 (port 4 of adapter 3)

Macros for manipulating port IDs are defined in the `mghdlc.h` header file:

```
MGSL_MAKE_PORT_ID(AdapterNumber, PortNumber)
MGSL_GET_ADAPTER(PortID)
MGSL_GET_PORT(PortID)

unsigned int port_id;
HANDLE dev;

/* operate on port 3 of first adapter */
port_id = MGSL_MAKE_PORT_ID(1, 3);

rc = MgslOpen (port_id, &dev);
if (rc != ERROR_SUCCESS)
    printf("MgslOpen error=%d\n", rc);
else
    /* dev contains valid device handle */
```

MGSLOPENBYNAME

```
ULONG MgslOpenByName(char *name, HANDLE *dev);
```

Arguments

name	port name string
dev	pointer to returned port handle

Return Value

ERROR_SUCCESS	call success
ERROR_DEVICE_IN_USE	port is in use by another application
ERROR_BAD_DEVICE	port_id does not exist or is not functioning

This function opens a serial device identified by name and returns a port handle for use by other API functions. Call `MgslClose` to close the returned handle when it is no longer needed.

The format of the name depends on the hardware type:

Single Port Adapter	MGHDLCx, where x = adapter number Example: MGHDL3
Multiple Port Adapter	MGMPxPy, where x = adapter number and y = port number Example: MGMP2P4

The port name can be obtained from the `MgslEnumeratePorts` call and from the Windows Device Manager.

```
HANDLE dev;

/* open port 3 of first adapter */
rc = MgslOpenByName("MGMP1P3", &dev);
if (rc != ERROR_SUCCESS)
    printf("MgslOpenByName error=%d\n", rc);
else
    /* dev contains valid device handle */
```

MGSLOPENTRACEHANDLE

```
ULONG MgsOpenTraceHandle(ULONG port_id, HANDLE *dev);
```

Arguments

port_id	port identifier
dev	pointer to returned port handle

Return Value

ERROR_SUCCESS	call success
ERROR_DEVICE_IN_USE	port is in use by another tracing application
ERROR_BAD_DEVICE	port_id does not exist or is not functioning
ERROR_ACCESS_DENIED	insufficient privilege to open a trace handle

This function opens a serial device identified by `port_id` and returns a port handle for use by other API functions. Call `MgsClose` to close the returned handle when it is no longer needed. The port ID can be obtained from `MgsEnumeratePorts`. This call requires Administrative privilege to succeed.

This call allows two processes to access a single port, one for normal use and the other for tracing. Normally only a single process can open a port. This function is used by the `mgsltrc.exe` tracing utility included with the Serial API. Source code for `mgsltrc.exe` is provided. The tracing API calls allow a custom serial application to integrate tracing into an application.

```
unsigned int port_id;
HANDLE dev;

/* operate on port 3 of first adapter */
port_id = MGSL_MAKE_PORT_ID(1,3);

rc = MgsOpenTraceHandle(port_id, &dev);
if (rc != ERROR_SUCCESS)
    printf("MgsOpenTraceHandle error=%d\n", rc);
else
    /* dev contains valid device handle */
```

MGSLPUTTRACEEVENT

```
ULONG Mgs1PutTraceEvent(HANDLE dev, MGSL_TRACE_EVENT *event);
```

Arguments

dev	returned port handle
event	MGSL_TRACE_EVENT structure to add to trace buffer

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	port handle is invalid
ERROR_INVALID_PARAMETER	event buffer is invalid

This function adds a trace event to a serial device's trace buffer. The event is described by an `MGSL_TRACE_EVENT` structure. The call fills out the `EventType`, `DataLength` and `EventData` fields. The API sets the `TimeStamp` field. See the documentation for the `MGSL_TRACE_EVENT` structure for a description of the fields and associated constants.

```
MGSL_TRACE_EVENT event;

/* initialize structure */

rc = Mgs1PutTraceEvent(dev, &event);
if (rc != ERROR_SUCCESS)
    /* process error */
```

MGSLRECEIVE

```
ULONG Mgs1Receive(HANDLE dev, MGSL_RECEIVE_REQUEST *req, OVERLAPPED *ol);
```

Arguments

dev	open port handle
req	pointer to receive request structure
ol	pointer to Windows overlapped structure for asynchronous notification of request completion

Return Value

ERROR_SUCCESS	receive data returned
ERROR_IO_PENDING	waiting for receive data (monitor overlapped structure)
ERROR_INVALID_HANDLE	port handle is invalid
ERROR_INVALID_PARAMETER	request buffer is invalid
ERROR_NOT_READY	receiver is disabled and no data is available

This function returns received data to the application. The receive request structure and overlapped structure must be initialized before each call. If data is available, `ERROR_SUCCESS` is returned. If data is not available and the receiver is enabled then `ERROR_IO_PENDING` is returned and the caller monitors the `hEvent` member of the overlapped structure for notification of request completion.

Call `Mgs1CancelReceive` to stop a pending call to `Mgs1Receive`. When the cancellation is complete, the `hEvent` member of the overlapped structure will be signaled.

This sample code demonstrates request preparation, call and processing. `CreateEvent`, `ResetEvent` and `WaitForSingleObject` are Windows system calls. Refer to Windows SDK documentation for details. The request and overlapped structures **must** remain valid (heap or stack allocation) until the request completes.

```
OVERLAPPED ol;
int buffer_size = 4096;
MGSL_RECEIVE_REQUEST *req;

ol.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);
req = malloc(sizeof(MGSL_RECEIVE_REQUEST) + buffer_size);

/* preparation and call */
req->DataLength = buffer_size;
ResetEvent(ol.hEvent);
rc = MgsIReceive(dev, req, &ol);
if (rc == ERROR_IO_PENDING) {
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS)
    /* MgsIReceive error */

/* process completed request */
if (req->Status == RxStatus_OK)
    /* req->DataLength set to count of data in req->DataBuffer */
else
    /* receive error (CRC etc), no data returned */
```

The amount of data available in the API buffers is obtained using `MgsIGetOption` with `MGSL_OPT_RX_COUNT`. Refer to the `MGSL_RECEIVE_REQUEST` structure reference for detailed descriptions of the fields and values.

Raw, Bisync/Monosync Buffer Fill Level:

For raw, bisync and monosync modes, `MgsIReceive` returns data when a driver receive buffer (256 bytes for PCI cards, 128 bytes for USB device) fills. At low data rates this may cause too much delay between receipt of a byte and that byte being returned to the application.

The application can reduce the number of bytes returned per call by setting the `DataLength` member of the `MGSL_RECEIVE_REQUEST` structure to the desired count. When the value of `DataLength` changes from the previous value, the receiver is reset discarding all data. Values over the default value (256 for PCI, 128 for USB) result in the default number of bytes returned per call.

The value also controls the data transfer mode used by hardware (PIO or DMA).

128 to 256	DMA mode, value MUST be a multiple of 4
1 to 127	PIO mode, any value in this range is valid

Use lower values as needed for lower data rates and lower latency. At high data rates PIO mode may cause data loss.

Polled Mode

Use `MgslSetOption` with `MGSL_OPT_RX_POLL` to enable polled mode. When enabled, `MgslReceive` returns `ERROR_BUSY` and cancels the request instead of returning `ERROR_IO_PENDING` if no data is available to return. The application must still allocate an overlapped structure and event even when using polled mode.

MGSLREAD

```
int MgslRead(HANDLE dev, unsigned char *buf, int size);
```

Arguments

<code>dev</code>	open port handle
<code>buf</code>	pointer to buffer to hold receive data
<code>size</code>	size of buffer in bytes

Return Value

Number of bytes returned in buffer, or zero if timeout or error.

`MgslRead` returns received data to the application when available. `MgslRead` is a simplified wrapper function for the `MgslReceive`. If you do not need the error reporting and asynchronous notification features of `MgslReceive`, use the simpler `MgslRead`.

```
unsigned char buf[4096];
int size = sizeof(buf);
int count;

/* get receive data */
count = MgslRead(dev, buf, size);
if (count) {
    /* count bytes returned in buf */
} else {
    /* no data available (polled) or error (blocking) */
}
```

The amount of data available in the API buffers is obtained using `MgslGetOption` with `MGSL_OPT_RX_COUNT`.

`MgslRead` behavior depends on the serial protocol.

HDLC/SDLC

Each buffer returned by the API is one frame of variable size. Set `size` argument to largest expected frame size (typically 4K). Actual frame size is reported by return code.

Monosync/Bisync/Raw/Asynchronous/Isosynchronous

Each buffer returned by the API is a fixed size block of data, set by the `size` argument, and does not imply any message boundaries. In these modes the API does not detect message boundaries, which is the responsibility of the application. Data is not returned until `size` bytes are received.

For best performance set `size` to 256 to match the default hardware data transfer size. At low data rates this may cause too much delay (latency) between receipt of a byte and that byte being returned to the application. Use a lower value of `size` to reduce latency. When the `size` argument changes from the previous value, the receiver is

reset, discarding all data and the new value is used. The value also controls the data transfer mode used by hardware (PIO or DMA/Packet).

128 to 256	DMA/Packet mode (MUST be a multiple of 4)
1 to 127	PIO mode, may cause data loss at high speed

Blocking and Polled Modes

`MgslRead` can be used in blocking or polled mode by calling `MgslSetOption` with the `MGSL_OPT_RX_POLL` identifier. When this option is enabled, `MgslRead` returns immediately with a return code of zero when no data is available (polling mode). When not enabled, `MgslRead` blocks until data is available (blocking mode = default).

A blocked call to `MgslRead` can be aborted by calling `MgslCancelReceive` from a different application thread.

MGSLREADWITHSTATUS

```
int MgslReadWithStatus(HANDLE dev, unsigned char *buf,
    int size, int *status);
```

Arguments

<code>dev</code>	open port handle
<code>buf</code>	pointer to buffer to hold receive data
<code>size</code>	size of buffer in bytes
<code>status</code>	buffer to hold returned status

Return Value

Number of bytes returned in buffer, or zero if error.

`MgslReadWithStatus` returns received data and/or receive error indications to the application when available. `MgslReadWithStatus` is similar to `MgslRead` except receive errors can be returned in addition to data. Inspect the returned status value to determine if data is valid or an error was encountered.

```
unsigned char buf[4096];
int size = sizeof(buf);
int count;
int status;

/* get receive data or error indication */
count = MgslReadWithStatus(dev, buf, size, &status);
if (status == RxStatus_OK) {
    // count bytes of valid data returned
} else if (status == RxStatus_CrcError) {
    // CRC error (HDLC only)
} else if (status == RxStatus_Abort) {
    // receive idle or abort pattern (HDLC only)
} else if (status == RxStatus_ShortFrame) {
    // malformed/short frame (HDLC only)
} else if (status == RxStatus_Cancel) {
    // polled mode = no data or indication available
    // blocked mode = application canceled blocked call
}
```

Refer to `MgslRead` documentation in the previous section for more details. `MgslRead` is the preferred call unless an application processes receive errors or must inspect received CRC values. Refer to the receive-status sample program that is part of the HDLC sample project for more information on processing receive errors and inspecting received CRC values.

MGSLRESETTRACEBUFFERS

```
ULONG MgslResetTraceBuffers(HANDLE dev);
```

Arguments

<code>dev</code>	returned port handle
------------------	----------------------

Return Value

<code>ERROR_SUCCESS</code>	call success
<code>ERROR_INVALID_HANDLE</code>	port handle is invalid

This function discards all data in the API trace buffers for a serial device.

```
rc = MgslResetTraceBuffers(dev);
if (rc != ERROR_SUCCESS)
    /* call error */
```

MGSLSETIDLEMODE

```
ULONG MgslSetIdleMode(HANDLE dev, ULONG idle);
```

Arguments

<code>dev</code>	returned port handle
<code>idle</code>	idle or sync pattern

Return Value

<code>ERROR_SUCCESS</code>	call success
<code>ERROR_INVALID_HANDLE</code>	port handle is invalid

This function sets the idle or sync pattern. In bisync mode, this sets a 16-bit sync pattern. In monosync mode, this sets an 8-bit sync pattern. In other modes, this sets a pattern transmitted when there is no data to send.

<code>HDLC_TXIDLE_FLAG</code>	8-bit flag pattern (0x7e)
<code>HDLC_TXIDLE_ALT_ZEROS_ONES</code>	8-bit alternating zero and one pattern (0xaa)
<code>HDLC_TXIDLE_ZEROS</code>	8-bit all zero pattern (0x00)
<code>HDLC_TXIDLE_ONES</code>	8-bit all one pattern (0xff), also called HDLC abort
<code>HDLC_TXIDLE_CUSTOM8</code>	arbitrary 8-bit pattern in bits [7:0]
<code>HDLC_TXIDLE_CUSTOM16</code>	arbitrary 16-bit pattern in bits [15:0]

The following code demonstrates the call.

```

unsigned char syn1 = 0x32;
unsigned char syn2 = 0x32;

/* set 16-bit sync pattern for bisync mode */
rc = Mgs1SetIdleMode(dev, HDLC_TXIDLE_CUSTOM_16 | (syn2 << 8) | syn1);
if (rc != ERROR_SUCCESS)
    /* call error */

/* set 8-bit sync pattern for monosync mode */
rc = Mgs1SetIdleMode(dev, HDLC_TXIDLE_CUSTOM_8 | syn1);
if (rc != ERROR_SUCCESS)
    /* call error */

/* set flag idle for HDLC mode */
rc = Mgs1SetIdleMode(dev, HDLC_TXIDLE_FLAGS);
if (rc != ERROR_SUCCESS)
    /* call error */

```

MGSLSETGPIO

```
ULONG Mgs1SetGpio(HANDLE dev, GPIO_DESC *gpio);
```

Arguments

dev	returned port handle
gpio	pointer to GPIO structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	port handle is invalid
ERROR_INVALID_PARAMETER	invalid GPIO structure

This function sets general purpose I/O (GPIO) signal directions and states as described in a `GPIO_DESC` structure. Each bit of the structure fields represent a single signal, with GPIO #0 located in bit 0, GPIO #1 located in bit 1, etc. The number of GPIO signals depends on the specific hardware. Refer to the hardware user's guide for your hardware for a description of available GPIO signals.

```

GPIO_DESC gpio;
ULONG rc;

memset(&gpio, 0, sizeof(gpio));
gpio.dmask = (1 << 4); /* set direction of GPIO[4] */
gpio.dir   = (1 << 4); /* GPIO[4] set to output */
gpio.smask = (1 << 4); /* set state of GPIO[4] */
gpio.dir   = (1 << 4); /* GPIO[4] set high */

rc = Mgs1SetGpio(dev, &gpio);
if (rc != ERROR_SUCCESS)
    /* call error */

```

MGSLSetOption

```
ULONG Mgs1SetOption(HANDLE dev, UINT option, UINT value);
```

Arguments

dev	handle to open device
option	option identifier
value	new option value

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid option identifier or value

This function sets the specified configuration value. These settings take effect at application run time. Some of these settings can be set at driver load time (system boot) with `Mgs1SetPortConfigEx`. Settings that have an equivalent `Mgs1SetPortConfigEx` value are noted below.

Most `Mgs1SetOption` settings take effect immediately, some are deferred until the next call to `Mgs1SetParams`. Deferred options are identified in the descriptions below. Unless otherwise specified, the option takes effect immediately.

The following is a list of option identifiers.

MGSL_OPT_AUXCLK_ENABLE	Select AUXCLK output function. 1 = clock output (default), BRG enabled and used as clock source 0 = static low, BRG enabled for internal clock 2 = static high, BRG disabled (not compatible if internal clock required)
MGSL_OPT_CLOCK_BASE_FREQ	Set base clock frequency. Use only with hardware ordered with a custom fixed frequency oscillator or hardware with a frequency synthesizer configured for a custom base clock (PCI Express/USB). This option takes effect immediately and must be set before calling <code>Mgs1SetParams</code> so generated clocks are calculated correctly. See <code>fsynth.cpp</code> sample code for details.
MGSL_OPT_CLOCK_BASE_SOURCE	Select base clock source. Only USB and PCI Express devices have a frequency synthesizer. See <code>fsynth.cpp</code> sample code for details. MGSL_CLOCK_BASE_FIXED = Fixed frequency oscillator (default) MGSL_CLOCK_BASE_SYNTH = Frequency synthesizer
MGSL_OPT_DPLL_RESET	Controls DPLL reset behavior. 0 = no automatic DPLL reset 1 = automatically reset DPLL when enabling receiver 2 = one time manual DPLL reset
MGSL_OPT_ENABLE_LOCALLOOPBACK	Local Loopback output state. 0=off, 1=on, default=off
MGSL_OPT_ENABLE_REMOTELOOPBACK	Remote Loopback output state. 0=off, 1=on, default=off
MGSL_OPT_HALF_DUPLEX	0=disabled, 1=enabled, default=disabled When enabled, differential (RS422/V.35) outputs (TxD,AUXCLK,RTS,DTR) are enabled when sending data, otherwise outputs are disabled (high impedance). When sending data, the receive data input is ignored.
MGSL_OPT_INTERFACE	Select interface type (RS232, V.35, RS422) with <code>MGSL_INTERFACE_XXX</code> macros. Only valid for USB or SyncLink PCIe hardware. Set this option at driver load time with <code>Mgs1SetPortConfigEx</code> .

MGSL_OPT_MSB_FIRST	Serial bit order. 0=LSB first, 1=MSB first, default = LSB first This option does not take effect until the next call to <code>MgslSetParams</code> .
MGSL_OPT_NO_TERMINATION	Disable differential (RS-422/V.35) input termination. Hardware using jumpers or switches to control termination ignore this option. Set this option at driver load time with <code>MgslSetPortConfigEx</code> .
MGSL_OPT_RTS_DRIVER_CONTROL	0=disabled, 1=enabled, default=disabled When enabled, RTS signal state controls differential (RS422/V.35) output drivers. RTS on = outputs enabled RTS off = outputs disabled (high impedance) Cards without RTS maintain internal RTS state to support this feature. Set this option at driver load time with <code>MgslSetPortConfigEx</code> .
MGSL_OPT_RS422_OE	SyncLink USB/PCIe Only: 8 bit value controls RS422 output enable behavior and state for four serial signal outputs. [7] TXD OE Select (0=auto, 1=manual) [6] AUXCLK OE Select (0=auto, 1=manual) [5] DTR OE Select (0=auto, 1=manual) [4] RTS OE Select (0=auto, 1=manual) [3] TXD OE Manual State (0=tristate/disabled, 1=enabled) [2] AUXCLK OE Manual State (0=tristate/disabled, 1=enabled) [1] DTR OE Manual State (0=tristate/disabled, 1=enabled) [0] AUXCLK OE Manual State (0=tristate/disabled, 1=enabled). Select automatic or manual behavior. Automatic behavior is always enabled unless half duplex or RTS control selected. Manual behavior sets output enable state to that selected by manual state bits.
MGSL_OPT_RX_DISCARD_TOO_LARGE	0=disabled, 1=enabled, default = disabled Silently discard receive frames larger than <code>MgslReceive</code> buffer.
MGSL_OPT_RX_ERROR_MASK	0=disabled, 1=enabled, default=disabled Silently discard HDLC receive frames with errors (CRC, etc).
MGSL_OPT_RX_COUNT	Read only value indicates number of bytes in receive buffers.
MGSL_OPT_TX_COUNT	Read only value indicates number of bytes in send buffers.
MGSL_OPT_RX_POLL	Enable polling mode for <code>MgslReceive</code> calls. When enabled, <code>MgslReceive</code> returns <code>ERROR_BUSY</code> and cancels the request instead of <code>ERROR_IO_PENDING</code> if no data is available to return.
MGSL_OPT_TX_POLL	Enable polling mode for <code>MgslTransmit</code> calls. When enabled, <code>MgslTransmit</code> returns <code>ERROR_BUSY</code> and cancels the request instead of <code>ERROR_IO_PENDING</code> if no send buffers are free to hold data.
MGSL_OPT_TX_IDLE_COUNT	Statistic value incremented when transmitter becomes idle after sending data. Clear the count by calling <code>MgslSetOption</code> with value of 0.
MGSL_OPT_TDM	32-bit value containing TDM options. See TDM protocol section and sample code for details.
MGSL_OPT_UNDERRUN_COUNT	Statistic value incremented for each transmitter underrun. Clear the count by calling <code>MgslSetOption</code> with value of 0.
MGSL_OPT_XSYNC	Extended Sync (XSync) protocol synchronization pattern.
MGSL_OPT_XCTRL	Extended Sync (XSync) protocol options. See XSync sample code for details. xctrl[31:19] reserved, must be zero xctrl[18:17] sync size 00=1 byte, 01=2 bytes, 10=3 bytes, 11= 4bytes xctrl[16] 1=fixed size receive blocks as specified in xctrl[15:0] xctrl[15:0] receive block size – 1 (0 to 65535 = 1 to 65536 bytes)

```

/* set RTS to control serial output state (on or tristate) */
rc = Mgs1SetOption(dev, MGSL_OPT_RTS_DRIVER_CONTROL, 1);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetOption error=%d\n", rc);

```

MGSLSETPARAMS

```

ULONG Mgs1SetParams(HANDLE dev, MGSL_PARAMS *params);

```

Arguments

dev	handle to open device
params	pointer to configuration structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid option identifier or value

This function sets the specified configuration values. The options are usually set once per communications session. Calling this function resets the transmitter and receiver, discarding all buffered data. Refer to the `MGSL_PARAMS` structure section for a description of the fields and values.

```

MGSL_PARAMS params;

/* initialize structure with desired settings */

rc = Mgs1SetParams(dev, &params);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetParams error=%d\n", rc);

```

MGSLSETPORTCONFIGEX

```

ULONG Mgs1SetPortConfigEx(ULONG port_id, MGSL_PORT_CONFIG_EX *config);

```

Arguments

port_id	integer port identifier
config	pointer to configuration structure

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid option identifier or value

This function sets the specified configuration values that are used at driver load time. Refer to the `MGSL_PORT_CONFIG_EX` structure section for a description of the fields and values. These options are usually set in the Windows Device Manager, but this call is implemented so the options can be set programmatically.

```

MGSL_PORT_CONFIG_EX config;
unsigned int port_id;

/* operate on port 3 of first adapter */
port_id = MGSL_MAKE_PORT_ID(1,3);

/* initialize structure with desired settings */

rc = Mgs1SetPortConfigEx(port_id, &config);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetPortConfigEx error=%d\n", rc);

```

MGSLSETSERIALSIGNALS

```
ULONG Mgs1SetSerialSignals(HANDLE dev, UCHAR *signals);
```

Arguments

dev	handle to open device
signals	new serial signals value

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle

This function sets the serial control output states using `SerialSignal_XXX` macros defined in the `mghdlc.h` header file. A set bit indicates an active signal. Depending on the application, some signals may not be used or may be used for a non-standard purpose.

<code>SerialSignal_DTR</code>	Data Terminal Ready output (serial device is active)
<code>SerialSignal_RTS</code>	Request to Send output (serial device wants to send data)

```

/* turn on both DTR and RTS output signals */
rc = Mgs1SetSerialSignals(dev, SerialSignal_RTS + SerialSignal_DTR);
if (rc != ERROR_SUCCESS)
    printf("Mgs1SetSerialSignals error=%d\n", rc);

```

MGSLSETTRACELEVEL

```
ULONG Mgs1SetTraceLevel(HANDLE dev, ULONG *level);
```

Arguments

dev	handle to open device
level	new trace level value

Return Value

ERROR_SUCCESS	call success
ERROR_INVALID_HANDLE	invalid device handle

This function sets the current trace level for a serial device using `TraceLevel_XXX` macros. Most applications do not use this call. The `mgsltrc.exe` trace utility provided with the Serial API uses this call.

TraceLevel_API	API calls recorded
TraceLevel_Status	serial input (CTS, DCD, DSR, RI) events recorded
TraceLevel_Transmit	transmit events (underrun, completion) recorded
TraceLevel_Receive	receive events (overrun, completion, idle) recorded
TraceLevel_Data	send and receive data contents recorded
TraceLevel_DataLink	high level link layer events (SNRM, RR, etc) recorded
TraceLevel_Error	driver error conditions recorded
TraceLevel_Info	driver general events recorded
TraceLevel_Detail	driver detailed operation recorded

MGSLTRANSMIT

```
ULONG Mgs1Transmit(HANDLE dev, UCHAR *buf, ULONG size, ULONG *status, OVERLAPPED *ol);
```

Arguments

dev	handle to open device
buf	send data buffer
size	number of bytes in send data buffer (may be zero, see below)
status	returned send status (may be NULL, see below)
ol	pointer to Windows overlapped structure used for asynchronous notification of send completion

Return Value

ERROR_SUCCESS	call success
ERROR_IO_PENDING	waiting for call to complete (monitor overlapped structure)
ERROR_BUSY	previous Mgs1Transmit call is pending
ERROR_INVALID_HANDLE	invalid device handle

This function performs one of three functions depending on the arguments:

If `status` is NULL, call completes after buffering data to send. This is the typical case.

If `status` is not NULL, call completes after sending data and `status` indicates success or error.

If `size` is zero, call completes when previously buffered data is sent.

ERROR_SUCCESS indicates immediate completion and ERROR_IO_PENDING indicates the application should monitor the `hEvent` member of the overlapped structure for indication of call completion. Call `Mgs1CancelTransmit` to cancel a pending call before normal completion. When the cancellation completes, the `event` member of the overlapped structure is signaled.

If `size` is not zero, this call enables the transmitter. The transmitter remains enabled sending the idle pattern after all data is sent.

These values are returned if the `status` argument is not NULL. Values other than `TxStatus_OK` indicate data was not sent. Send status is rarely required by an application. Requiring status for each call prevents a continuous flow of data and reduces throughput.

TxStatus_OK	data sent successfully
TxStatus_Underrun	hardware not supplied data fast enough
TxStatus_Cancel	user cancelled request before data sent
TxStatus_CtsFailure	CTS went inactive in middle of sending data

Call `MgslGetOption` with `MGSL_OPT_TX_COUNT` to get the number of buffered send bytes. This is useful for maintaining a continuous flow of data while limiting output latency by controlling the amount of buffered data.

Below is sample code demonstrating `MgslTransmit`.

```
int rc;
int size = 1024;
unsigned char buf[1024];
OVERLAPPED ol;

ol.hEvent = CreateEvent(NULL, TRUE, FALSE, NULL);

/* submit buffer to send, do not wait for send completion */
ResetEvent(ol.hEvent);
rc = MgslTransmit(dev, buf, size, NULL, &ol);
if (rc == ERROR_IO_PENDING) {
    /* wait for completion (wait for free buffer) */
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS) {
    /* MgslTransmit error */
} else {
    /* immediate completion */
}

/* wait for previously submitted data to be sent */
ResetEvent(ol.hEvent);
rc = MgslTransmit(dev, NULL, 0, NULL, &ol);
if (rc == ERROR_IO_PENDING) {
    /* wait for completion (wait for all sent) */
    rc = WaitForSingleObject(ol.hEvent, INFINITE);
    if (rc != WAIT_OBJECT_0)
        /* wait error */
} else if (rc != ERROR_SUCCESS) {
    /* MgslTransmit error */
} else {
    /* immediate completion */
}
```

Polled Mode

Use `MgslSetOption` with `MGSL_OPT_TX_POLL` to enable polled mode. When enabled, `MgslTransmit` returns `ERROR_BUSY` and cancels the request instead of returning `ERROR_IO_PENDING` if no free buffers are available to hold data. The application must still allocate an overlapped structure and event even when using polled mode.

MGSLWAITEVENT

```
ULONG MgsWaitEvent(HANDLE dev, ULONG mask, ULONG *events, OVERLAPPED *ol);
```

Arguments

dev	handle to open device
mask	bit flags indicating events of interest
events	returned event bit flags
ol	pointer to Windows overlapped structure used for asynchronous notification of request completion

Return Value

ERROR_SUCCESS	call success
ERROR_IO_PENDING	waiting for call to complete (monitor overlapped structure)
ERROR_BUSY	a previous MgsWaitEvent call is still pending
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid events buffer

This function waits for one or more events specified in the `mask` argument to occur. When the first specified event occurs, the call completes with the `events` argument set to reflect which events occurred.

The return code `ERROR_SUCCESS` indicates immediate completion and `ERROR_IO_PENDING` indicates the application should monitor the event member of the overlapped structure for indication of call completion.

Call `MgsCancelWaitEvent` to stop a pending call to `MgsWaitEvent`. When the cancellation is complete, the event member of the overlapped structure will be signaled.

Both `mask` and `events` use the following bit flags defined in the `mghdlc.h` header file:

<code>MgsEvent_DsrActive</code>	wait for DSR (Data Set Ready) input active
<code>MgsEvent_DsrInactive</code>	wait for DSR (Data Set Ready) input inactive
<code>MgsEvent_CtsActive</code>	wait for CTS (Clear to Send) input active
<code>MgsEvent_CtsInactive</code>	wait for CTS (Clear to Send) input inactive
<code>MgsEvent_DcdActive</code>	wait for DCD (Data Carrier Detect) input active
<code>MgsEvent_DcdInactive</code>	wait for DCD (Data Carrier Detect) input inactive
<code>MgsEvent_RiActive</code>	wait for RI (Ring Indicator) input active
<code>MgsEvent_RiInactive</code>	wait for RI (Ring Indicator) input inactive
<code>MgsEvent_ExitHuntMode</code>	wait for receiver to become active (sync/flag detect)
<code>MgsEvent_IdleReceived</code>	wait for receiver to become idle (idle detect)

MGSLWAITGPIO

```
ULONG MgsWaitGpio(HANDLE dev, GPIO_DESC *gpio, OVERLAPPED *ol);
```

Arguments

dev	handle to open device
gpio	pointer to GPIO structure
ol	pointer to Windows overlapped structure used for asynchronous notification of request completion

Return Value

ERROR_SUCCESS	call success
ERROR_IO_PENDING	waiting for call to complete (monitor overlapped structure)
ERROR_BUSY	a previous MgsWaitGpio call is still pending
ERROR_INVALID_HANDLE	invalid device handle
ERROR_INVALID_PARAMETER	invalid events buffer

This function waits for one or more GPIO signals to reach the specified states. When the first specified state occurs, the call completes with the `gpio` argument set to reflect the current GPIO signal states. The `dmask` and `dir` fields of the `GPIO_DESC` structure are ignored.

Before making this call, set the bits in the `smask` field of the `GPIO_DESC` structure to indicate which GPIO signals to monitor. Set bits in the `state` field to specify the target state.

When the call completes, the `state` member of the `GPIO_DESC` structure is set to indicate the state of all signals at the time the first target state is reached.

The return code `ERROR_SUCCESS` indicates immediate completion and `ERROR_IO_PENDING` indicates the application should monitor the event member of the overlapped structure for indication of call completion.

Call `MgsCancelWaitGpio` to stop a pending call to `MgsWaitGpio`. When the cancellation is complete, the event member of the overlapped structure will be signaled.

MGSLWAITALLSENT

```
int MgsWaitAllSent(HANDLE dev);
```

Arguments

dev	open port handle
-----	------------------

Return Value

Zero if success (all data sent) or error code (timeout or other error).

Check data passed to `MgsWrite` or `MgsTransmit` has been completely sent. Use this call to determine when it is safe to reset the transmitter or close the port.

```

int rc;

/* check for all buffered data sent */
rc = MgsWaitAllSent(dev);
if (!rc) {
    /* all buffered data sent, safe to close port */
} else {
    /* busy sending buffered data (polled) or error (blocking) */
}

```

MgsWaitAllSent is a simplified wrapper function for the original MgsTransmit function. If you do not need the extra error reporting and asynchronous notification features of MgsTransmit, use the simpler MgsWaitAllSent.

Blocking and Polled Modes

MgsWaitAllSent can be used in blocking or polled mode by calling MgsSetOption with the MGSL_OPT_TX_POLL identifier. When this option is enabled, MgsWaitAllSent returns immediately with a return code of zero when all data is sent or non zero if still sending (polled mode). When not enabled, MgsWaitAllSent blocks until all buffered data has been sent (blocking mode = default).

A blocked call to MgsWaitAllSent can be aborted by calling MgsCancelTransmit from a different application thread.

```

int rc;

/* submit send data to API with MgsTransmit or MgsWrite */

/* block until all data is completely sent (INFINITE = no timeout) */
rc = MgsWaitAllSent(dev, INFINITE);
if (!rc) {
    /* all data sent */
} else {
    /* timeout or error */
}

/* safe to close port or wait for response */

```

The amount of queued send data in the API buffers is obtained using MgsGetOption with MGSL_OPT_TX_COUNT.

MGSLWRITE

```
int MgsWrite(HANDLE dev, unsigned char *buf, int size);
```

Arguments

dev	open port handle
buf	pointer to buffer containing send data
size	size of send data in bytes

Return Value

Number of bytes accepted by API or zero if buffers are full (polled) or error (blocking).

MgslWrite is a simplified wrapper function for MgslTransmit which passes data to the API to send as soon as possible.

```
unsigned char buf[4096];
int size = sizeof(buf);
int count;

/* initialize buffer with application data */

/* send data */
count = MgslWrite(dev, buf, size);
if (count) {
    /* count bytes stored in API send buffers */
} else {
    /* API send buffers full (polled) or error (blocked) */
}
```

The amount of queued send data in the API buffers is obtained using MgslGetOption with MGSL_OPT_TX_COUNT.

A blocked call to MgslWrite can be aborted by calling MgslCancelTransmit from a different application thread.

The format of the data depends on the protocol. For frame oriented SDLC/HDLC, each call sends a single frame of data. For other protocols, each call sends data with no formatting, requiring the application to implement message boundaries.

Blocking and Polled Modes

MgslWrite can be used in blocking or polled mode by calling MgslSetOption with the MGSL_OPT_TX_POLL identifier. When this option is enabled, MgslWrite returns immediately with a return code of zero when all API send buffers are full (polling mode). When not enabled, MgslWrite blocks until API send buffers are available and data is accepted (blocking mode = default).

A blocked call to MgslWrite can be aborted by calling MgslCancelTransmit from a different application thread.

STRUCTURES

The following C language structures are defined in the `mghdlc.h` header file. This header should be included in source files that access the serial API.

GPIO_DESC

This structure is used with the general purpose I/O (GPIO) API calls `MgslGetGpio`, `MgslSetGpio`, `MgslWaitGpio`.

```
typedef struct _GPIO_DESC
{
    UINT state;
    UINT smask;
    UINT dir;
    UINT dmask;
} GPIO_DESC;
```

All fields are 32 bits. Each bit represents an I/O signal. I/O signal 0 is bit 0, signal 1 is bit 1, etc.

<code>state</code>	each bit represents the state of an I/O signal
<code>smask</code>	specifies which bits in state field are used
<code>dir</code>	each bit specifies the direction of an I/O signal (0=input, 1=output)
<code>dmask</code>	specifies which bits in dir field are used

MGSL_ASSIGNED_RESOURCES

This structure is used with the API call `MgslGetAssignedResources`. Only the `SerialNumber` field is used. All other fields are unused and undefined. The serial number is only available for SyncLink USB devices.

```
typedef struct _MGSL_ASSIGNED_RESOURCES
{
    ULONG BusType;
    ULONG BusNumber;
    ULONG DeviceNumber;
    ULONG IrqLevel;
    ULONG DmaChannel;
    ULONG IoAddress1;
    ULONG IoAddress2;
    ULONG IoAddress3;
    ULONG MemAddress1;
    ULONG MemAddress2;
    ULONG MemAddress3;
    USHORT DeviceId;
    USHORT SubsystemId;
    char SerialNumber[MGSL_MAX_SERIAL_NUMBER];
} MGSL_ASSIGNED_RESOURCES, *PMGSL_ASSIGNED_RESOURCES;
```

MGSL_PARAMS

This structure is used with the configuration API calls `MgslGetParams` and `MgslSetParams`.

```
typedef struct _MGSL_PARAMS
{
    /* common */

    ULONG      Mode;          /* HDLC, asynchronous, raw, bisync, monosync */
    UCHAR      Loopback;      /* internal loopback mode */
    USHORT     Flags;         /* bit field flags */

    /* synchronous modes */

    UCHAR      Encoding;      /* serial encoding NRZ, NRZI, etc. */
    ULONG      ClockSpeed;    /* external clock speed in bits per second */
    UCHAR      Addr;          /* rx HDLC address filter, 0xFF = disable */
    USHORT     CrcType;       /* None, CRC16-CCITT, or CRC32-CCITT */
    UCHAR      PreambleLength;
    UCHAR      PreamblePattern;

    /* asynchronous mode */

    ULONG      DataRate;      /* async data rate in bits per second */
    UCHAR      DataBits;      /* 5 to 8 */
    UCHAR      StopBits;      /* 1 or 2 */
    UCHAR      Parity;        /* none, even, odd */
} MGSL_PARAMS, *PMGSL_PARAMS;
```

Note: Many constants are defined as `HDLC_XXX` for historical reasons, but apply to all synchronous modes (raw/bisync/monosync/HDLC).

MODE

The mode field of the `MGSL_PARAMS` structure specifies the communications protocol with an `MGSL_MODE_XXX` macro. The mode determines the framing, synchronization, transparency and clocking characteristics of the serial protocol.

MGSL_MODE_ASYNC	character oriented no external clocks per character hardware framing per character parity check (none/even/odd) used for isochronous mode when <code>DataRate</code> set to 0
MGSL_MODE_HDLC	bit synchronous hardware framing and synchronization (flags) hardware transparency (0 bit stuff/removal) hardware CRC check/generation (none/16 bit/32 bit) Note: SDLC and HDLC are the same in this context

MGSL_MODE_RAW	bit synchronous no hardware framing no hardware synchronization no hardware transparency
MGSL_MODE_BISYNC	byte synchronous 16 bit hardware synchronization no hardware framing no hardware transparency
MGSL_MODE_MONOSYNC	byte synchronous 8 bit hardware synchronization no hardware framing no hardware transparency
MGSL_MODE_TDM	SyncLink PCIe, GT4e, USB Only byte synchronous external control signal synchronization external clock signal framing defined by fixed size data grouping

LOOPBACK

The `Loopback` field of the `MGSL_PARAMS` structure enables or disables the internal loopback mode. 0 = normal operation, 1 = loopback mode. When enabled, the transmit data signal is connected internally to the receive data signal and clocks are generated internally by the baud rate generator as specified by the `ClockSpeed` field. Use internal loopback mode to test the operation of the serial controller without external line drivers or devices.

FLAGS

The `Flags` field of the `MGSL_PARAMS` structure specifies options using `HDLC_FLAG_XXX` macros. The `HDLC_FLAG` prefix is used for historical reasons. Unless otherwise specified flags apply to all modes.

Receive Clock Source

These flags select the serial receive clock source.

HDLC_FLAG_RXC_DPLL	Recover receive clock from receive data signal. Specify data rate in <code>ClockSpeed</code> member of <code>MGSL_PARAMS</code> structure.
HDLC_FLAG_RXC_BRG	Generate receive clock internally. Specify data rate in <code>ClockSpeed</code> member of <code>MGSL_PARAMS</code> structure.
HDLC_FLAG_RXC_RXCPIN	Receive clock is supplied by external device on RxCPIN input pin.
HDLC_FLAG_RXC_TXCPIN	Receive clock is supplied by external device on TxCPIN input pin.
HDLC_FLAG_RXC_INV	Combine with receive clock source flag to invert receive clock polarity.

Transmit Clock Source

These flags select the serial transmit clock source.

HDLC_FLAG_TXC_DPLL	Recover transmit clock from receive data signal. Specify data rate in <code>ClockSpeed</code> member of <code>MGSL_PARAMS</code> structure.
HDLC_FLAG_TXC_BRG	Generate transmit clock internally. Specify data rate in <code>ClockSpeed</code> member of <code>MGSL_PARAMS</code> structure.
HDLC_FLAG_TXC_RXCPIN	Transmit clock is supplied by external device on RxC input pin.
HDLC_FLAG_TXC_TXCPIN	Transmit clock is supplied by external device on TxC input pin.
HDLC_FLAG_TXC_INV	Combine with transmit clock source flag to invert transmit clock polarity.

Digital Phase Lock Loop Divisor

The DPLL is used to recover a clock from the receive data signal. This is done using a sample/reference clock that is a multiple of the expected data rate. This multiple is either 8 or 16. A higher sample rate (larger divisor) results in a more accurate recovered clock. A lower divisor allows a higher expected data rate. The sample clock is limited by the base clock frequency (default 14.7456MHz).

HDLC_FLAG_DPLL_DIV8 data rate = reference clock divided by 8

HDLC_FLAG_DPLL_DIV16 data rate = reference clock divided by 16

Miscellaneous Flags (any combination allowed)

HDLC_FLAG_AUTO_CTS	Enable transmitter only when CTS input is active.
HDLC_FLAG_AUTO_DCD	Enable receiver only when DCD input is active.
HDLC_FLAG_AUTO_RTS	When set, the driver automatically asserts RTS at when sending data and negates RTS when done sending. If RTS is active when a transmit request is made, the driver will not manipulate the state of RTS.

ENCODING

Encoding specifies physical data signal representation of logical 1 or 0. Equivalent encoding names are shown in parenthesis. Use BIPHASE encodings with clock recovery to guarantee one data transition per bit. The levels specified below are the data signal from the serial controller, but before the line drivers which invert the signal.

HDLC_ENCODING_NRZ	NRZ (NRZ-L) unencoded data signal. high = 1, low = 0
HDLC_ENCODING_NRZB	NRZB inverted data signal. high = 0, low = 1
HDLC_ENCODING_NRZI_MARK	(NRZ-M) invert at start of bit if 1
HDLC_ENCODING_NRZI_SPACE	(NRZI or NRZ-S) invert at start of bit if 0
HDLC_ENCODING_BIPHASE_MARK	(FM1) invert at start of bit, invert at bit center if 1
HDLC_ENCODING_BIPHASE_SPACE	(FM0) invert at start of bit, invert at bit center if 0
HDLC_ENCODING_BIPHASE_LEVEL	(Manchester) start of bit: high=1, low=0, invert at bit center
HDLC_ENCODING_DIFF_BIPHASE_LEVEL	invert at start of bit if 1, invert at bit center

CLOCKSPEED

The `ClockSpeed` field of the `MGSL_PARAMS` structure specifies the data rate of the generated (BRG) or recovered (DPPL) clock. A clock generated by the BRG is output on the AUXCLK output pin for use by an external device. Set to zero to disable clock generation.

The clock is generated by dividing a fixed base clock by a 16-bit integer divisor:

$$\text{divisor} = (\text{base clock}/\text{data rate}) - 1$$

Only discrete rates can be generated exactly because the divisor is a 16-bit integer. The default base clock of 14.7456MHz allows exact generation of common rates: 9600, 57600, 115200 etc.

The serial card can be purchased with a different base clock. This option is installed at the factory. When a base clock other than 14.7456MHz is installed, the driver must be configured to use the new value by calling `MgslSetOption(MGSL_OPT_CLOCK_BASE_FREQ)`.

ADDR

8-bit address field that filters received SDLC/HDLC frames. 0xFF = return all frames (no filtering), otherwise discard received HDLC frames with addresses other than 0xFF (broadcast) or `Addr` value.

CRC TYPE

The `CrcType` member of the `MGSL_PARAMS` structure specified the frame check type used with SDLC/HDLC frames. The selected Cyclic Redundancy Check (CRC) code is appended to sent frames and verified on receive frames.

<code>HDLC_CRC_NONE</code>	Don't send CRCs on transmit, don't check CRCs on receive.
<code>HDLC_CRC_16_CCITT</code>	16 bit CRC Polynomial: $x^{16} + x^{12} + x^5 + 1$ init value = all 1s
<code>HDLC_CRC_32_CCITT</code>	32 bit CRC Polynomial: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ init value = all 1s
<code>HDLC_CRC_RETURN_CRC</code>	combine with <code>HDLC_CRC_16_CCITT</code> or <code>HDLC_CRC_32_CCITT</code> to return CRC value (2 or 4 bytes) to application as the final bytes in the receive frame buffer
<code>HDLC_CRC_RETURN_CRCERR_FRAME</code>	combine with <code>HDLC_CRC_16_CCITT</code> or <code>HDLC_CRC_32_CCITT</code> to return received frames with CRC errors to application

CRC ERRORS AND ACCESSING

CRC CODES

By default HDLC frames with CRC errors are discarded. The API can be configured to return both good frames and frames with CRC errors to the application. Frames with CRC errors are identified by the status code in the receive request structure.

Add the `HDLC_CRC_RETURN_CRCERR_FRAME` flag to the CRC type in the `MGSL_PARAMS` structure to receive both good and bad frames as shown below.

```
params.CrcType = HDLC_CRC_16_CCITT | HDLC_CRC_RETURN_CRCERR_FRAME;
```

By default the API strips the CRC value from a received frame and only passes back the data. Add the `HDLC_CRC_RETURN_CRC` flag to the CRC type in the `MGSL_PARAMS` structure to append the 2 or 4 byte CRC value to the end of the frame passed back to the application.

```
params.CrcType = HDLC_CRC_16_CCITT | HDLC_CRC_RETURN_CRC;
```

These two CRC flags can be combined or used independently to achieve the desired behavior. The following returns all frames, regardless of CRC errors, and appends the CRC value to the end of the frame buffer.

```
params.CrcType = HDLC_CRC_16_CCITT | HDLC_CRC_RETURN_CRCERR_FRAME |  
                HDLC_CRC_RETURN_CRC;
```

PREAMBLEPATTERN

A preamble is a pattern sent before an SDLC/HDLC frame. The usual purpose of a preamble is to synchronize a remote DPLL that is recovering clock information from a data stream. The length of the preamble is specified in the `PreambleLength` member of the `MGSL_PARAMS` structure.

<code>HDLC_PREAMBLE_PATTERN_NONE</code>	no preamble (preamble_length ignored)
<code>HDLC_PREAMBLE_PATTERN_ZEROS</code>	all zeroes
<code>HDLC_PREAMBLE_PATTERN_FLAGS</code>	all flags
<code>HDLC_PREAMBLE_PATTERN_10</code>	alternating 1's and 0's
<code>HDLC_PREAMBLE_PATTERN_01</code>	alternating 0's and 1's
<code>HDLC_PREAMBLE_PATTERN_ONES</code>	all ones

PREAMBLELENGTH

This member of the `MGSL_PARAMS` structure selects the length in bytes of the preamble pattern using the following macros:

```
HDLC_PREAMBLE_LENGTH_8BITS
HDLC_PREAMBLE_LENGTH_16BITS
HDLC_PREAMBLE_LENGTH_32BITS
HDLC_PREAMBLE_LENGTH_64BITS
```

DATARATE

This member of the `MGSL_PARAMS` structure selects the data rate for asynchronous operation. In asynchronous mode the clock is generated internally by the BRG at x8 or x16 the data rate specified in this field. The data rate must match that of the remote device.

DATABITS

This member of the `MGSL_PARAMS` structure selects the number of data bits per character for asynchronous operation. This value must match that of the remote device. Valid values are 5 to 8.

STOPBITS

This member of the `MGSL_PARAMS` structure selects the number of stop bits transmitted per character for asynchronous operation. Valid values are 1 or 2. SyncLink device receivers always recognize 1 or 2 stop bits automatically without configuration.

PARITY

This member of the `MGSL_PARAMS` structure selects the character parity for asynchronous operation. This value must match that of the remote device. When enabled, the selected parity bit is added to transmitted characters and verified on received characters. Parity bits are stripped from received data before data is returned to an application. Characters received with a parity error are discarded. Valid values are specified with the following macros:

`ASYNC_PARITY_NONE` no parity used
`ASYNC_PARITY_EVEN` 0 or 1 bit added to maintain even number of 1 bits in character
`ASYNC_PARITY_ODD` 0 or 1 bit added to maintain odd number of 1 bits in character

MGSL_PORT_CONFIG_EX

This structure is used with the configuration API calls `MgslGetPortConfigEx` and `MgslSetPortConfigEx`. The options specified with this structure take effect at driver load time. These options can also be set in the Windows Device Manager.

```
typedef struct _MGSL_PORT_CONFIG_EX
{
    ULONG BaseAddress; /* obsolete, ignored */
    ULONG IrqLevel;    /* obsolete, ignored */
    ULONG DmaChannel;  /* obsolete, ignored */
    ULONG BusType;     /* obsolete, ignored */
    ULONG BusNumber;   /* obsolete, ignored */
    ULONG DeviceID;    /* obsolete, ignored */
    ULONG MaxFrameSize; /* max number of bytes per HDLC frame */
    ULONG Flags;       /* various bit flag options */
} MGSL_PORT_CONFIG_EX, *PMGSL_PORT_CONFIG_EX;
```

MaxFrameSize

Set the maximum number of bytes expected per frame of data. Received frames larger than this value are discarded. This also controls the maximum number of bytes accepted per `MgslTransmit` call.

Flags

This member controls the serial interface physical characteristics.

Hardware using jumpers or switches to select interface type, including GT series, ignore `MGSL_INTERFACE_XXX` flags.

<code>MGSL_INTERFACE_MASK</code>	a bit mask that covers all bits used to select interface type
<code>MGSL_INTERFACE_DISABLE</code>	serial interface is disabled (high impedance)
<code>MGSL_INTERFACE_RS232</code>	serial interface is RS232 (single ended)
<code>MGSL_INTERFACE_V35</code>	serial interface is V.35 (mixed differential/single ended)
<code>MGSL_INTERFACE_RS422</code>	serial interface is RS422/485 (differential)

MGSL_NO_TERMINATION	when set, input termination is disabled for differential modes when clear, differential inputs use 120 Ohm termination Hardware using jumpers or switches to control input termination, including GT series, ignore this flag.
MGSL_RTS_DRIVER_CONTROL	when set, RTS controls differential (RS422/V.35) outputs RTS on = outputs enabled RTS off = outputs disabled (tri-state, high impedance) Cards without RTS maintain internal RTS state to support this feature.

MGSL_RECEIVE_REQUEST

This structure is used with the `MgslReceive` API call.

```
typedef struct _MGSL_RECEIVE_REQUEST
{
    ULONG    Status;           /* returned request status */
    ULONG    DataLength;       /* returned data length */
    Char     DataBuffer[1];    /* variable length data buffer */
} MGSL_RECEIVE_REQUEST, *PMGSL_RECEIVE_REQUEST;
```

The structure defines a header for a receive request. Memory for a receive request should include the size of data buffer required by the application. For example, the following code allocates a receive request with a 4096 byte buffer:

```
MGSL_RECEIVE_REQUEST *req;

req = (MGSL_RECEIVE_REQUEST*)malloc(sizeof(MGSL_RECEIVE_REQUEST) + 4096);
```

The `DataLength` field of the structure should be initialized before **every** call to `MgslReceive` to indicate the size of the buffer.

```
req->DataLength = 4096;
rc = MgslReceive(hDevice, req, 01);
```

On request completion, the `Status` field indicates success or error. Most error codes apply only to SDLC/HDLC mode. Unless the application is maintaining error statistics, the specific error code can be ignored. When `Status` is `RxStatus_OK`, data is returned in `DataBuffer` and `DataLength` is set to the number of returned bytes.

<code>RxStatus_OK</code>	data returned in <code>DataBuffer</code> , <code>DataLength</code> set to count of bytes
<code>RxStatus_CrcError</code>	SDLC/HDLC frame with a CRC error was received and discarded
<code>RxStatus_FifoOverrun</code>	hardware FIFO was full when data was received causing data loss
<code>RxStatus_ShortFrame</code>	SDLC/HDLC frame with only one data byte was received and discarded
<code>RxStatus_Abort</code>	SDLC/HDLC abort or idle sequence was received
<code>RxStatus_BufferOverrun</code>	API buffers were full when data was received causing data loss
<code>RxStatus_Cancel</code>	receive request was cancelled by user before normal completion
<code>RxStatus_BufferTooSmall</code>	SDLC/HDLC frame larger than the receive request buffer was received

C#

The module `mgapi.cs` provides a C# interface to SyncLink serial devices and is used to build the `mgapi.dll` C# API assembly. It is built on top of the C API and provides both a C# optimized interface and a C API wrapper. New C# applications should use the C# optimized interface. The C wrapper is used for quickly porting C applications to C# with minimal changes. The wrapper follows the naming conventions of the C API. This reduces the effort needed to port existing code.

Note: The API module has been tested with C# version 4 and .NET Framework 4

Samples are Visual Studio solutions/projects. Visual Studio 2010SP1 or later is required. Samples have been tested with Visual Studio 2010SP1 and Visual Studio 2022.

A C# Visual Studio solution is provided containing `mgapi.cs` and sample programs for each protocol. The API assembly must be added to application references in the Visual Studio project.

```
// C# API (new C# apps)
using Port = Microgate.SerialApi.Port;

// C# wrapper of C API (porting C apps)
// Microgate namespace contains SerialApi class
using Microgate;
```

Contents of `mgapi\csharp` directory:

<code>csharp.sln</code>	Visual Studio solution containing sample projects
<code>mgapi\</code>	C# API module
<code>hdlc\</code>	HDLC/SDLC sample
<code>async\</code>	asynchronous/isochronous sample
<code>bisync\</code>	bisync/monosync sample
<code>raw\</code>	raw bitstream sample
<code>tdm\</code>	TDM sample
<code>2wire\</code>	2-wire (bussed) sample
<code>signals\</code>	control/status signal sample

A C# sample is run as shown below for the HDLC sample on device `mghdlc1`:

```
C:\mgapi\csharp\hdlc\bin\x64\Release>hdlc mghdlc1
```

PORT OBJECT

A `Port` object is required for most tasks. A port name must be supplied. Port names for single port adapters have the form `MGHDLCx`, where 'x' is the adapter number. Port names for multiport adapters have the form `MGMPxPy`, where x is the adapter number and 'y' is the port number. The `Port` class function `enumerate` returns a list of available port names.

```
string[] names = Port.enumerate();
foreach (string name in names)
    Console.WriteLine(name);

port = new Port("MGHDL1");
```

OPEN/CLOSE

The `Port` method `open` claims a port for use and raises an exception if an error occurs. The `Port` method `is_open` determines if the port is open. Other attributes, properties and methods must be accessed only when the port is open.

```
try {
    port.open();
}
catch (FileNotFoundException) {
    Console.WriteLine("port not found");
}
catch (UnauthorizedAccessException) {
    Console.WriteLine("access denied or port in use");
}
catch (Win32Exception) {
    Console.WriteLine("open error");
}
```

The `Port` method `close` releases the port from use. A port must be closed to allow use by other processes. Ports are automatically closed when a process exits or the `Port` object is deallocated.

```
port.close();
```

CONFIGURATION

A port must be configured to match application specific requirements. Perform initial configuration with a `Port.Settings` object. The `Port` method `apply_settings()` cancels current operations and shuts down the hardware.

```
var settings = new Port.Settings();
settings.protocol = Port.HDLC;
settings.encoding = Port.MANCHESTER;
settings.crc = Port.CRC16;
settings.transmit_clock = Port.INTERNAL;
settings.receive_clock = Port.RECOVERED;
settings.internal_clock_rate = 9600;
settings.transmit_preamble_pattern = 0x7e;
settings.transmit_preamble_bits = 8;
port.apply_settings(settings);
```

During operation, alter port behavior using various properties.

```
// set pattern sent when transmitter is enabled but
// there is no data to send. HDLC flag = 0x7e
port.transmit_idle_pattern = 0x7e;
```

Set persistent default state that spans system restart and device ejection.

```
// RTS controls RS422 output enable
Port.Defaults defaults = port.get_defaults();
defaults.rts_output_enable = true;
port.set_defaults(defaults);
```

SEND DATA

The `Port` method `write` accepts a `byte[]` containing data to send. It blocks until the data is queued and enables the transmitter if needed. The `Port` method `flush` blocks until all queued data has been sent. Both methods return `true` on success or `false` if cancelled by another thread. If the `Port` property `blocked_io` is `false` (polling) both methods return `false` instead of blocking.

```
var send_data = new byte[] {0xff, 0x01};

port.blocked_io = true;
if (port.write(send_data))
    Console.WriteLine("data queued");
else
    Console.WriteLine("blocked write cancelled");
if (port.flush())
    Console.WriteLine("all data sent");
else
    Console.WriteLine("blocked flush cancelled");

port.blocked_io = false;
if (port.write(send_data))
    Console.WriteLine("data queued");
else
    Console.WriteLine("send queue full");
if (port.flush())
    Console.WriteLine("all data sent");
else
    Console.WriteLine("busy sending data");
```

The `byte[]` length must not exceed the `Port.Defaults` attribute `max_data_size`.

HDLC/SDLC protocol

`write` sends a variable size frame. Hardware adds flags and CRC to mark frame boundary.

TDM protocol

`write` sends one or more frames of fixed size set by TDM configuration.

Other protocols

`write` sends a variable length stream of bytes.

RECEIVE DATA

The `Port` method `read` accepts an optional `size` argument (default value of 1) and returns a `byte[]` containing received data. It blocks until data is available or returns `null` if cancelled by another thread. The `Port` method `enable_receiver` must be called before data can be received. If the `Port` property `blocked_io` is `false` (polling) `read` returns `null` instead of blocking.

```
port.blocked_io = true;
receive_data = port.read(size); // block until data available
if (receive_data != null)
    Console.WriteLine("{0} bytes received", receive_data.Length);
else
    Console.WriteLine("blocked read cancelled");

port.blocked_io = false;
receive_data = port.read(size); // poll for data
if (receive_data != null)
    Console.WriteLine("{0} bytes received", receive_data.Length);
else
    Console.WriteLine("no data available");
```

The `size` argument meaning depends on the protocol:

HDLC/SDLC

`Size` argument is ignored and should be omitted for clarity. `read` blocks until a variable size frame is received.

TDM

`Size` argument is ignored and should be omitted for clarity. `read` blocks until a fixed size frame or group of frames (as defined by TDM configuration) is received.

Other Protocols

`read` blocks until `size` bytes are received. `size` defaults to 1 and must be in the range 1 to the `Port.Defaults` attribute `max_data_size`.

CLASS REFERENCE

PORT

A `Port` object is required for most tasks. A port name must be supplied. Port names for single port adapters have the form `MGHDLcX`, where 'x' is the adapter number. Port names for multiport adapters have the form `MGMPxPy`, where x is the adapter number and 'y' is the port number.

```
port = new Port("MGHDLc1");
```

METHODS

`apply_settings`

This method applies configuration contained in a [Port.Settings](#) object to hardware. Hardware is shutdown, blocked `read` and `write` calls are cancelled and the new configuration is applied.

```
var settings = new Port.Settings(); // allocate
settings.protocol = Port.HDLC; // change
port.apply_settings(settings); // apply
```

`close`

This method releases a port from use so it may be used by other processes. A port is automatically closed when a process exits or the port object is deallocated. Methods and properties other than `open` and `is_open` must not be accessed when a port is closed. Closing an already closed port has no effect. Port properties may change while port is closed.

`disable_receiver`

This method disables the receiver hardware and cancels blocked read calls. When disabled, the receiver ignores the receive data input.

`disable_transmitter`

This method disables the transmitter hardware and cancels blocked write and flush calls. When disabled, the transmit data output is held in a constant one state.

`enable_receiver`

This method enables the receiver hardware. When enabled, the receiver accepts data from the receive data input.

`enable_transmitter`

This method enables the transmitter hardware. The transmitter enters the idle state and sends the transmit idle pattern, except for asynchronous/isochronous protocols which always send all ones when idle. When data is passed to the `write` method, the transmitter becomes active and sends the data. After all data is sent, the transmitter is idle.

enumerate

This class method returns a list of valid port names.

```
string[] names = Port.enumerate();
foreach (string name in names)
    Console.WriteLine(name);
```

get_defaults

This method returns default configuration contained in a [Port.Defaults](#) object.

```
Port.Defaults defaults = port.get_defaults();
Console.WriteLine("max_data_size = {0}", defaults.max_data_size);
```

get_settings

This method returns current settings contained in a [Port.Settings](#) object.

```
Port.Settings settings = port.get_settings();
if (settings.protocol == Port.HDLC)
    Console.WriteLine("HDLC protocol");
```

flush

The Port method `flush` blocks until all queued send data has been sent. `true` is returned on success or `false` if cancelled by another thread. If the Port property `blocked_io` is `false` (polling), `false` is returned instead of blocking.

```
port.blocked_io = true;
if (port.flush())
    Console.WriteLine("all data sent");
else
    Console.WriteLine("blocked flush cancelled");

port.blocked_io = false;
if (port.flush())
    Console.WriteLine("all data sent");
else
    Console.WriteLine("busy sending data");
```

is_open

Return `true` when port is open or `false` when port is closed.

open

The `Port` method `open` claims a port for use and raises an exception if an error occurs. The `Port` method `is_open` determines if the port is open. Other properties and methods should be accessed only when the port is open.

```
try {
    port.open();
}
catch (FileNotFoundException) {
    Console.WriteLine("port not found");
}
catch (UnauthorizedAccessException) {
    Console.WriteLine("access denied or port in use");
}
catch (Win32Exception) {
    Console.WriteLine("open error");
}
```

read

This method accepts an optional `size` argument (default value of 1) and returns a `byte[]` containing received data. It blocks until data is available or returns `null` if cancelled by another thread. The `Port` method `enable_receiver` must be called before data can be received. If the `Port` property `blocked_io` is `false` (polling) `read` returns `null` instead of blocking.

```
port.blocked_io = true;
receive_data = port.read(size); // block until data available
if (receive_data != null)
    Console.WriteLine("{0} bytes received", receive_data.Length);
else
    Console.WriteLine("blocked read cancelled");

port.blocked_io = false;
receive_data = port.read(size); // poll for data
if (receive_data != null)
    Console.WriteLine("{0} bytes received", receive_data.Length);
else
    Console.WriteLine("no data available");
```

The `size` argument depends on the protocol:

HDLC/SDLC

Size argument is ignored and should be omitted for clarity. `read` blocks until a variable size frame is received.

TDM

Size argument is ignored and should be omitted for clarity. `read` blocks until a fixed size frame or group of frames (as defined by TDM configuration) is received.

Other Protocols

`read` blocks until `size` bytes are received. `size` defaults to 1 and must be in the range 1 to the `Port.Defaults` attribute `max_data_size`.

set_defaults

This method sets persistent default configuration contained in a [Port.Defaults](#) object. Calling this method requires administrative privilege.

```
// RTS controls RS422 output enable
Port.Defaults defaults = port.get_defaults();
defaults.rts_output_enable = true;
port.set_defaults(defaults);
```

set_fsynth_rate

This method programs the frequency synthesizer to the specified rate and returns `true` if successful. The frequency synthesizer replaces the default base clock of 14.7456MHz. Use the frequency synthesizer only when generating an

internal clock at a rate that is not a divisor of the default base clock. This method internally sets the property [base clock rate](#) to the same value.

All ports on devices with multiple ports share the same base clock. **Do not call this method when any port on the device is sending or receiving.**

The internal clock generator is used for:

- clock generation for synchronous protocols
- reference clock for asynchronous protocol (at 8 or 16 times data rate)
- reference clock for clock recovery (at 8 or 16 times data rate)

The API tries to use x16 reference clocks for maximum accuracy. If the x16 reference clock is not a divisor of the base clock, the API falls back to x8 reference clocks.

This method uses a table of supported frequencies. If an unsupported frequency is specified then `false` is returned. The tables are located in the API module `mgapi.cs`. New table entries can be created as described in the [frequency synthesizer](#) section. The frequency synthesizer is only available on PCI Express cards and the SynLink USB.

wait

This method waits for a specified event (hardware or signal state). The method blocks until one or more specified events occur. The method accepts an integer containing bit flags specifying events to wait for and returns an integer indicating which events occurred. An optional timeout in milliseconds causes the method to return 0 if the timeout expires.

```
Port.DSR_ON, Port.DSR_OFF
Port.CTS_ON, Port.CTS_OFF
Port.DCD_ON, Port.DCD_OFF
Port.RI_ON, Port.RI_OFF
Port.RECEIVE_ACTIVE, Port.RECEIVE_IDLE

// wait forever for CTS on or DCD off
uint events = port.wait(Port.CTS_ON | Port.DCD_OFF);
if (events != 0) {
    if ((events & Port.CTS_ON) != 0)
        Console.WriteLine("CTS is on");
    if ((events & Port.DCD_OFF) != 0)
        Console.WriteLine("DCD is off");
} else
    Console.WriteLine("error");

// wait 1 second for CTS on or DCD off
events = port.wait(Port.CTS_ON | Port.DCD_OFF, 1000);
if (events != 0) {
    if ((events & Port.CTS_ON) != 0)
        Console.WriteLine("CTS is on");
    if ((events & Port.DCD_OFF) != 0)
        Console.WriteLine("DCD is off");
} else
    Console.WriteLine("timeout");
```

write

`write` accepts a `byte[]` containing data to send. It blocks until the data is queued and enables the transmitter if needed. `true` is returned if data is queued or `false` if a blocked write is cancelled by another thread. If the Port property `blocked_io` is `false` (polling), `false` is returned instead of blocking. To determine when data has been completely sent call the `flush` method.

```
var send_data = new byte[] {0xff, 0x01};

port.blocked_io = true;
if (port.write(send_data))
    Console.WriteLine("data queued");
else
    Console.WriteLine("blocked write cancelled");

port.blocked_io = false;
if (port.write(send_data))
    Console.WriteLine("data queued");
else
    Console.WriteLine("send queue full");
```

The `byte[]` length must not exceed the `Port.Defaults` attribute `max_data_size`.

HDLC/SDLC protocol

`write` sends a variable size frame. Hardware adds flags and CRC to mark frame boundary.

TDM protocol

`write` sends one or more frames of fixed size set by TDM configuration.

Other protocols

`write` sends a variable length stream of bytes.

PROPERTIES AND ATTRIBUTES

base_clock_rate

This integer property is the base clock rate used in calculations for internal clock generation. Setting this property **does not** alter the actual base clock like [set fsynth rate](#). Use this property to set the base clock when a custom fixed frequency oscillator is installed on the SyncLink device. This property is shared by all ports on an adapter.

base_clock_source

This integer property selects the base clock source.

Port.FIXED_OSCILLATOR = Fixed frequency oscillator (default)

Port.FREQUENCY_SYNTHESIZER = Frequency synthesizer (PCI Express and USB only)

blocked_io

This Boolean property controls the behavior of the `read`, `write` and `flush` methods. This property is set to `True` when the `open` method is called.

`true` (blocking)

read blocks until data is available then returns a byte[]
write blocks until data is queued then returns true
flush blocks until all data is sent then returns true

false (polling)

read returns null instead of blocking

write and flush return false instead of blocking.

cts (Clear To Send)

This read only Boolean property reports the CTS input state. true = on, false = off.

```
if (port.cts)
    Console.WriteLine("CTS is on");
```

dcd (Data Carrier Detect)

This read only Boolean property reports the DCD input state. true = on, false = off.

```
if (port.dcd)
    Console.WriteLine("DCD is on");
```

dsr (Data Set Ready)

This read only Boolean property reports the DSR input state. true = on, false = off.

```
if (port.dsr)
    Console.WriteLine("DSR is on");
```

dtr (Data Terminal Ready)

This Boolean property controls the DTR output state. true = on, false = off.

```
port.dtr = true; // turn on DTR
```

gpio

This attribute is an array of 32 [Port.GPIO](#) objects with each object representing a general purpose I/O signal. Refer to the hardware manual (PDF) for how many GPIO signals are available on a specific device. GPIO signals are controlled and monitored through the output and state properties.

```
// set GPIO #6 direction to output
port.gpio[6].output = true;
// set GPIO #6 state to low
port.gpio[6].state = false;
// set GPIO #7 direction to input
port.gpio[7].output = false;
if (port.gpio[7].state)
    Console.WriteLine("GPIO #7 is on");
else
    Console.WriteLine("GPIO #7 is off");
```

half_duplex

This Boolean property enables a feature where differential (RS422/V.35) outputs are enabled when sending data and disabled when not sending data. The receive data input is disabled when sending data. This is used when

transmit and receive data share the same wires. Applications requiring more control over output enable timing should use the `rts_output_enable` property.

ignore_read_errors

This Boolean property controls how receive errors are handled in HDLC/SDLC mode. When `True`, receive errors are silently discarded. Applications should only set this to `false` for diagnostic and logging purposes. This property is set to `true` by the `open` method.

interface_type

This integer property selects the serial interface standard for SyncLink USB and SyncLink PCIe devices. GT series cards use jumpers to select the serial interface and ignore this property. Interface values are defined by constants.

```
// Port.RS232 = single ended, unbalanced signals (V.28)
// Port.V35 = data/clocks differential (V.11), others single ended (V.28)
// Port.RS422 = differential signals (V.11)
// Port.RS530A = differential signals (V.11), except LL and RL

port.interface_type = Port.RS422;
```

ll (Local Loopback)

This Boolean property controls the LL output state.

```
port.ll = true; // turn on local loopback output
```

name

This read only string property contains the port name.

output_control

This integer property controls individual differential (RS422/V.11) outputs on the SyncLink USB. The integer contains bit flags controlling behavior and state for four output signals. This property does not affect PCI cards or single ended outputs.

```
[7] TXD select (0=auto, 1=manual)
[6] AUXCLK select (0=auto, 1=manual)
[5] DTR select (0=auto, 1=manual)
[4] RTS select (0=auto, 1=manual)
[3] TXD state (0=tristate/disabled, 1=enabled)
[2] AUXCLK state (0=tristate/disabled, 1=enabled)
[1] DTR state (0=tristate/disabled, 1=enabled)
[0] AUXCLK state (0=tristate/disabled, 1=enabled)
```

Auto (0) enables the differential output as described by other properties. Manual (1) enables outputs as selected by state bits in this property.

receive_count

This integer read only property indicates how many bytes are available to the application using the `read` method. The granularity of the count depends on the specific hardware and protocol. The count does not include data stored in hardware but not yet transferred to system memory. Use this property to poll for receive data.

ri (Ring Indicator)

This read only Boolean property reports the RI input state. `true` = on, `false` = off.

```
if (port.ri)
    Console.WriteLine("RI is on");
```

rl (Remote Loopback)

This Boolean property controls the RL output state.

```
port.rl = true; // turn on remote loopback output
```

rts (Request To Send)

This Boolean property controls the RTS output state. `true` = on, `false` = off.

```
port.rts = true; // turn on RTS output
```

rts_output_enable

Enable differential (RS422/V.35) outputs when RTS is on and disable (high impedance) them when RTS is off. Use when transmit and receive data share the same wires. Applications that do not require control over output enable timing should use the `half_duplex` property. Cards without RTS signal have internal RTS state to support this feature.

```
port.rts_output_enable = true;
port.rts = true; // turn on outputs
// transmit data
port.rts = false; // turn off outputs (tristate/hi-Z)
// receive data
```

signals

This integer property contains control output and status input states with each signal represented by a bit. Individual signal bits are defined as constants. Each access (get or set) of this property results in a hardware access.

```
// Port.DTR = data terminal ready output
// Port.RTS = ready to send output
// Port.CTS = clear to send input
// Port.DSR = data set ready input
// Port.DCD = data carrier detect input
// Port.RI  = ring indicator input

// turn on DTR output
port.signals |= Port.DTR;

// turn off RTS output
port.signals &= ~Port.RTS;

if ((port.signals & Port.DCD) != 0)
    Console.WriteLine("DCD input is on");
else
    Console.WriteLine("DCD input is off");
```

Each signal has a property that is an alias to the `signals` property.

```
// turn on DTR output
port.dtr = true;
// turn off RTS output
port.rts = false;

if (port.dcd)
    Console.WriteLine("DCD input is on");
else
    Console.WriteLine("DCD input is off");
```

transmit_count

This integer read only property indicates the number of bytes in the API send queue. The count does not include data transferred to hardware but not yet sent. The count granularity depends on the specific hardware and protocol. Use this property to maintain queue levels in a desired range for the purpose of controlling transmit latency. Do NOT use this property as an indication of all sent as a zero value may not reflect data still stored in the hardware. Use the [flush](#) method to determine when all data has been sent.

termination

This Boolean property controls termination of differential (RS422/V.11) inputs on the SyncLink USB/PCIe. When `true`, differential inputs are terminated with 120 ohms. When `false`, differential inputs are not terminated. GT cards select termination with jumpers or switch settings and are not affected by this property.

transmit_idle_pattern

This property specifies the pattern sent on the transmit data output when the transmitter is idle (enabled but no data to send). This can be an arbitrary 8 or 16 bit value. The idle pattern may be used by a remote device to distinguish between an inactive and idle connection. The transmit idle pattern may have additional protocol specific purpose. For monosync and bisync protocols, this property becomes an alias for `sync_pattern`.

PORT.DEFAULTS

A `Port.Defaults` object contains default configuration that is persistent across system restarts and device ejection. Defaults are accessed with the [Port.get_defaults](#) and [Port.set_defaults](#) methods.

PROPERTIES AND ATTRIBUTES

interface_type

This Boolean attribute sets the default state of the [Port.interface_type](#) property. This value is labeled "Serial interface" in the Windows device manager properties for SyncLink devices.

max_data_size

This integer attribute specifies the maximum number of bytes that are returned by `read` or supplied to `write`. The default value is 4096. This value is labeled "Max Frame Size" in the Windows device manager properties for SyncLink devices.

rts_output_enable

This Boolean attribute sets the default state of the [Port.rts_output_enable](#) property. This value is labeled "Disable RS422/485 outputs when RTS is off" in the Windows device manager properties for SyncLink devices.

termination

This Boolean attribute sets the default state of the [Port.termination](#) property. This value is labeled "Serial interface" in the Windows device manager properties for SyncLink devices.

PORT.GPIO

A `Port.GPIO` object represents a single general purpose I/O signal. PCI Express and USB serial devices have GPIO signals accessible to headers on the device PCB. Refer to the hardware manual (PDF) for information on how many GPIO signals are available and header pin assignments.

PROPERTIES AND ATTRIBUTES

state

This Boolean property contains the GPIO signal state: `true` = high, `false` = low.

output

This Boolean property contains the GPIO signal direction: `true` = output, `false` = input. Some GPIO signals have a fixed direction, see hardware manual for details.

PORT.SETTINGS

A `Port.Settings` object contains configuration used with the `Port` methods `get_settings` and `apply_settings`.

PROPERTIES AND ATTRIBUTES

async_data_bits

This integer attribute specifies the number of data bits in an asynchronous character. Valid values: 5 to 8

async_data_rate

This integer attribute specifies the data rate for the asynchronous protocol in bits per second. The internal clock generator runs at 8 or 16 times the data rate to supply the asynchronous receiver sampling clock.

async_stop_bits

This integer attribute specifies the number of stop bits in an asynchronous character. Valid values: 1 or 2

async_parity

This attribute specifies the parity used in an asynchronous character. Valid values: `Port.OFF`, `Port.EVEN`, `Port.ODD`.

auto_cts

This Boolean attribute controls automatic processing of the CTS (clear to send) input. When `true`, the transmitter waits for CTS on before sending data.

auto_dcd

This Boolean attribute controls automatic processing of the DCD (data carrier detect) input. When `true`, the receiver is disabled when DCD is off.

auto_rts

This Boolean attribute controls automatic processing of the RTS (request to send) output. When `true`, RTS is on when data is available to send and off when all data has been sent. This feature only applies when RTS starts in the off state. If RTS is manually set to on then RTS stays on regardless of this attribute.

crc

This attribute selects the frame check sequence used with the HDLC (SDLC) protocol. Other protocols must set this attribute to `Port.OFF`. Valid values are listed below. The same CRC must be used by all endpoints.

<code>Port.OFF</code>	CRC disabled
<code>Port.CRC16</code>	16-bit CRC Polynomial: $x^{16} + x^{12} + x^5 + 1$ init value = all 1s
<code>Port.CRC32</code>	32-bit CRC Polynomial: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ init value = all 1s

discard_data_with_error

This Boolean attribute determines how the API handles HDLC (SDLC) frames with a frame check error. When set to `true` (default), frames with errors are silently discarded. When `false`, frames with errors are returned to the application. Only applications that wish to inspect invalid data for diagnostics, logging and recovery purposes should set this to `false`.

discard_received_crc

This Boolean attribute controls handling of received CRC values for the HDLC (SDLC) protocol. When `true` (default), the CRC is discarded and the application only gets data. When `false`, the CRC is returned as the final bytes of the frame to the application. Only applications that wish to inspect the CRC value for diagnostics, logging or recovery purposes should set this to `false`.

encoding

This attribute selects the encoding of serial data. Encoding defines how individual logical bits (0 or 1) are translated to physical data signal levels (high or low). Serial link endpoints must use the same encoding. Valid encoding values are listed below. Refer to the [Encoding](#) section for details of each value.

```
Port.NRZ
Port.NRZB
Port.NRZI
Port.NRZI_MARK
Port.FM0
Port.FM1
Port.MANCHESTER
Port.DIFF_BIPHASE_LEVEL
```

hdlc_address_filter

This integer attribute selects an 8-bit value used to filter received HDLC (SDLC) frames. Other protocols ignore this attribute. A value of `0xFF` (default) disables filtering and returns all frames to the application. Other values indicate the expected HDLC station address so that only frames with the broadcast address (`0xFF`) and the specified station address are returned to the application. Other frames are silently discarded.

internal_clock_rate

This integer attribute is the **data rate** used to setup the internal clock generator. When using the internal clock directly as a data clock it runs at this rate. When using clock recovery the internal clock runs at 8 or 16 times this rate to create a **reference clock** for sampling the receive data input and recovering a data clock.

The internal clock is generated by dividing the hardware base clock (default 14.7456MHz) by a 16-bit integer. If the specified rate is not a divisor of the base clock it is approximated by selecting the next slower rate that is a divisor. For details, refer to the [clock generator](#) section.

The API tries to use a 16x reference clock for maximum accuracy. If a 16x reference clock is not a divisor of the base clock, the API falls back to an 8x reference clock. If an x8 reference clock is not a divisor of the base clock then the specified data rate can't be precisely recovered.

The base clock can be adjusted on PCI Express and USB devices to allow precise internal clock rates when the specified rate (or reference clock) is not a divisor of the default 14.7456MHz base clock. Refer to the [set_fsynth_rate](#) method.

internal_loopback

This Boolean attribute enables internal loopback of data signals for diagnostic or development purposes. When set to `true`, the transmit data output is connected internally to the data input and data clocks are internally generated.

msb_first

This Boolean attribute selects the serial bit order. Most protocols must use least significant bit (LSB) first. Some protocols (RAW and TDM) may optionally use most significant bit (MSB) first by setting this attribute to `true`.

protocol

This attribute selects the serial protocol. Valid values are listed below. Refer to the [Protocol](#) section for details.

<code>Port.HDLC</code>	HDLC, also called SDLC
<code>Port.ASYNC</code>	Asynchronous and Isochronous
<code>Port.BISYNC</code>	Bi-synchronous
<code>Port.MONOSYNC</code>	Mono-synchronous
<code>Port.RAW</code>	Raw bit stream
<code>Port.TDM</code>	Time Division Multiplexing

receive_clock

This attribute selects the source of the receive clock.

<code>Port.TXC_INPUT</code>	TxC clock input
<code>Port.RXC_INPUT</code>	RxC clock input
<code>Port.INTERNAL</code>	internal clock generator
<code>Port.RECOVERED</code>	clock recovered from receive data input

receive_clock_invert

This attribute controls receive clock polarity. When set to default value of `false`, the receive clock uses normal polarity. When set to `true`, the receive clock uses inverted clock polarity. Endpoints must use the same clock polarity.

sync_pattern

This integer attribute specifies the synchronization pattern used for byte synchronous protocols (monosync, bisync, xsync) and should not be used for other protocols. The receiver searches for the sync pattern to determine byte alignment and returns data following a sync pattern to the application until the receiver is disabled or forced to the idle state so it searches for the next sync pattern.

tdm_sync_delay

This integer attribute specifies the number of clock cycles data is delayed from the start of a TDM sync pulse. Valid values are 0 (no delay), 1 and 2.

tdm_sync_frame

This Boolean attribute specifies the length of a TDM sync pulse. `True` = one frame, `False` = use `tdm_sync_short`

tdm_sync_short

This Boolean attribute specifies the length of a TDM sync pulse. `true` = one clock cycle, `false` = one slot length

tdm_sync_invert

This Boolean attribute specifies the polarity of the TDM sync signal. `false` = normal, `true` = inverted

tdm_frame_count

This integer attribute specifies the number of TDM frames returned to the application by the `read` method. Valid values are 1 to 256. The frame count time the fixed frame size must not exceed `Port.Defaults.max_data_size`. This allows better performance by reducing the number of `read` calls when the frame size is small and the data rate is high.

tdm_slot_count

This integer attribute specifies the number of TDM slots in a frame. Valid values are in the range 2 to 32 or 384.

tdm_slot_bits

This integer attribute specifies the number of bits in a TDM slot. Valid values are 8, 12, 16, 20, 24, 28 and 32.

transmit_clock

This attribute selects the source of the transmit clock.

<code>Port.TXC_INPUT</code>	TxC clock input
<code>Port.RXC_INPUT</code>	RxC clock input
<code>Port.INTERNAL</code>	internal clock generator
<code>Port.RECOVERED</code>	clock recovered from receive data input

transmit_clock_invert

This attribute controls transmit clock polarity. When set to default value of `false`, the transmit clock uses normal polarity. When set to `true`, the transmit clock uses inverted clock polarity. Endpoints must use the same clock polarity.

transmit_preamble_bits

This integer attribute selects the number of bits of preamble to prepend to sent data. A preamble is used to assist clock recovery from a data signal. Only synchronous protocols using clock recovery should set this attribute to a non-zero value. The only valid values are 8, 16, 32 and 64.

transmit_preamble_pattern

This 8-bit integer attribute selects the pattern that is prepended to sent data. A preamble is used to assist clock recovery from a data signal. Only synchronous protocols using clock recovery should set this attribute. The length of the preamble is selected by the [transmit_preamble_bits](#) attribute.

xsync_block_size

This integer attribute sets the fixed block size for the xsync protocol. If set to 0, data is returned as a stream of bytes and the application determines block boundaries. Non zero values set the fixed block size with one block returned per `read` call. Valid values are 1 to 65535.

xsync_sync_size

This integer attribute sets the sync pattern size in bytes for the xsync protocol. Valid values are 1 to 4.

PYTHON

The module `mgapi.py` provides a Python interface to Synclink serial devices. It is built on top of the C API and provides both a Python optimized interface and a C API wrapper. New Python applications should use the Python optimized interface. The C wrapper is used for quickly porting C applications to Python with minimal changes. The wrapper follows the naming conventions of the C API. This violates Python style standards but reduces the effort needed to port existing code.

WARNING: The API module uses Python 3 and is incompatible with Python 2.

A Python installation is required to run the Python sample code. The API module must be imported into a Python application with one of the following lines:

```
# Python API (new Python apps)
from mgapi import Port

# Python wrapper of C API (porting C apps)
import mgapi
```

The module must be found in one of three ways:

- `mgapi.py` is located in same directory as application.
- `mgapi.py` is located in the Python system path.
- `mgapi.tar.gz` package is installed.

The package is installed using the Python installation program (`pip`) in a command prompt with Administrator privilege:

```
C:\mgapi\python>pip install mgapi.tar.gz
```

Contents of `mgapi\python` directory:

<code>mgapi.py</code>	Python API module
<code>mgapi.tar.gz</code>	Installable Python API package
<code>samples\hdlc.py</code>	HDLC/SDLC sample
<code>samples\async.py</code>	asynchronous sample
<code>samples\bisync.py</code>	bisync/monosync sample
<code>samples\raw.py</code>	raw bitstream sample
<code>samples\tdm.py</code>	TDM sample
<code>samples\2wire.py</code>	2-wire (bussed) sample
<code>samples\commapi.py</code>	asynchronous using pyserial interface
<code>samples\signals.py</code>	control/status signal sample
<code>samples\cwrapper.py</code>	C API wrapper sample (for porting C applications)

A Python application is run as shown below for the HDLC sample on device `mghdlc1`:

```
C:\mgapi\python\samples>python hdlc.py mghdlc1
```

PORT OBJECT

A `Port` object is required for most tasks. A port name must be supplied. Port names for single port adapters have the form `MGHDLCx`, where 'x' is the adapter number. Port names for multiport adapters have the form `MGMPxPy`, where x is the adapter number and 'y' is the port number. The `Port` class function `enumerate` returns a list of available port names.

```
names = Port.enumerate()
for name in names:
    print(name)

port = Port('MGHDLCL')
```

OPEN/CLOSE

The `Port` method `open` claims a port for use and raises an exception if an error occurs. The `Port` method `is_open` determines if the port is open. Other attributes, properties and methods must be accessed only when the port is open.

```
try:
    port.open()
except FileNotFoundError:
    print('port not found')
except PermissionError:
    print('access denied or port in use')
except OSError:
    print('open error')
if port.is_open():
    print('port is open')
```

The `Port` method `close` releases the port from use. A port must be closed to allow use by other processes. Ports are automatically closed when a process exits or the `Port` object is deallocated.

```
port.close()
```

CONFIGURATION

A port must be configured to match application specific requirements. Perform initial configuration with a `Port.Settings()` object. The `Port` method `apply_settings()` cancels current operations and shuts down the hardware.

```
settings = Port.Settings()
settings.protocol = Port.HDLC
settings.encoding = Port.MANCHESTER
settings.crc = Port.CRC16
settings.transmit_clock = Port.INTERNAL
settings.receive_clock = Port.RECOVERED
settings.internal_clock_rate = 9600
settings.transmit_preamble_pattern = 0x7e
settings.transmit_preamble_bits = 8
port.apply_settings(settings)
```

During operation, alter port behavior using various properties.

```
# set pattern sent when transmitter is enabled but
# there is no data to send. HDLC flag = 0x7e
port.transmit_idle_pattern = 0x7e
```

Set persistent default state that spans system restart and device ejection.

```
# RTS controls RS422 output enable
defaults = port.get_defaults()
defaults.rts_output_enable = True
port.set_defaults(defaults)
```

SEND DATA

The `Port` method `write` accepts a `bytearray` containing data to send. It blocks until the data is queued and enables the transmitter if needed. The `Port` method `flush` blocks until all queued data has been sent. Both methods return `True` on success or `False` if cancelled by another thread. If the `Port` property `blocked_io` is `False` (polling) both methods return `False` instead of blocking.

```
send_data = bytearray.fromhex('FF01')

port.blocked_io = True
if port.write(send_data):
    print('data queued')
else:
    print('blocked write cancelled')
if port.flush():
    print('all data sent')
else:
    print('blocked flush cancelled')

port.blocked_io = False
if port.write(send_data):
    print('data queued')
else:
    print('send queue full')
if port.flush():
    print('all data sent')
else:
    print('busy sending data')
```

The `bytearray` length must not exceed the `Port.Defaults` attribute `max_data_size`.

HDLC/SDLC protocol

`write` sends a variable size frame. Hardware adds flags and CRC to mark frame boundary.

TDM protocol

`write` sends one or more frames of fixed size set by TDM configuration.

Other protocols

`write` sends a variable length stream of bytes.

RECEIVE DATA

The `Port` method `read` accepts an optional `size` argument (default value of 1) and returns a `bytearray` containing received data. It blocks until data is available or returns `None` if cancelled by another thread. The `Port` method `enable_receiver` must be called before data can be received. If the `Port` property `blocked_io` is `False` (polling) `read` returns `None` instead of blocking.

```
port.blocked_io = True
receive_data = port.read(size) # block until data available
if receive_data:
    print(len(receive_data), 'bytes received')
else:
    print('blocked read cancelled')

port.blocked_io = False
receive_data = port.read(size) # poll for data
if receive_data:
    print(len(receive_data), 'bytes received')
else:
    print('no data available')
```

The `size` argument meaning depends on the protocol:

HDLC/SDLC

`Size` argument is ignored and should be omitted for clarity. `read` blocks until a variable size frame is received.

TDM

`Size` argument is ignored and should be omitted for clarity. `read` blocks until a fixed size frame or group of frames (as defined by TDM configuration) is received.

Other Protocols

`read` blocks until `size` bytes are received. `size` defaults to 1 and must be in the range 1 to the `Port.Defaults` attribute `max_data_size`.

CLASS REFERENCE

PORT

A `Port` object is required for most tasks. A port name must be supplied. Port names for single port adapters have the form `MGHDLCx`, where 'x' is the adapter number. Port names for multiport adapters have the form `MGMPxPy`, where x is the adapter number and 'y' is the port number.

```
port = Port('MGHDL1')
```

METHODS

`apply_settings`

This method applies configuration contained in a [Port.Settings](#) object to hardware. Hardware is shutdown, blocked `read` and `write` calls are cancelled and the new configuration is applied.

```
settings = Port.Settings()      # allocate
settings.protocol = Port.HDLC   # change
port.apply_settings(settings)   # apply
```

`close`

This method releases a port from use so it may be used by other processes. A port is automatically closed when a process exits or the port object is deallocated. Methods and properties other than `open` and `is_open` must not be accessed when a port is closed. Closing an already closed port has no effect. Port properties may change while port is closed.

`disable_receiver`

This method disables the receiver hardware and cancels blocked read calls. When disabled, the receiver ignores the receive data input.

`disable_transmitter`

This method disables the transmitter hardware and cancels blocked write and flush calls. When disabled, the transmit data output is held in a constant one state.

`enable_receiver`

This method enables the receiver hardware. When enabled, the receiver accepts data from the receive data input.

`enable_transmitter`

This method enables the transmitter hardware. The transmitter enters the idle state and sends the transmit idle pattern, except for asynchronous/isochronous protocols which always send all ones when idle. When data is passed to the `write` method, the transmitter becomes active and sends the data. After all data is sent, the transmitter is idle.

enumerate

This class method returns a list of valid port names.

```
names = Port.enumerate()
for name in names:
    print(name)
```

get_defaults

This method returns default configuration contained in a [Port.Defaults](#) object.

```
defaults = port.get_defaults()
print('max_data_size =', defaults.max_data_size)
```

get_settings

This method returns current settings contained in a [Port.Settings](#) object.

```
settings = port.get_settings()
if settings.protocol == Port.HDLC:
    print('HDLC protocol')
```

flush

The Port method flush blocks until all queued send data has been sent. True is returned on success or False if cancelled by another thread. If the Port property blocked_io is False (polling), False is returned instead of blocking.

```
port.blocked_io = True
if port.flush():
    print('all data sent')
else:
    print('blocked flush cancelled')

port.blocked_io = False
if port.flush():
    print('all data sent')
else:
    print('busy sending data')
```

is_open

Return True when port is open or False when port is closed.

open

The `Port` method `open` claims a port for use and raises an exception if an error occurs. The `Port` method `is_open` determines if the port is open. Other properties and methods should be accessed only when the port is open.

```
try:
    port.open()
except FileNotFoundError:
    print('port not found')
except PermissionError:
    print('access denied or port in use')
except OSError:
    print('open error =', port.error)
if port.is_open():
    print('port is open')
```

read

This method accepts an optional `size` argument (default value of 1) and returns a `bytearray` containing received data. It blocks until data is available or returns `None` if cancelled by another thread. The `Port` method `enable_receiver` must be called before data can be received. If the `Port` property `blocked_io` is `False` (polling) `read` returns `None` instead of blocking.

```
port.blocked_io = True
receive_data = port.read(size) # block until data available
if receive_data:
    print(len(receive_data), 'bytes received')
else:
    print('blocked read cancelled')

port.blocked_io = False
receive_data = port.read(size) # poll for data
if receive_data:
    print(len(receive_data), 'bytes received')
else:
    print('no data available')
```

The `size` argument depends on the protocol:

HDLC/SDLC

Size argument is ignored and should be omitted for clarity. `read` blocks until a variable size frame is received.

TDM

Size argument is ignored and should be omitted for clarity. `read` blocks until a fixed size frame or group of frames (as defined by TDM configuration) is received.

Other Protocols

`read` blocks until `size` bytes are received. `size` defaults to 1 and must be in the range 1 to the `Port.Defaults` attribute `max_data_size`.

set_defaults

This method sets persistent default configuration contained in a [Port.Defaults](#) object. Calling this method requires administrative privilege.

```
defaults = port.get_defaults()
defaults.max_data_size = 8192
port.set_defaults(defaults)
```

set_fsynth_rate

This method programs the frequency synthesizer to the specified rate and returns `True` if successful. The frequency synthesizer replaces the default base clock of 14.7456MHz. Use the frequency synthesizer only when generating an

internal clock at a rate that is not a divisor of the default base clock. This method internally sets the property [base clock rate](#) to the same value.

All ports on devices with multiple ports share the same base clock. **Do not call this method when any port on the device is sending or receiving.**

The internal clock generator is used for:

- clock generation for synchronous protocols
- reference clock for asynchronous protocol (at 8 or 16 times data rate)
- reference clock for clock recovery (at 8 or 16 times data rate)

The API tries to use x16 reference clocks for maximum accuracy. If the x16 reference clock is not a divisor of the base clock, the API falls back to x8 reference clocks.

This method uses a table of supported frequencies. If an unsupported frequency is specified then `False` is returned. The tables are located in the API module `mgapi.py`. New table entries can be created as described in the [frequency synthesizer](#) section. The frequency synthesizer is only available on PCI Express cards and the SynLink USB.

wait

This method waits for a specified event (hardware or signal state). The method blocks until one or more specified events occur. The method accepts an integer containing bit flags specifying events to wait for and returns an integer indicating which events occurred. An optional timeout in milliseconds causes the method to return 0 if the timeout expires.

```
Port.DSR_ON, Port.DSR_OFF
Port.CTS_ON, Port.CTS_OFF
Port.DCD_ON, Port.DCD_OFF
Port.RI_ON, Port.RI_OFF
Port.RECEIVE_ACTIVE, Port.RECEIVE_IDLE

# wait forever for CTS on or DCD off
events = port.wait(Port.CTS_ON | Port.DCD_OFF)
if events:
    if events & Port.CTS_ON:
        print('CTS is on')
    if events & Port.DCD_OFF:
        print('DCD is off')
else:
    print('error')

# wait 1 second for CTS on or DCD off
events = port.wait(Port.CTS_ON | Port.DCD_OFF, 1000)
if events:
    if events & Port.CTS_ON:
        print('CTS is on')
    if events & Port.DCD_OFF:
        print('DCD is off')
else:
    print('timeout')
```

write

`write` accepts a `bytearray` containing data to send. It blocks until the data is queued and enables the transmitter if needed. `True` is returned if data is queued or `False` if a blocked `write` is cancelled by another thread. If the `Port` property `blocked_io` is `False` (polling), `False` is returned instead of blocking. To determine when data has been completely sent call the `flush` method.

```
send_data = bytearray.fromhex('FF01')

port.blocked_io = True
if port.write(send_data):
    print('data queued')
else:
    print('blocked write cancelled')

port.blocked_io = False
if port.write(send_data):
    print('data queued')
else:
    print('send queue full')
```

The `bytearray` length must not exceed the `Port.Defaults` attribute `max_data_size`.

HDLC/SDLC protocol

`write` sends a variable size frame. Hardware adds flags and CRC to mark frame boundary.

TDM protocol

`write` sends one or more frames of fixed size set by TDM configuration.

Other protocols

`write` sends a variable length stream of bytes.

PROPERTIES AND ATTRIBUTES

base_clock_rate

This integer property is the base clock rate used in calculations for internal clock generation. Setting this property **does not** alter the actual base clock like [set_fsynth_rate](#). Use this property to set the base clock when a custom fixed frequency oscillator is installed on the SyncLink device. This property is shared by all ports on an adapter.

base_clock_source

This integer property selects the base clock source.

`Port.FIXED_OSCILLATOR` = Fixed frequency oscillator (default)

`Port.FREQUENCY_SYNTHESIZER` = Frequency synthesizer (PCI Express and USB only)

blocked_io

This Boolean property controls the behavior of the `read`, `write` and `flush` methods. This property is set to `True` when the `open` method is called.

`True` (blocking)

read blocks until data is available then returns a bytearray
write blocks until data is queued then returns True
flush blocks until all data is sent then returns True

False (polling)

read returns None instead of blocking
write and flush return False instead of blocking.

cts (Clear To Send)

This read only Boolean property reports the CTS input state. True = on, False = off.

```
if port.cts:
    print('CTS is on')
```

dcd (Data Carrier Detect)

This read only Boolean property reports the DCD input state. True = on, False = off.

```
if port.dcd:
    print('DCD is on')
```

dsr (Data Set Ready)

This read only Boolean property reports the DSR input state. True = on, False = off.

```
if port.dsr:
    print('DSR is on')
```

dtr (Data Terminal Ready)

This Boolean property controls the DTR output state. True = on, False = off.

```
port.dtr = True # turn on DTR
```

gpio

This attribute is a list of 32 [Port.GPIO](#) objects with each object representing a general purpose I/O signal. Refer to the hardware manual (PDF) for how many GPIO signals are available on a specific device. GPIO signals are controlled and monitored through the `output` and `state` properties.

```
# set GPIO #6 direction to output
port.gpio[6].output = True
# set GPIO #6 state to low
port.gpio[6].state = False
# set GPIO #7 direction to input
port.gpio[7].output = False
if port.gpio[7].state:
    print('GPIO #7 is on')
else:
    print('GPIO #7 is off')
```

half_duplex

This Boolean property enables a feature where differential (RS422/V.35) outputs are enabled when sending data and disabled when not sending data. The receive data input is disabled when sending data. This is used when

transmit and receive data share the same wires. Applications requiring more control over output enable timing should use the `rts_output_enable` property.

ignore_read_errors

This Boolean property controls how receive errors are handled in HDLC/SDLC mode. When `True`, receive errors are silently discarded. Applications should only set this to `False` for diagnostic and logging purposes. This property is set to `True` by the `open` method.

interface

This integer property selects the serial interface standard. Hardware using jumpers or switches to select the serial interface, including the GT series, ignore this property. Interface values are defined by constants.

```
# Port.RS232 = single ended, unbalanced signals (V.28)
# Port.V35 = data/clocks differential (V.11), others single ended (V.28)
# Port.RS422 = differential signals (V.11)
# Port.RS530A = differential signals (V.11), except

port.interface = Port.RS422
```

ll (Local Loopback)

This Boolean property controls the LL output state.

```
port.ll = True # turn on local loopback output
```

name

This read only string property contains the port name.

output_control

This integer property controls individual differential (RS422/V.11) outputs on the SyncLink USB. The integer contains bit flags controlling behavior and state for four output signals. This property does not affect PCI cards or single ended outputs.

```
[7] TXD select (0=auto, 1=manual)
[6] AUXCLK select (0=auto, 1=manual)
[5] DTR select (0=auto, 1=manual)
[4] RTS select (0=auto, 1=manual)
[3] TXD state (0=tristate/disabled, 1=enabled)
[2] AUXCLK state (0=tristate/disabled, 1=enabled)
[1] DTR state (0=tristate/disabled, 1=enabled)
[0] AUXCLK state (0=tristate/disabled, 1=enabled)
```

Auto (0) enables the differential output as described by other properties. Manual (1) enables outputs as selected by state bits in this property.

receive_count

This integer read only property indicates how many bytes are available to the application using the `read` method. The granularity of the count depends on the specific hardware and protocol. The count does not include data stored in hardware but not yet transferred to system memory. Use this property to poll for receive data.

ri (Ring Indicator)

This read only Boolean property reports the RI input state. True = on, False = off.

```
if port.ri:
    print('RI is on')
```

rl (Remote Loopback)

This Boolean property controls the RL output state.

```
port.rl = True # turn on remote loopback output
```

rts (Request To Send)

This Boolean property controls the RTS output state. True = on, False = off.

```
port.rts = True # turn on RTS output
```

rts_output_enable

Enable differential (RS422/V.35) outputs when RTS is on and disable (high impedance) them when RTS is off. Use when transmit and receive data share the same wires. Applications that do not require control over output enable timing should use the `half_duplex` property. Cards without RTS signal have internal RTS state to support this feature.

```
port.rts_output_enable = True
port.rts = True # turn on outputs
# transmit data
port.rts = False # turn off outputs (tristate/hi-Z)
# receive data
```

signals

This integer property contains control output and status input states with each signal represented by a bit. Individual signal bits are defined as constants. Each access (get or set) of this property results in a hardware access.

```
# Port.DTR = data terminal ready output
# Port.RTS = ready to send output
# Port.CTS = clear to send input
# Port.DSR = data set ready input
# Port.DCD = data carrier detect input
# Port.RI  = ring indicator input

# turn on DTR output
port.signals |= Port.DTR

# turn off RTS output
port.signals &= ~Port.RTS

if port.signals & Port.DCD:
    print('DCD input is on')
else:
    print('DCD input is off')
```

Each signal has a property that is an alias to the `signals` property.

```
# turn on DTR output
port.dtr = True
# turn off RTS output
port.rts = False

if port.dcd:
    print('DCD input is on')
else:
    print('DCD input is off')
```

transmit_count

This integer read only property indicates the number of bytes in the API send queue. The count does not include data transferred to hardware but not yet sent. The count granularity depends on the specific hardware and protocol. Use this property to maintain queue levels in a desired range for the purpose of controlling transmit latency. Do NOT use this property as an indication of all sent as a zero value may not reflect data still stored in the hardware. Use the [flush](#) method to determine when all data has been sent.

termination

This Boolean property controls termination of differential (RS422/V.11) inputs on the SyncLink USB and SyncLink PCIe hardware. When `True`, differential inputs are terminated with 120 ohms. When `False`, differential inputs are not terminated. GT series cards select termination with jumpers or switch settings and are not affected by this property.

transmit_idle_pattern

This property specifies the pattern sent on the transmit data output when the transmitter is idle (enabled but no data to send). This can be an arbitrary 8 or 16 bit value. The idle pattern may be used by a remote device to distinguish between an inactive and idle connection. The transmit idle pattern may have additional protocol specific purpose. For monosync and bisync protocols, this property becomes an alias for `sync_pattern`.

PORT.DEFAULTS

A `Port.Defaults` object contains default configuration that is persistent across system restarts and device ejection. Defaults are accessed with the [Port.get_defaults](#) and [Port.set_defaults](#) methods.

PROPERTIES AND ATTRIBUTES

interface

This Boolean attribute sets the default state of the [Port.interface](#) property. This value is labeled "Serial interface" in the Windows device manager properties for SyncLink devices.

max_data_size

This integer attribute specifies the maximum number of bytes that are returned by `read` or supplied to `write`. The default value is 4096. This value is labeled "Max Frame Size" in the Windows device manager properties for SyncLink devices.

`rts_output_enable`

This Boolean attribute sets the default state of the `Port.rts_output_enable` property. This value is labeled "Disable RS422/485 outputs when RTS is off" in the Windows device manager properties for SyncLink devices.

`termination`

This Boolean attribute sets the default state of the `Port.termination` property. This value is labeled "Serial interface" in the Windows device manager properties for SyncLink devices.

PORT.GPIO

A `Port.GPIO` object represents a single general purpose I/O signal. PCI Express and USB serial devices have GPIO signals accessible to headers on the device PCB. Refer to the hardware manual (PDF) for information on how many GPIO signals are available and header pin assignments.

PROPERTIES AND ATTRIBUTES

`state`

This Boolean property contains the GPIO signal state: `True` = high, `False` = low.

`output`

This Boolean property contains the GPIO signal direction: `True` = output, `False` = input. Some GPIO signals have a fixed direction, see hardware manual for details.

PORT.SETTINGS

A `Port.Settings` object contains configuration used with the `Port` methods `get_settings` and `apply_settings`.

PROPERTIES AND ATTRIBUTES

`async_data_bits`

This integer attribute specifies the number of data bits in an asynchronous character. Valid values: 5 to 8

`async_data_rate`

This integer attribute specifies the data rate for the asynchronous protocol in bits per second. The internal clock generator runs at 8 or 16 times the data rate to supply the asynchronous receiver sampling clock.

`async_stop_bits`

This integer attribute specifies the number of stop bits in an asynchronous character. Valid values: 1 or 2

`async_parity`

This attribute specifies the parity used in an asynchronous character. Valid values: `Port.OFF`, `Port.EVEN`, `Port.ODD`.

`auto_cts`

This Boolean attribute controls automatic processing of the CTS (clear to send) input. When `True`, the transmitter waits for CTS on before sending data.

auto_dcd

This Boolean attribute controls automatic processing of the DCD (data carrier detect) input. When `True`, the receiver is disabled when DCD is off.

auto_rts

This Boolean attribute controls automatic processing of the RTS (request to send) output. When `True`, RTS is on when data is available to send and off when all data has been sent. This feature only applies when RTS starts in the off state. If RTS is manually set to on then RTS stays on regardless of this attribute.

crc

This attribute selects the frame check sequence used with the HDLC (SDLC) protocol. Other protocols must set this attribute to `Port.OFF`. Valid values are listed below. The same CRC must be used by all endpoints.

<code>Port.OFF</code>	CRC disabled
<code>Port.CRC16</code>	16-bit CRC Polynomial: $x^{16} + x^{12} + x^5 + 1$ init value = all 1s
<code>Port.CRC32</code>	32-bit CRC Polynomial: $x^{32} + x^{26} + x^{23} + x^{22} + x^{16} + x^{12} + x^{11} + x^{10} + x^8 + x^7 + x^5 + x^4 + x^2 + x + 1$ init value = all 1s

discard_data_with_error

This Boolean attribute determines how the API handles HDLC (SDLC) frames with a frame check error. When set to `True` (default), frames with errors are silently discarded. When `False`, frames with errors are returned to the application. Only applications that wish to inspect invalid data for diagnostics, logging and recovery purposes should set this to `False`.

discard_received_crc

This Boolean attribute controls handling of received CRC values for the HDLC (SDLC) protocol. When `True` (default), the CRC is discarded and the application only gets data. When `False`, the CRC is returned as the final bytes of the frame to the application. Only applications that wish to inspect the CRC value for diagnostics, logging or recovery purposes should set this to `False`.

encoding

This attribute selects the encoding of serial data. Encoding defines how individual logical bits (0 or 1) are translated to physical data signal levels (high or low). Serial link endpoints must use the same encoding. Valid encoding values are listed below. Refer to the [Encoding](#) section for details of each value.

```
Port.NRZ
Port.NRZB
Port.NRZI
Port.NRZI_MARK
Port.FM0
Port.FM1
Port.MANCHESTER
Port.DIFF_BIPHASE_LEVEL
```

hdlc_address_filter

This integer attribute selects an 8-bit value used to filter received HDLC (SDLC) frames. Other protocols ignore this attribute. A value of `0xFF` (default) disables filtering and returns all frames to the application. Other values indicate the expected HDLC station address so that only frames with the broadcast address (`0xFF`) and the specified station address are returned to the application. Other frames are silently discarded.

internal_clock_rate

This integer attribute is the **data rate** used to setup the internal clock generator. When using the internal clock directly as a data clock it runs at this rate. When using clock recovery the internal clock runs at 8 or 16 times this rate to create a **reference clock** for sampling the receive data input and recovering a data clock.

The internal clock is generated by dividing the hardware base clock (default 14.7456MHz) by a 16-bit integer. If the specified rate is not a divisor of the base clock it is approximated by selecting the next slower rate that is a divisor. For details, refer to the [clock generator](#) section.

The API tries to use a 16x reference clock for maximum accuracy. If a 16x reference clock is not a divisor of the base clock, the API falls back to an 8x reference clock. If an x8 reference clock is not a divisor of the base clock then the specified data rate can't be precisely recovered.

The base clock can be adjusted on PCI Express and USB devices to allow precise internal clock rates when the specified rate (or reference clock) is not a divisor of the default 14.7456MHz base clock. Refer to the [set fsynth_rate](#) method.

internal_loopback

This Boolean attribute enables internal loopback of data signals for diagnostic or development purposes. When set to `True`, the transmit data output is connected internally to the data input and data clocks are internally generated.

msb_first

This Boolean attribute selects the serial bit order. Most protocols must use least significant bit (LSB) first. Some protocols (RAW and TDM) may optionally use most significant bit (MSB) first by setting this attribute to `True`.

protocol

This attribute selects the serial protocol. Valid values are listed below. Refer to the [Protocol](#) section for details.

<code>Port.HDLC</code>	HDLC, also called SDLC
<code>Port.ASYNC</code>	Asynchronous and Isochronous
<code>Port.BISYNC</code>	Bi-synchronous
<code>Port.MONOSYNC</code>	Mono-synchronous
<code>Port.RAW</code>	Raw bit stream
<code>Port.TDM</code>	Time Division Multiplexing

receive_clock

This attribute selects the source of the receive clock.

<code>Port.TXC_INPUT</code>	TxC clock input
<code>Port.RXC_INPUT</code>	RxC clock input
<code>Port.INTERNAL</code>	internal clock generator
<code>Port.RECOVERED</code>	clock recovered from receive data input

receive_clock_invert

This attribute controls receive clock polarity. When set to default value of `False`, the receive clock uses normal polarity. When set to `True`, the receive clock uses inverted clock polarity. Endpoints must use the same clock polarity.

sync_pattern

This integer attribute specifies the synchronization pattern used for byte synchronous protocols (monosync, bisync, xsync) and should not be used for other protocols. The receiver searches for the sync pattern to determine byte alignment and returns data following a sync pattern to the application until the receiver is disabled or forced to the idle state so it searches for the next sync pattern.

tdm_sync_delay

This integer attribute specifies the number of clock cycles data is delayed from the start of a TDM sync pulse. Valid values are 0 (no delay), 1 and 2.

tdm_sync_frame

This Boolean attribute specifies the length of a TDM sync pulse. `True` = one frame, `False` = use `tdm_sync_short`

tdm_sync_short

This Boolean attribute specifies the length of a TDM sync pulse. `True` = one clock cycle, `False` = one slot length

tdm_sync_invert

This Boolean attribute specifies the polarity of the TDM sync signal. `False` = normal, `True` = inverted

tdm_frame_count

This integer attribute specifies the number of TDM frames returned to the application by the `read` method. Valid values are 1 to 256. The frame count time the fixed frame size must not exceed `Port.Defaults.max_data_size`. This allows better performance by reducing the number of `read` calls when the frame size is small and the data rate is high.

tdm_slot_count

This integer attribute specifies the number of TDM slots in a frame. Valid values are in the range 2 to 32 or 384.

tdm_slot_bits

This integer attribute specifies the number of bits in a TDM slot. Valid values are 8, 12, 16, 20, 24, 28 and 32.

transmit_clock

This attribute selects the source of the transmit clock.

<code>Port.TXC_INPUT</code>	TxC clock input
<code>Port.RXC_INPUT</code>	RxC clock input
<code>Port.INTERNAL</code>	internal clock generator
<code>Port.RECOVERED</code>	clock recovered from receive data input

transmit_clock_invert

This attribute controls transmit clock polarity. When set to default value of `False`, the transmit clock uses normal polarity. When set to `True`, the transmit clock uses inverted clock polarity. Endpoints must use the same clock polarity.

transmit_preamble_bits

This integer attribute selects the number of bits of preamble to prepend to sent data. A preamble is used to assist clock recovery from a data signal. Only synchronous protocols using clock recovery should set this attribute to a non-zero value. The only valid values are 8, 16, 32 and 64.

transmit_preamble_pattern

This 8-bit integer attribute selects the pattern that is prepended to sent data. A preamble is used to assist clock recovery from a data signal. Only synchronous protocols using clock recovery should set this attribute. The length of the preamble is selected by the [transmit_preamble_bits](#) attribute.

xsync_block_size

This integer attribute sets the fixed block size for the xsync protocol. If set to 0, data is returned as a stream of bytes and the application determines block boundaries. Non zero values set the fixed block size with one block returned per `read` call. Valid values are 1 to 65535.

xsync_sync_size

This integer attribute sets the sync pattern size in bytes for the xsync protocol. Valid values are 1 to 4.

PROTOCOLS

This section provides an overview of supported serial protocols. Protocols define how framing, transparency, and timing are implemented. The SDK provides sample code for each protocol. In general, both ends of a serial link must use the same protocol. In some instances, the correct protocol choice will be identified explicitly by application requirements. In less common cases, an application must choose the closest protocol and perform extra tasks to adapt it to specific requirements.

The most common and easy to use protocols are asynchronous (used by common serial ports on PCs) and HDLC. Hardware handles most of the details for these protocols.

Monosync, bisync, XSync are older, less common protocols originally used on IBM mainframes. There are many variations of these protocols. The hardware support is basic, allowing a high degree of application customization.

TDM is a specialized modern protocol often used in defense industry hardware.

Raw bitstream protocol has the most basic hardware support, doing nothing more than serializing/deserializing data with no framing or transparency. The application is responsible for manipulating this raw stream to detect data boundaries and alignment. It offers the most flexibility for rare cases where the other protocols can't be adapted for use. It requires large amounts of complex processing by the application and should only be used as a last resort.

Framing

Framing is a hardware mechanism to identify data boundaries and optionally check data integrity. HDLC uses flag patterns to mark the start and end of frames and CRC for integrity. Asynchronous data uses start/stop bits on each character with parity for integrity. Bisync, monosync and XSync use sync patterns to indicate start of data and the application is responsible for defining and detecting the data format and length. TDM uses a sync signal to mark the start of fixed length frames. Raw protocol has no hardware framing and the application is responsible for interpreting the stream of bits.

Transparency

Transparency is a mechanism to distinguish between data and non data patterns. HDLC uses zero insertion/deletion to distinguish between data and flags. TDM uses external sync signals to identify frame boundaries so data transparency is not necessary. Asynchronous/Isochronous use framing on individual characters so data transparency is not necessary. Monosync, Bisync, XSync, Raw Synchronous require the application to distinguish between data and boundary/sync patterns.

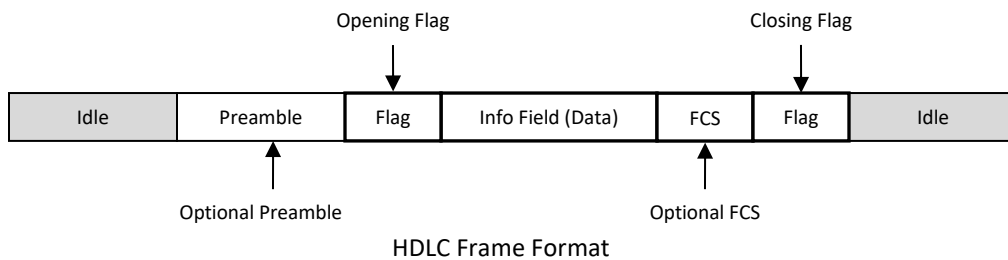
Timing

Serial communication requires a timing mechanism to coordinate data transfer. This can be an external clock signal, an internally generated clock, or a clock recovered from a data signal. The clock frequency determines the data transfer rate. The receiver and transmitter have separate clock configuration.

HDLC/SDLC

High Level Data Link Control (HDLC) is an international standard (ISO3309) based on SDLC (Synchronous Data Link Control), a protocol developed by IBM. This document uses HDLC to refer to both HDLC and SDLC.

Data is grouped into an information field of two or more bytes. The information field may be followed by an optional frame check sequence (FCS) such as CRC16 or CRC32. The FCS is calculated on the bits in the information field. The information field and FCS are framed with a non data pattern 01111110 (0x7e) called a flag. The collection of an opening flag, information field, FCS, and a closing flag is called a frame. A frame in progress can be aborted before the closing flag by sending a non data pattern called an abort, which is 7 or more consecutive ones. Aborted frames or frames with a FCS error should be ignored by the receiver.



An optional preamble may be sent before each frame. The preamble is useful for synchronizing a DPLL for clock recovery. The preamble pattern should be chosen to provide the maximum transitions for a given serial encoding standard. Refer to the DPLL section for details. Preambles are usually not used for applications with a separate data clock signal.

Leading bytes of the information field contain variable length address and control fields. The serial controller does not process the address or control fields, and treats the entire information field as data. Interpretation of the address and control fields is the responsibility of the application.

Data transparency is provided to distinguish between data and flag or abort patterns. This is accomplished with zero insertion and deletion. The controller automatically inserts a zero after any sequence of five consecutive ones in send data and automatically deletes a zero after any sequence of five consecutive ones in receive data. Zero insertion and deletion is only applied to the information field and FCS.

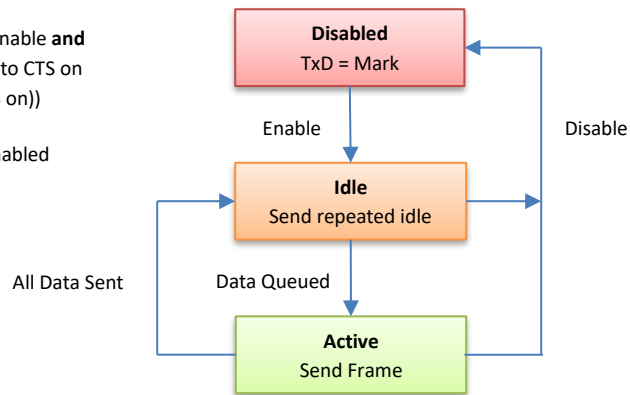
HDLC may use separate data clock signals or can recover data clocks from a data signal using DPLL (digital phase locked loop) clock recovery. There is one clock cycle per bit.

TRANSMIT

The HDLC transmitter has three states: disabled, idle and active. The transmitter starts in the disabled state with the transmit data signal set to a constant mark. When software enables the transmitter with a bit in a control register the transmitter becomes idle. An idle transmitter sends a user configurable idle pattern, usually all ones or repeated flags. When software provides data to send, the transmitter becomes active and sends a frame containing the data. When the frame completes, the transmitter becomes idle. Software can disable the transmitter at any time using control bits in a register.

Enable = Software Enable **and**
(Auto CTS off **or** (Auto CTS on
option **and** CTS on))

Disable = not Enabled



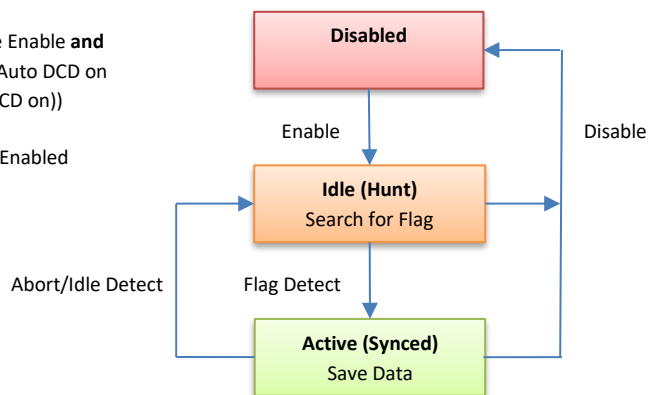
HDLC Transmit State Diagram

RECEIVE

The HDLC receiver has three states: disabled, idle (hunt), and active (syncd). The receiver start in the disabled state. When software enables the receiver with a bit in a control register, the receiver becomes idle and starts hunting for an opening flag. When a flag is detected, the receiver is active. An active receiver stores data between flags. When an abort sequence is detected, the receiver becomes idle. Software can disable the receiver at any time using control bits in a register.

Enable = Software Enable **and**
(Auto DCD off **or** (Auto DCD on
option **and** DCD on))

Disable = not Enabled

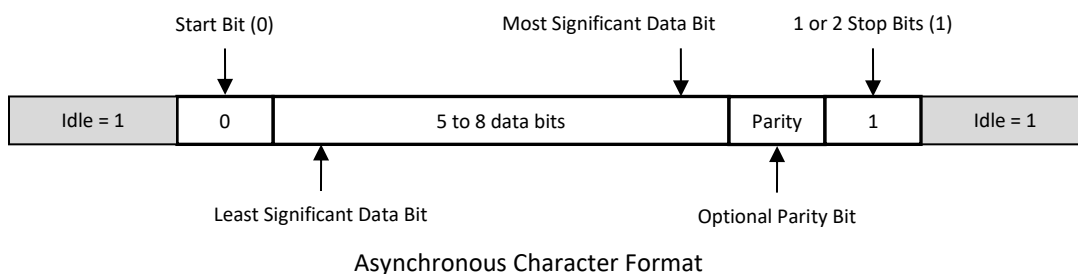


HDLC Receive State Diagram

ASYNCHRONOUS

Asynchronous communication frames each character with a single start bit and one or two stop bits. Data length is configurable for 5 to 8 data bits per character. An optional parity bit (odd or even) is appended to the data. The idle line state is a logical 1. The start bit is a logical 0. Stop bits are a logical 1. Data is transmitted least significant bit first followed by the optional parity bit. The total character size is the combination of the start bit, data bits, optional parity bit, and stop bits. The total character size range is 7 to 12 bits. The number of data bits, stop bits, and use of parity must be configured in advance to match the settings of a remote station.

Data clocks are generated internally. The data rate must be chosen in advance to match that of a remote station. The receive clock runs at 8 or 16 times the selected data rate. This clock is used to sample the receive data line. The start bit is detected as the falling edge from the idle condition or the stop bits of the preceding character (1 to 0).



ISOCHRONOUS

Isochronous is identical to asynchronous as described in the previous section, except a separate physical clock signal is used with the same frequency as the data rate (1x clock). The asynchronous sample code provided by the SDK had a flag that can be set to use isochronous instead of asynchronous. When set, external clock signals are used instead of internal timing.

RAW SYNCHRONOUS

Raw synchronous operation performs no framing or synchronization. Data is sent bit for bit as supplied by the application. Data is received bit for bit as seen on the receive data signal. Because there is no framing or synchronization, received data may appear bit shifted from the transmit data. This is due to the receiver starting to assemble receive bytes in the middle of a transmit byte. The application must detect the shift as demonstrated in the raw sample code.

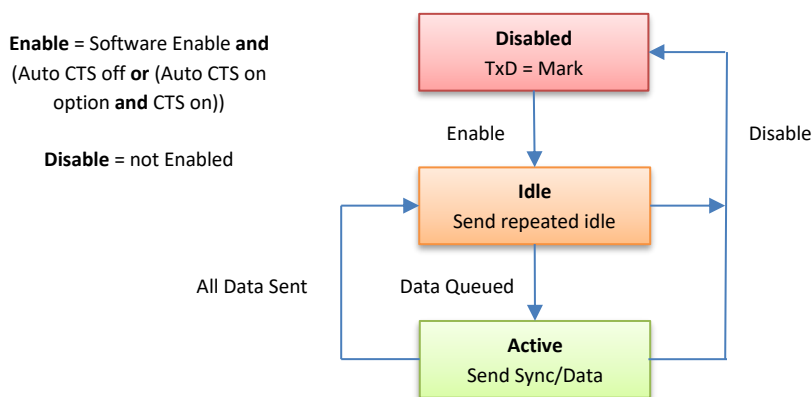
Raw synchronous may use separate data clock signals or can recover data clocks from a data signal using DPLL (digital phase locked loop) clock recovery. There is one clock cycle per bit.

TRANSMIT

There are three transmit states: disabled, idle and active. The transmitter starts disabled. A disabled transmitter holds the transmit data signal at constant one. When software enables the transmitter, it becomes idle. An idle transmitter repeatedly sends an application configured idle pattern, one to two bytes in length. When software writes data to the API the transmitter becomes active and sends data. When all data is sent the transmitter becomes idle.

When an application sends data via the API (write) the transmitter is automatically enabled.

Transmit enable is a combination of software transmit enable and the optional auto CTS feature. When auto CTS is used, CTS must be on to enable transmit. When auto CTS is not used, transmit enable depends only on the software transmit enable.



Raw Transmit State Diagram

RECEIVE

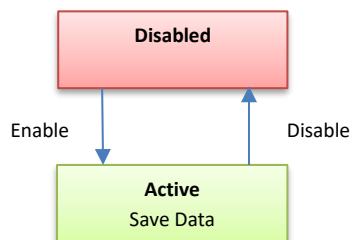
There are two receive states: disabled and active. The receiver starts disabled. When software enables the receiver, it becomes active. An active receiver stores received data. Data is not byte aligned and may appear bit shifted from sent data. The receiver remains active until disabled by software or DCD off when auto DCD used.

Receive enable is a combination of software receive enable and the optional auto DCD feature. When auto DCD is used, DCD must be on to enable receive. When auto DCD is not used, receive enable depends only on the software receive enable.

The application determines data block boundaries based on application specific requirements, possibly using start of block or end of block characters.

Enable = Software Enable **and**
(Auto DCD off **or** (Auto DCD on
option **and** DCD on))

Disable = not Enabled

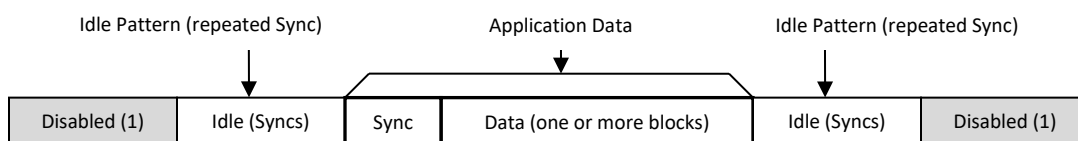


Raw Receive State Diagram

MONOSYNC AND BISYNC

Monosync and Bisync are byte oriented synchronous protocols. A synchronization (sync) pattern tells a receiver to start saving data. The sync pattern is configurable to an arbitrary value from one (monosync) to two (bisync) bytes in length. The format of data following the sync pattern is application defined. It is the responsibility of the application to determine data block boundaries and command the receiver to search for the next sync pattern when appropriate.

Monosync/Bisync may use separate data clock signals or can recover data clocks from a data signal using DPLL (digital phase locked loop) clock recovery. There is one clock cycle per bit.



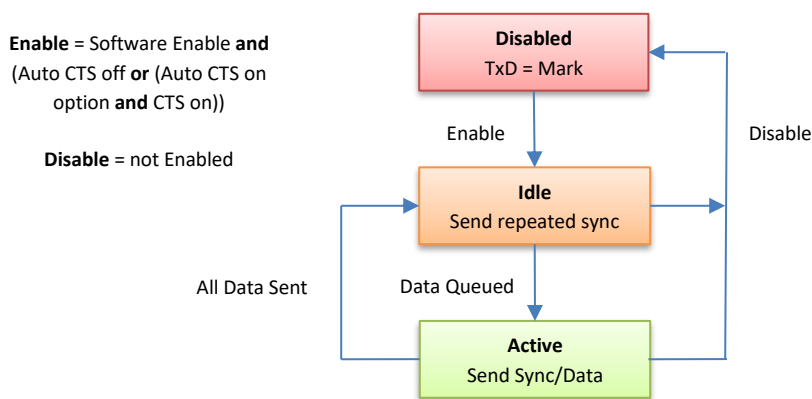
Monosync/Bisync Format

TRANSMIT

There are three transmit states: disabled, idle and active. The transmitter starts disabled. A disabled transmitter holds the transmit data signal at constant one. When software enables the transmitter, it becomes idle. An idle transmitter repeatedly sends an application configured sync pattern, one to two bytes in length. When software writes data to the API the transmitter becomes active and sends data. When all data is sent the transmitter becomes idle.

The transmitter does not automatically add a leading sync pattern to send data. The application must add the sync pattern to data supplied to the API. When an application sends data via the API (write) the transmitter is automatically enabled.

Transmit enable is a combination of software transmit enable and the optional auto CTS feature. When auto CTS is used, CTS must be on to enable transmit. When auto CTS is not used, transmit enable depends only on the software transmit enable.



Monosync/Bisync Transmit State Diagram

RECEIVE

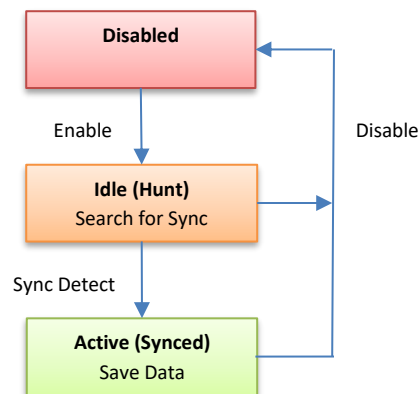
There are three receive states: disabled, idle and active. The receiver starts disabled. When software enables the receiver, it becomes idle and searches (hunts) for a sync pattern. When a sync pattern is detected, the receiver becomes active (synced). An active receiver stores received data. Data is byte aligned to the sync pattern. The receiver remains active until disabled by software or DCD off when auto DCD used. Applications force an active receiver to hunt mode by disabling then re-enabling it.

Receive enable is a combination of software receive enable and the optional auto DCD feature. When auto DCD is used, DCD must be on to enable receive. When auto DCD is not used, receive enable depends only on the software receive enable.

The application determines data block boundaries based on application specific requirements, possibly using start of block or end of block characters. Typically, an application enables the receiver and processes received data until the end of block condition is detected. The application then forces the receiver to idle/hunt.

Enable = Software Enable **and**
(Auto DCD off **or** (Auto DCD on
option **and** DCD on))

Disable = not Enabled



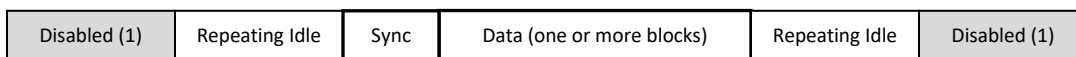
Monosync/Bisync Receive State Diagram

EXTENDED SYNC (XSYNC)

Extended sync (XSync) is a byte oriented synchronous protocol. A synchronization (sync) pattern tells a receiver to start saving data. The sync pattern is configurable to an arbitrary value from one to four bytes in length. The format of data following the sync pattern is application defined. It is the responsibility of the application to determine data block boundaries and command the receiver to search for the next sync pattern when appropriate.

XSync with a one byte sync pattern is known as monosync. XSync with a two byte sync pattern is known as bisync. New monosync and bisync applications should use the XSync protocol. The separate monosync and bisync protocols described in this document are maintained for compatibility with legacy software.

XSync may use separate data clock signals or can recover data clocks from a data signal using DPLL (digital phase locked loop) clock recovery. There is one clock cycle per bit.



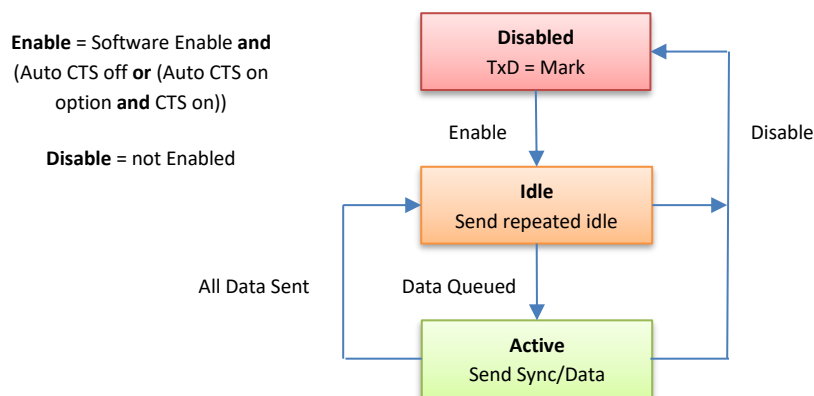
Xsync Format

TRANSMIT

There are three transmit states: disabled, idle and active. The transmitter starts disabled. A disabled transmitter holds the transmit data signal at constant one. When software enables the transmitter, it becomes idle. An idle transmitter repeatedly sends an application configured idle pattern, one to two bytes in length. When software writes data to the API the transmitter becomes active and sends data. When all data is sent the transmitter becomes idle.

The transmitter does not automatically add a leading sync pattern to send data. The application must add the sync pattern to data supplied to the API. When an application sends data via the API (write) the transmitter is automatically enabled.

Transmit enable is a combination of software transmit enable and the optional auto CTS feature. When auto CTS is used, CTS must be on to enable transmit. When auto CTS is not used, transmit enable depends only on the software transmit enable.



XSync Transmit State Diagram

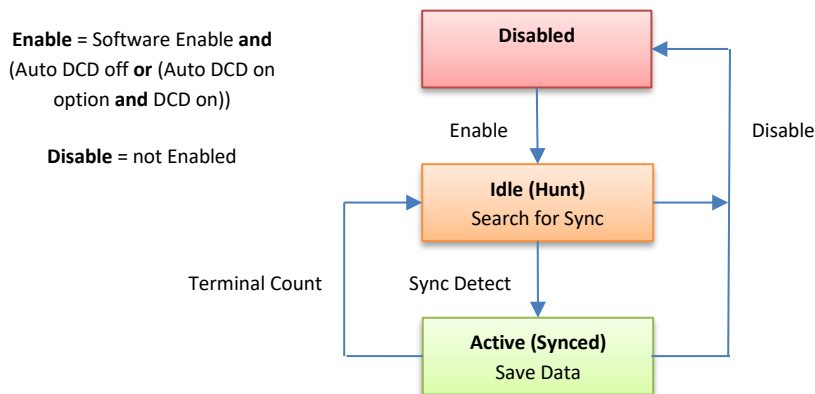
RECEIVE

There are three receive states: disabled, idle and active. The receiver starts disabled. When software enables the receiver, it becomes idle and searches (hunts) for a sync pattern. When a sync pattern is detected, the receiver becomes active (synced). An active receiver stores received data. Data is byte aligned to the sync pattern. The receiver remains active until disabled by software or DCD off when auto DCD used. Applications force an active receiver to hunt mode by disabling then re-enabling it.

Receive enable is a combination of software receive enable and the optional auto DCD feature. When auto DCD is used, DCD must be on to enable receive. When auto DCD is not used, receive enable depends only on the software receive enable.

The application determines data block boundaries based on application specific requirements, possibly using start of block or end of block characters. Typically, an application enables the receiver and processes received data until the end of block condition is detected. The application then forces the receiver to idle/hunt.

The optional terminal count feature forces an active receiver to Idle when the specified number of bytes have been received. Use this feature when data blocks are fixed length.



XSync Receive State Diagram

TIME DIVISION MULTIPLEXING (TDM)

Note: TDM is only supported on SyncLink PCIe and USB devices shipped February 2016 and later. Contact Microgate for information on updating these listed devices to support TDM if purchased before that date.

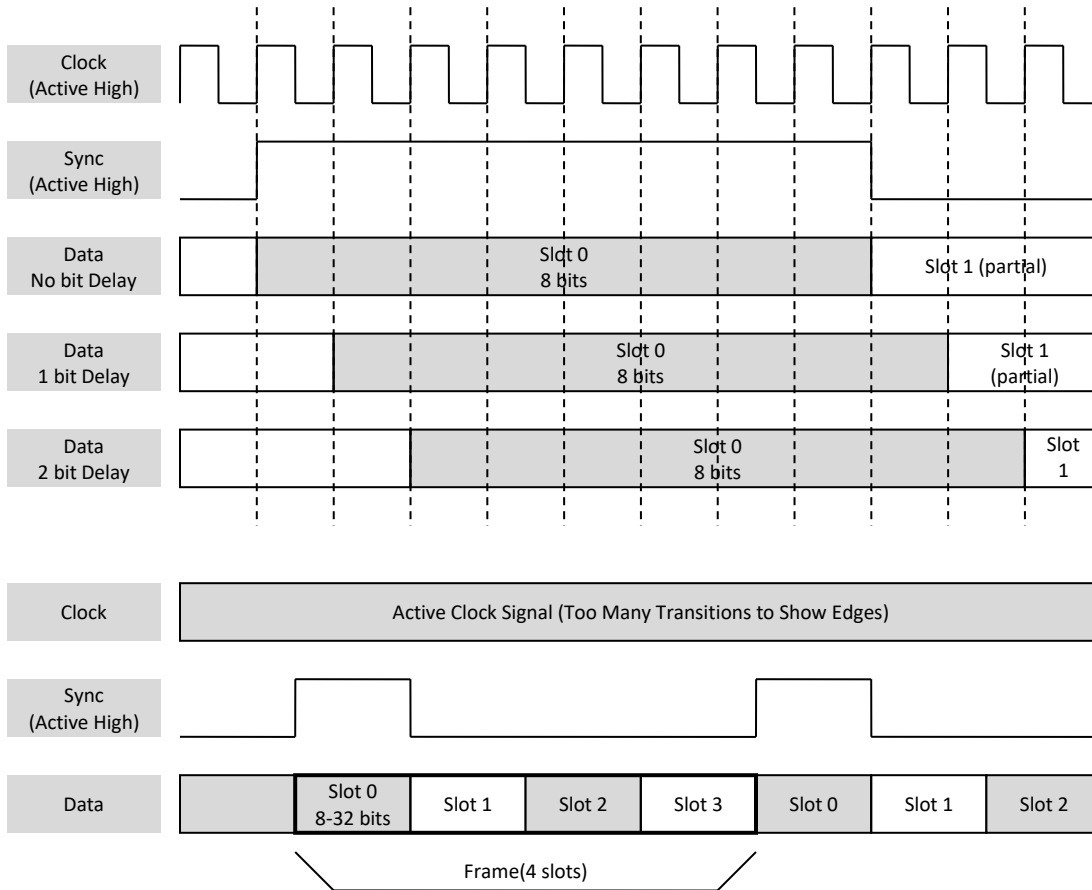
Time Division Multiplexing divides a serial signal by time into two or more **slots**. A **frame** is an ordered set of slots. Each slot represents a communication channel or data sample. The TDM implementation described here is compatible with the TDM mode of the multichannel audio serial port (McASP) of Texas Instruments controllers.

A data signal carries slots and frames. A slot contains 8 to 32 bits of data, in increments of 4 bits. A frame contains 2 to 32, or 384 slots. Slots within a frame are always contiguous. Multiple frames may or may not be contiguous. Data signal polarity (meaning of high or low signal) is selectable. The serial bit order is selectable, least significant bit (LSB) first or most significant bit (MSB) first. For example, a 12 bit slot with hexadecimal value 123 is sent as (first to last) 1100_0100_1000 for LSB first or 0001_0010_0011 for MSB first.

A clock signal provides timing information for the sync and data signals. Each clock cycle equals a single data bit. Selectable clock polarity determines which clock edge (rising or falling) changes output signals and which samples input signals.

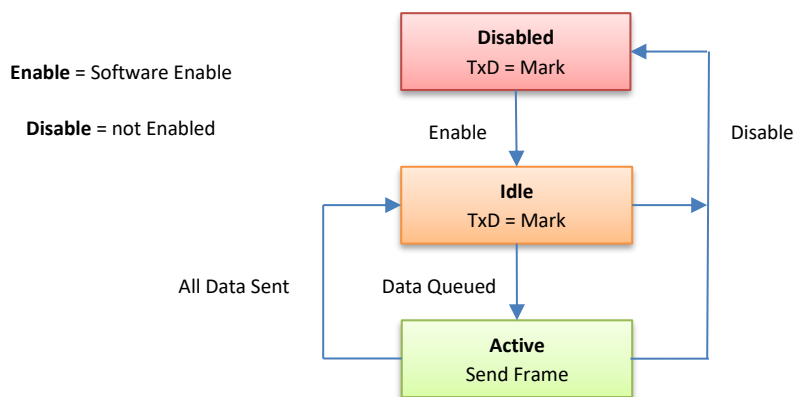
A sync signal identifies the start of a frame. The sync signal is one bit or one slot in length. The first bit of a frame occurs with the start of the sync signal or may be delayed up to two clock cycles using the sync delay option. Sync signal polarity, active high or low, is selectable. Transmit sync length is selectable (bit or slot). The receiver automatically accepts either bit or slot length sync pulses.

The SyncLink GT4e, PCIe 2-Port and USB devices have an option to extend the sync signal length to cover the entire frame. This option is not part of the McASP TDM specification. When sending a frame with this option, the sync signal is asserted for the length of the frame. When receiving a frame with this option, the returned frame size is limited to the length of the sync signal or the configured frame size, whichever is smaller.



TRANSMIT

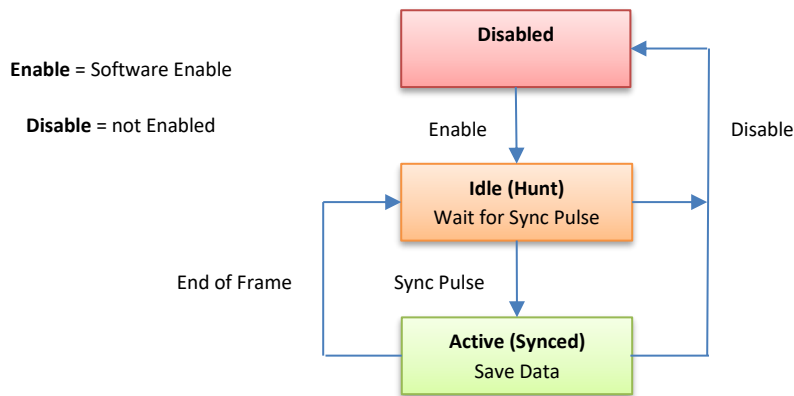
The TDM transmitter has three states: disabled, idle and active and starts disabled. When enabled, the transmitter becomes idle. When supplied data, the transmitter becomes active. When all data is sent, the transmitter becomes idle. The transmitter may be disabled at any time, discarding unsent data.



TDM Transmitter State Diagram

RECEIVE

The TDM receiver has three states: disabled, idle (hunt) and active (syncd), and starts disabled. When enabled, the receiver becomes idle. When a sync pulse is detected, the receiver is active. An active receiver stores data until a frame completes. When a frame completes, the receiver becomes idle. The receiver may be disabled at any time.



TDM Receive State Diagram

TDM SIGNAL MAPPING	
TDM Signal	Physical Serial Signal
Transmit Data (output)	TxD
Transmit Sync (output)	RTS
Transmit Clock (output)	AUXCLK
Receive Data (input)	RxD
Receive Sync (input)	DCD
Receive Clock (input)	RxC

Note: The PCIe 4-Port adapter has only data and clock signals to achieve higher port density in a small form factor. This prevents using TDM, which requires RTS and DCD. To use TDM, use an adapter which has a full set of control and status signals: SyncLink PCIe 2-Port, SyncLink USB, SyncLink GT4e.

ENCODING

Serial encoding converts a logical one or zero into a coded signal used on the physical connection between end points. Send data is encoded and receive data is decoded. Send and receive must use the same encoding. The table below describes each encoding standard.

The NRZ family (NRZ, NRZB, NRZI-space, NRZI-mark) has zero or one signal transitions per bit located at the start of the bit cell. The receiver samples the data signal at the center of the bit cell. NRZ type encoding is usually used with synchronous protocols that have a separate data clock signal. NRZ has fewer transitions per bit than biphas encoding which allows higher data rates for a bandwidth limited physical connection.

Note: NRZI (no -space or -mark) is often used to mean NRZI-space.

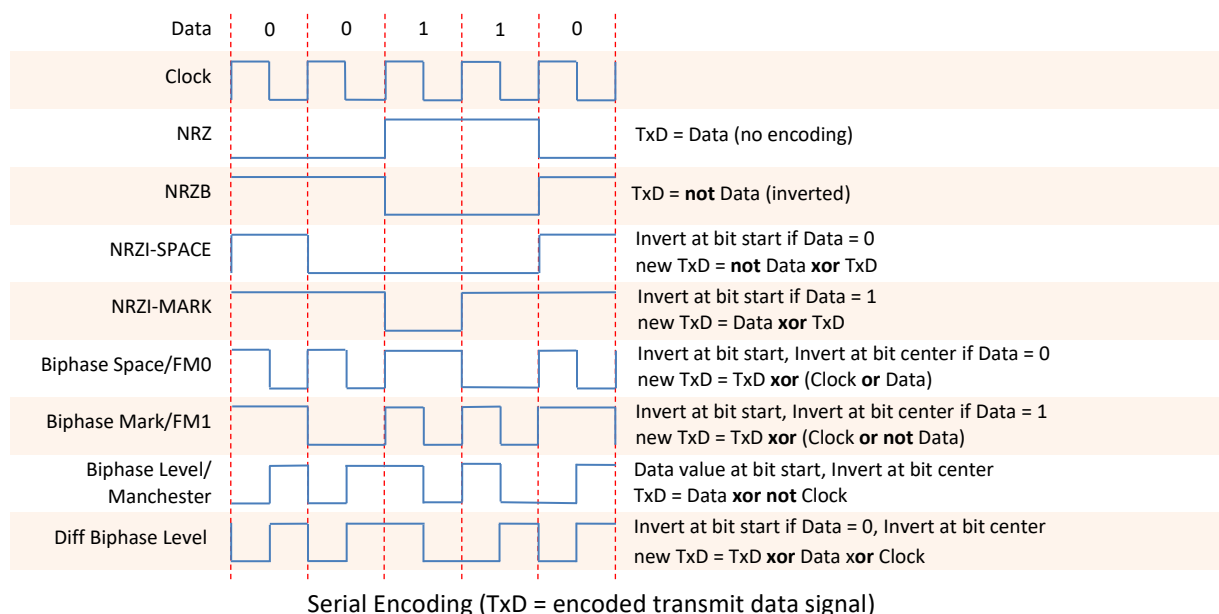
The biphas family has one or two transitions per bit located at the beginning and center of the bit cell. The receiver samples the data signal at $\frac{1}{4}$ and $\frac{3}{4}$ of the bit cell length. Biphas encoding is usually used with DPLL clock recovery as it guarantees at least one transition per bit cell to keep the recovered clock synchronized.

NRZ and NRZB depend only on the current Data value.

Biphase-Level (Manchester) depends on the current Data and Clock values. Data 0 is encoded as a 0 to 1 transition at bit center. Data 1 is encoded as a 1 to 0 transition at bit center.

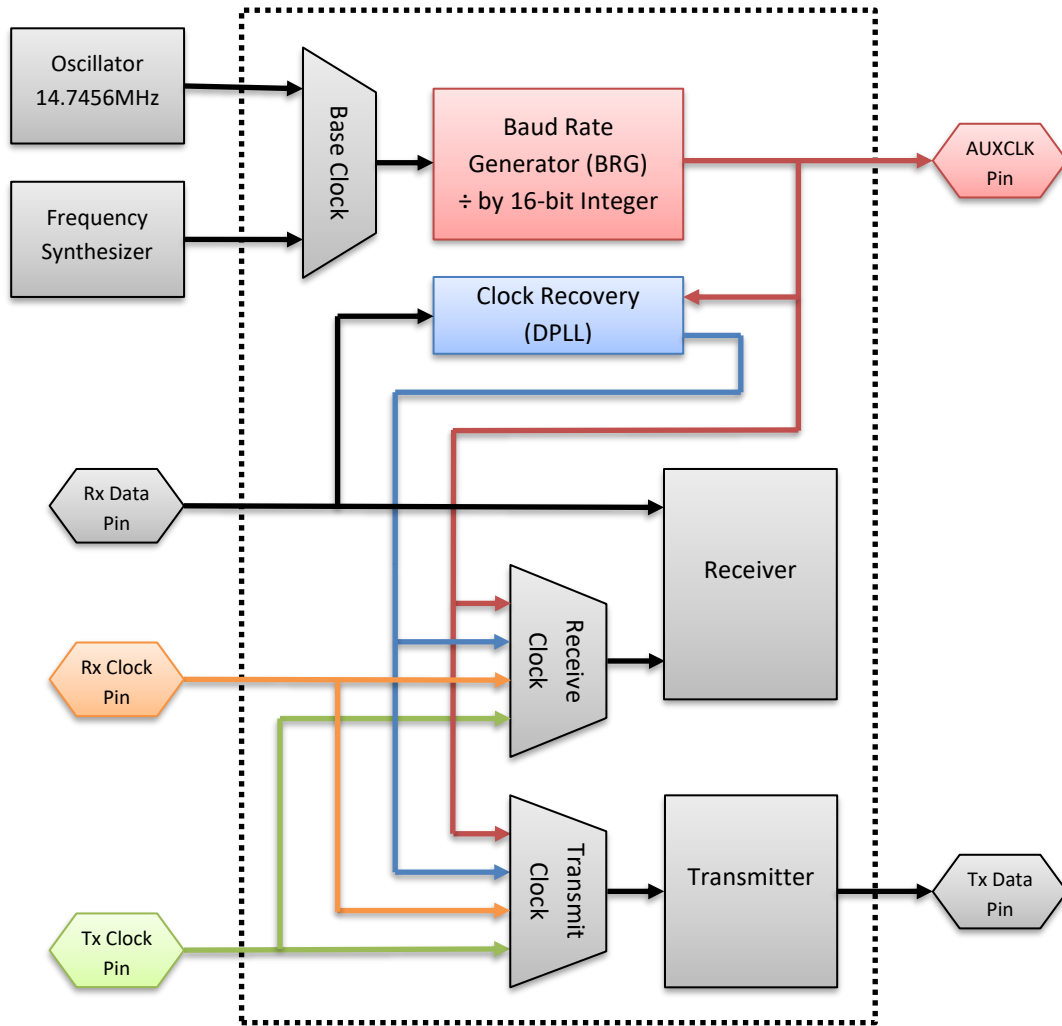
NRZI-Space, NRZI-Mark depend on the previous TxD value and the current Data value. Data is encoded as the absence or presence of a transition at the bit start.

Biphase-Space (FM0), Biphase-Mark (FM1) and Differential Biphase-Level depend on the previous TxD value and the current Data and Clock values.



SERIAL CLOCKS

Serial communication requires timing information to identify when a data signal changes (transmit) and is sampled (receive). Timing information may be carried by dedicated clock signals (TxC, RxC, AUXCLK), recovered from a data signal (DPLL) or generated internally (BRG). The clocking configuration depends on application requirements.



SyncLink Clocking Diagram

BASE CLOCK AND FREQUENCY SYNTHESIZER

All ports on a SyncLink device share a **base clock**. The default base clock is a fixed frequency 14.7456MHz oscillator. A programmable frequency synthesizer may be selected as the base clock and configured to other rates up to 80MHz. Sample code demonstrating frequency synthesizer use is included with the SDK. The synthesizer is configured from a table of pre-calculated rates. Contact Microgate support for rates not in the sample code tables.

BAUD RATE GENERATOR

Each port has a baud rate generator (BRG). A base clock is divided by 1 plus a 16-bit integer (0 to 65535) to generate the BRG clock.

$$f_{BRG} = \frac{f_{BASE}}{divisor + 1} \quad divisor = \left(\frac{f_{BASE}}{f_{BRG}} \right) - 1$$

The BRG clock may be used internally as a data clock, a reference clock for clock recovery (DPLL) or asynchronous receive data sampling. The BRG clock may be output on the AUXCLK (DTE Transmit Clock) pin to provide timing information to other devices.

To generate a clock rate exactly, base clock/BRG must be an integer. If base clock/BRG is not an integer, it is rounded up resulting in a BRG rate lower than specified. The default 14.7456MHz base clock supports exact generation of standard data rates like 9600, 38400, 115200, etc. When designing a serial clock configuration, verify BRG rates can be generated exactly. Use the frequency synthesizer as base clock if the default base clock does not allow generation of the desired rate.

Examples:

- ✓ 9600bps: base clock (oscillator)/BRG = 14.7456MHz/9600 = 1536 (integer, BRG rate is exactly 9600)
- ✗ 1Mbps: base clock (oscillator)/BRG = 14.7456MHz/1MHz = 14.7456 (non-integer, BRG rate is 983040)
- ✓ 1Mbps: base clock (synthesizer@20MHz)/BRG = 20MHz/1MHz = 20 (integer, BRG rate is exactly 1Mbps)

CLOCK RECOVERY

A digital phase locked loop (DPLL) recovers a data clock from a data signal, removing the need for a dedicated clock signal. The recovered clock is aligned (synced) to transitions in the data signal. Transitions are guaranteed by data encoding and protocol selection. The data rate must be known in advance and specified in the **clock speed** (BRG) setting.

Technical Detail

When using clock recovery, the BRG generates two related clocks:

1. reference clock for DPLL, rate = clock speed x16 (preferred/default) or x8
2. data clock for transmitter, rate = clock speed

Typical Clock Recovery Configuration

Receive clock source = DPLL (recovered clock)

Transmit clock source = BRG (generated clock)

clock speed (BRG) = desired data rate

The default reference clock is clock speed x16. If an x16 reference clock is above 80MHz or does not evenly divide the base clock, the application may configure a less accurate x8 reference clock. The reference clock must not deviate more than +/- 1% of nominal value.

Encoding

Clock recovery typically uses biphas encoding (FM or Manchester) to guarantee data transitions every bit. Clock recovery may use NRZI encoding with SDLC/HDLC protocol which guarantees a transition every 6 bits. Other protocol and encoding combinations do not create data transitions necessary for clock recovery.

Preamble

When a data signal is turned off (tristate/hi-Z) between frames, a transmit preamble before each SDLC/HDLC frame is recommended to allow the DPLL to synchronize before the start of each frame. Below is a list of suggested preamble patterns for different serial encodings:

Encoding	Preamble Pattern
NRZI	zeros
BIPHASE_MARK (FM1)	zeros
BIPHASE_SPACE (FM0)	ones
BIPHASE_LEVEL (Manchester)	alternating 01 or flags

ASYNCHRONOUS PROTOCOL

Asynchronous protocol does not use dedicated clock signals. The receive data signal is sampled at x16 (preferred/default) or x8 the data rate to decode the data signal. The data rate must be known in advance and specified in the asynchronous **data rate** setting.

Technical Detail

When using asynchronous protocol, the BRG generates two related clocks:

1. sampling clock for receiver, rate = data rate x16 (preferred/default) or x8
2. data clock for transmitter, rate = data rate

When asynchronous protocol is selected, receive and transmit clock source are BRG and clock speed (BRG rate) is derived from the asynchronous **data rate** setting. Application clock source and clock speed (BRG) settings are ignored. The sampling clock defaults to x16 data rate. x8 is used if x16 can't be generated exactly or exceeds 80MHz. Sampling clock must not deviate more than +/- 1% of nominal value.

ISOCRONOUS PROTOCOL

The isochronous protocol uses the character framing (start/data/parity/stop bits) of the asynchronous protocol with dedicated clock signals for timing. To use isochronous protocol with the SyncLink API, select the asynchronous protocol with data rate (async) set to 0. Application specified clock source and clock speed (BRG) settings are used.

Typical Isochronous Configuration

protocol = asynchronous

data rate (async) = 0 (use application specified clock source and clock speed settings)

receive clock source = RxC pin (receive clock input from DCE)

transmit clock source = TxC pin (transmit clock input from DCE)

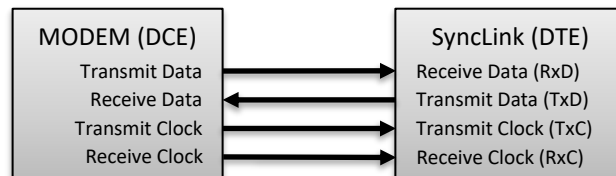
clock speed (BRG) = 0 (disabled)

Other aspects of the isochronous protocol are identical to the asynchronous protocol, including configuration of data, parity and stop bits. API read function behavior is identical to asynchronous.

CLOCK EXAMPLES

A list of common clock configurations follows. Refer to the Serial API Programming section for detailed settings syntax. Examine sample code in the SDK for more detail.

SYNCHRONOUS, DCE CLOCKS



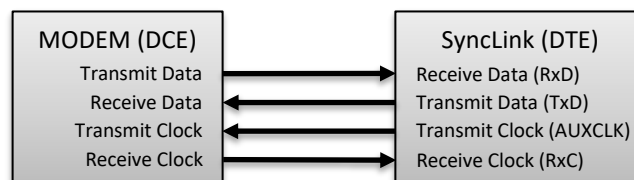
protocol = HDLC, Monosync, Bisync or Raw

receive clock source = RxC pin (receive clock input from DCE)

transmit clock source = TxC pin (transmit clock input from DCE)

Base clock/BRG/DPLL are unused, clock speed (BRG) is not specified by application.

SYNCHRONOUS, DCE RECEIVE CLOCK, DTE TRANSMIT CLOCK



protocol = HDLC, Monosync, Bisync or Raw

receive clock source = RxC pin (receive clock input from DCE)

transmit clock source = BRG (output on AUXCLK pin)

clock speed (BRG) = 9600bps

✓ base clock (oscillator) / BRG = $14745600/9600 = 1536$ (integer, BRG rate is exact)

base clock source = oscillator at 14.7456MHz (default)

clock speed (BRG) = 1Mbps

✗ base clock (oscillator) / BRG = $14745600/1000000 = 14.7456$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $20000000/1000000 = 20$ (integer, BRG rate is exact)

base clock source = frequency synthesizer at 20MHz

SYNCHRONOUS, CLOCK RECOVERY



protocol = HDLC, Monosync, Bisync or Raw

encoding = biphas level (Manchester)

transmit preamble = 8 bits long, alternating 01 pattern

receive clock source = DPLL (recovered clock)

transmit clock source = BRG (generated clock)

clock speed (BRG) = 9600bps

BRG generates DPLL reference clock at x16 data rate = $9600 * 16 = 153600$

✓ base clock (oscillator) / BRG = $14745600/153600 = 96$ (integer, BRG rate is exact)

base clock source = 14.7456MHz fixed frequency oscillator (default)

clock speed (BRG) = 1Mbps

BRG generates DPLL reference clock at x16 data rate = $1\text{MHz} * 16 = 16\text{MHz}$

✗ base clock (oscillator) / BRG = $14.7456\text{MHz}/16\text{MHz} = 0.9216$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $32\text{MHz}/16\text{MHz} = 2$ (integer, BRG rate is exact)

base clock source = frequency synthesizer at 32MHz

clock speed (BRG) = 10Mbps

✗ BRG generates DPLL reference clock at x16 data rate = $10\text{MHz} * 16 = 160\text{MHz}$ (exceeds 80MHz, use x8)

✓ BRG generates DPLL reference clock at x8 data rate = $10\text{MHz} * 8 = 80\text{MHz}$

✗ base clock (oscillator) / BRG = $14.7456\text{MHz}/80\text{MHz} = 0.18432$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $80\text{MHz}/80\text{MHz} = 1$ (integer, BRG rate is exact)

base clock source = frequency synthesizer at 80MHz

ASYNCHRONOUS



When asynchronous protocol is selected, receive and transmit clock source are BRG and the BRG rate is derived from the asynchronous data rate setting. Application specified clock source and clock speed (BRG) are ignored. The receive sampling clock defaults to data rate x16. If x16 can't be generated exactly or exceeds 80MHz, x8 is used.

data rate (async) = 9600bps

BRG generates sampling clock at x16 data rate = $9600 * 16 = 153600$

✓ base clock (oscillator) / BRG = $14745600/153600 = 96$ (integer, BRG rate is exact)

data rate (async) = 1Mbps

BRG generates sampling clock at x16 data rate = $1\text{MHz} * 16 = 16\text{MHz}$

✗ base clock (oscillator) / BRG = $14.7456\text{MHz}/16\text{MHz} = 0.9216$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $32\text{MHz}/16\text{MHz} = 2$ (integer, BRG rate is exact)

data rate (async) = 10Mbps

✗ BRG generates sampling clock at x16 data rate = $10\text{MHz} * 16 = 160\text{MHz}$ (exceeds 80MHz, use x8)

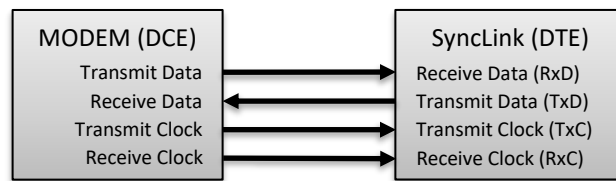
✓ BRG generates sampling clock at x8 data rate = $1\text{MHz} * 8 = 80\text{MHz}$

✗ base clock (oscillator) / BRG = $14.7456\text{MHz}/80\text{MHz} = 0.18432$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $80\text{MHz}/80\text{MHz} = 1$ (integer, BRG rate is exact)

base clock source = frequency synthesizer at 80MHz

ISOCRONOUS, DCE CLOCKS



protocol = asynchronous

data rate (async) = 0, use application specified clock sources and clock speed (BRG)

receive clock source = RxC pin (receive clock input from DCE)

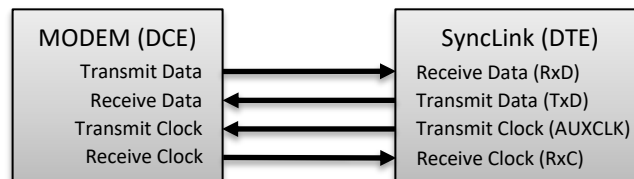
transmit clock source = TxC pin (transmit clock input from DCE)

clock speed (BRG) = 0 (disabled)

Notes

For the special case of asynchronous protocol with data rate (async) set to 0 (isochronous), the application specified clock source and clock speed (BRG) are used. In this example, the application sets the clock sources to DCE provided clocks and disables the BRG.

ISOCRONOUS, DCE RECEIVE CLOCK, DTE TRANSMIT CLOCK



protocol = asynchronous

data rate (async) = 0, use application specified clock sources and clock speed (BRG)

receive clock source = RxC pin (receive clock input from DCE)

transmit clock source = BRG (transmit clock output on SyncLink AUXCLK pin)

clock speed (BRG) = 9600bps

✓ base clock (oscillator) / BRG = $14745600 / 9600 = 1536$ (integer, BRG rate is exact)

base clock source = oscillator at 14.7456MHz (default)

clock speed (BRG) = 1Mbps

✗ base clock (oscillator) / BRG = $14745600 / 1000000 = 14.7456$ (non-integer, use synthesizer)

✓ base clock (synthesizer) / BRG = $20000000 / 1000000 = 20$ (integer, BRG rate is exact)

base clock source = frequency synthesizer at 20MHz

WINDOWS COMMUNICATION API (COM PORT MODE)

SyncLink serial devices support the Windows Communication API used with standard, asynchronous serial ports (COM ports). This allows SyncLink devices to work with programs that operate with standard COM ports using the Windows Communication API. Access the Windows Communication API using an alternate, API specific name for a SyncLink device as described in the following section. The Windows Communication API is described in the Windows Software Development Kit (SDK) available from Microsoft. Only asynchronous serial communications are available when using the Windows Communication API. HDLC or other synchronous protocols require the MicroGate Serial API.

Device Names

SyncLink devices have a name used with the MicroGate Serial API. Single port devices use MGHDLc x , where x is the device instance number (MGHDLc1, MGHDLc2, etc). Devices with multiple ports use MGMP x P y , where x is the device instance number and y is the port number. The MicroGate Serial API name is displayed in the Windows Device Manager.

SyncLink devices have an alternate name for use with the Windows Communication API. Single port devices use MGC x , where x is the device instance number (MGC1, MGC2, etc). Devices with multiple ports use MC x y , where x is the device instance number and y is the port number. MC11 = device 1, port 1. MC12 = device 1, port 2. The Windows Communication API device name does not appear in the Windows device manager, as it is an API specific alias of the actual device.

Both device names are created when a SyncLink device and drivers are installed. Normally, only one of the two device names should be in use, with the open name controlling the hardware. Both names may be used at the same time to auto-dial an SDLC link using asynchronous AT modem commands. When both names are open, the Windows Communication API name controls the hardware until DTR is negated in the Windows Communication API. This behavior allows an SDLC program to operate using the MicroGate Serial API, with an independent Windows Communication API program sending asynchronous AT commands to setup the link. When setup completes, the Windows Communication API program drops DTR or closes the port, returning control of the hardware to the SDLC program.

Sample code demonstrating the use of a SyncLink device with the Windows Communication API is located in the `c:\samples\commapi` directory of the MicroGate Serial API development package.